FemboyEngine2D — Полное справочное руководство

Архитектура, рендеринг, ECS, холсты анимации/ИИ с контрактами циклов блендинга и PIE-передачи, глобальный аудит, DOD-оптимизация и продакшн-дистрибуция

Version 0.1.0-alpha.1 (Alpha) - 2026-10-31 - Russian

Содержание

- 1. Область действия, соответствие стандарту и правила движка
- 2. Сборка, цели и флаги компилятора
- 3. Базовые типы, FORCEINLINE и семейство приведений
- 4. FrameArena: единственный кадровый аллокатор
- 5. JobSystem и потоковая диагностика
- 6. ECS, планирование и снапшоты
- 7. RHI, элементы отрисовки и статистика кадра
- 8. Материалы: кук графов, кэш пайплайнов и продакшн-путь
- 9. Физика: Box2D v3, суставы, ткань и SdfCollisionSystem
- 10. NetRollbackSession: детерминированный роллбэк
- 11. Аудио: микшер, DSP-граф, лимитер
- 12. PinkUI и WidgetCanvasSystem
- 13. Ядро доступности (BlindMode, DeafMode, FlashGuard)
- 14. Конвейер ассетов, VFS и локализация
- 15. Платформенные сервисы: Steam, Epic Games Store, GOG
- 16. Редактор: дашборд, шрифты, LogBoy, Scene Diagnostics
- 17. Пошагово: игра с пустой папки
- 18. Тесты, стилевой гейт и чек-лист релиза
- 19. Известные пробелы и рекомендуемая следующая работа
- 20. GOG Galaxy: провайдер, ABI шима и системы PostUpdate
- 21. Шрифтовой конвейер: fby_fontc, .fbyfont и стабильный буфер
- 22. Путь отрисовки ткани: от вершин Verlet до одного indirect-вызова
- 23. Атмосфера: погодный контекст, fBm-ветер и GPU-дождь с брызгами по SDF
- 24. ECS через композицию: 2D-шутер без наследования
- 25. Multi-Draw Indirect и кукинг текстур в продакшене
- 26. Структура репозитория, цели CMake и гейты
- 27. Контракт потоков: RHI, главный цикл ОС и TTS
- 28. 32-битный конвейер DrawBatch: конец потолка в 65 536 текстур
- 29. Атомарный хот-релоад шрифтов VFS и строгий контракт MountFonts
- 30. TTS: контракт главного потока и локализованный выбор голоса
- 31. Геймплей на состояниях: FemAnimGraph, ИИ StateTree и холст Animation Graph
- 32. Продвинутые AAA DOD-конвейеры: виртуальные текстуры, маски изменений компонентов, сетки релевантности и аудио-шины
- 33. Продакшн-дистрибуция: шифрование, сжатие LZMA и чек-лист релиза Steam/EGS/GOG

- 34. SystemGraph: контракт параллельного планирования, затем контракт циклов блендинга
- 35. Очистка Box2D при откате, затем PIE-передача анимации
- 36. Глобальный аудит: блокирующие находки и гейт
- 37. Профилирование кэш-миссов и векторизация к 8000+ FPS
- 38. Материалы в движке: кук, кэш и кадр
- 39. Приложение A - сгенерированный индекс символов
- 40. Приложение B - макросы и платформенные define

1. Область действия, соответствие стандарту и правила движка

Этот справочник описывает публичную поверхность FemboyEngine2D: все заголовки из Source/Engine и инфраструктуру редактора, с которой линкуется хост. Текст написан по дереву Release, поэтому описанный символ существует в поставляемых заголовках, а индекс в приложении генерируется из этих заголовков при каждом запуске.

Движок рассчитан на C++26 с GCC 15 или новее и Clang 19 или новее; релиз-кандидат под Windows собирается clang-cl со стандартной библиотекой MSVC. Четыре правила являются несущими, и каждый публичный API спроектирован вокруг них:

- Состояние симуляции — это простые trivially copyable данные. Ни виртуальных вызовов, ни владеющих указателей, ни std::string в компонентах, снапшотах и сетевых пакетах.
- Горячие пути не выделяют память. Кадровая память берётся из FrameArena, логи пишутся в фиксированные char-буферы, а колбэки — это указатель на функцию плюс void*, никогда std::function.
- Детерминизм — свойство данных, а не планировщика. Каждая подсистема отдаёт образ с контрольной суммой, и NetRollbackSession может воспроизвести только то, что воспроизводится байт в байт.
- Никакого RTTI в коде движка: dynamic_cast и typeid запрещены стилевым гейтом. Cast<> вместо этого обходит стабильные идентификаторы типов (см. раздел 3).

Стилевой гейт (Source/Tools/Dev/CheckStyle.py) проверяет правила именования и структуры: PascalCase имена файлов, Allman-скобки, обязательные фигурные скобки у управляющих конструкций, префикс In у параметров, префикс Scoped у локальных переменных, отсутствие комментариев в конце строки и кириллицы в исходниках движка.

2. Сборка, цели и флаги компилятора

Одно дерево CMake производит три вида артефактов: статическую библиотеку движка (FemboyEngine2D::Engine), библиотеку и приложение редактора (FemboyEngine2D::Editor) и офлайн-инструменты. Поставляемая игра линкует только движок; Dear ImGui, ImGuizmo, ImPlot и TOML-писатель не попадают в поставочные цели.

Опция CMake	По умолчанию	Действие
FBY_BUILD_EDITOR	ON	Собирает fby_editor, приложение редактора и плагины панелей.
FBY_BUILD_TESTS	OFF	Собирает набор ctest в Source/Tests.
FBY_BUILD_TOOLS	ON	FempakCooker, LocTool, SceneTool, ShaderCompiler, AotCompiler.
FBY_BUILD_PLATFORM_PROVIDERS	ON	Плагины провайдеров Steam и Epic Games Store.
FBY_RHI_BACKEND	auto	auto, vulkan, metal или null. auto выбирается по платформе.

```
cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=RelWithDebInfo \
      -DFBY_BUILD_TESTS=ON -DFBY_BUILD_EDITOR=ON
cmake --build build --target fby_editor
ctest --test-dir build --output-on-failure
python Source/Tools/Dev/CheckStyle.py
```

Обёртка компилятора (cmake/FbyCompiler.cmake) задаёт флаг C++26, набор предупреждений и переключатели санитайзеров. Не добавляйте флаги отдельным целям: цель, которой нужен другой диалект, не должна существовать.

3. Базовые типы, FORCEINLINE и семейство приведений

Definitions.hpp — контрактный заголовок движка. Он содержит макросы экспорта, макросы утверждений (FBY_ASSERT, FBY_VERIFY, FBY_UNREACHABLE), механизм стабильных TypeId, семейство приведений и макросы регистрации компонентов. Остальные заголовки включают его прямо или транзитивно.

```
#if defined(_MSC_VER)
#define FORCEINLINE __forceinline
#elif defined(__GNUC__) || defined(__clang__)
#define FORCEINLINE inline __attribute__((always_inline))
#else
#define FORCEINLINE inline
#endif
```

FORCEINLINE — единственное написание инлайна в публичных заголовках. Он применяется к аксессорам, горячей арифметике и шаблонным приведениям и никогда не заменяет измеренную оптимизацию. Существуют три семейства:

Символ	Контракт
FemCast<To>(value)	Непроверяемый static_cast со сворачиванием ссылок. Используется, когда конкретный тип доказан конструкцией (выход фабрики, тег хранилища). Нулевая цена в рантайме.
Cast<To>(object)	Проверяемое приведение. Апкаст — compile-time static_cast; даункаст сравнивает GetTypeId() объекта с StableTypeId<To>() и цепочкой родителей; несвязанные типы не компилируются. При несовпадении возвращается nullptr, исключений и RTTI нет.
CastComponent<To>(registry, entity)	ECS-написание Cast<>: EnTT проверяет тип компонента на этапе компиляции, а реестр проверяет сущность. Возвращает nullptr, если компонента у сущности нет.

```
// Downcast through the stable type table, not through RTTI.
Fby::TypeObject* base = factory.Create("Sprite");
if (Fby::Sprite* sprite = Fby::Cast<Fby::Sprite>(base))
{
    sprite->Animate(dt);
}

// Registration binds Sprite to TypeObject and derives its stable id.
FBY_REGISTER_TYPE(Fby::Sprite, Fby::TypeObject);
```

StableTypeId<T>() — consteval FNV-1a от зарегистрированного имени типа (или сигнатуры компилятора, если имя не закреплено). Смена зарегистрированного имени меняет id, поэтому имена компонентов входят в сериализованную схему, а Snapshot v4 хранит LayoutHash рядом с именем.

4. FrameArena: единственный кадровый аллокатор

FrameArena — это bump-указатель по памяти, которой владеет вызывающий код. Он сбрасывается в начале кадра и никогда не освобождает отдельные выделения. Системы, которым нужны временные массивы, берут его из SystemContext::frameArena; система, вызывающая operator new в кадровом пути, — дефект, а не стилистическое предпочтение.

```
Fby::FrameArena arena(storage);          // caller owns storage
arena.Reset();                           // once per frame
auto* items = arena.AllocateArray<MeshRenderItem>(count);
if (items == nullptr)
{
    // Arena exhausted: degrade, never allocate on the heap.
    return;
}
const std::size_t used = arena.Used();
const std::size_t capacity = arena.Capacity();
```

- Allocate() возвращает nullptr вместо исключения при исчерпании арены; вызывающий код обязан обработать отказ.
- AllocateArray<T>() проверяет std::is_trivially_copyable_v<T>: арена хранит только POD.
- Для типов с повышенным выравниванием нужно явно передать выравнивание; по умолчанию используется alignof(std::max_align_t).

5. JobSystem и потоковая диагностика

JobSystem владеет пулом std::jthread. Задача — это указатель на функцию, указатель на контекст и полуоткрытый диапазон, поэтому отправка задачи не выделяет память. ParallelFor делит диапазон на куски, позволяет каждому участнику брать следующий кусок из атомарного счётчика и всегда выполняет хотя бы один кусок на вызывающем потоке — так однопроцессорные машины остаются корректными, а повторный вход из воркера безопасным.

```
jobs.ParallelFor(entityCount, 4096, [&](std::uint32_t begin, std::uint32_t end)
```

```
{
    for (std::uint32_t i = begin; i < end; ++i)
    {
        Integrate(entities[i], dt);
    }
});
jobs.WaitIdle();

// Diagnostics: one row per thread, slot 0 is the main thread.
std::array<Fby::JobSystem::ThreadLoad, Fby::JobSystem::kMaxThreadSlots> loads{};
const std::uint32_t slots = jobs.SampleThreadLoad(loads);
```

ThreadLoad содержит busyNanoseconds (время внутри работы движка), taskCount и 16-байтное имя. Счётчики выровнены по кэш-линии на поток, поэтому два одновременно завершившихся воркера не конфликтуют. SceneDiagnosticsPanel превращает два замера в процент занятости каждого потока.

ThreadSlotCount() равен ThreadCount() + 1 и ограничен фиксированным массивом MCounters; устройство с более чем 64 воркерами сообщает первые 64 слота.

6. ECS, планирование и снапшоты

Реестр — это EnTT. Системы регистрируются в SystemGraph в одной из пяти фаз (PreUpdate, FixedUpdate, Update, PostUpdate, Render). SystemContext — единственный параметр системы; он содержит реестр, JobSystem, кадровую арену, рендерер, окно, VFS, AssetDatabase, аудио, физику, суставы, ткань, жидкость, стек камер, ядро доступности и атласы шрифтов.

```
Fby::Ecs::Schedule schedule;
schedule.Add(Fby::Ecs::Phase::FixedUpdate, "physics", &PhysicsStepSystem,
&physics);
schedule.Add(Fby::Ecs::Phase::Update, "movement", &MovementSystem, nullptr);
schedule.Add(Fby::Ecs::Phase::PostUpdate, "platform", &GogAchievementSystem,
&gog);

Fby::Ecs::SystemContext context{registry, jobs};
context.physics = &physics;
context.window = &window;
schedule.Run(Fby::Ecs::Phase::FixedUpdate, context);
```

Snapshot v4 — канонический образ реестра с контрольной суммой. Компоненты с рантайм-хэндлами (b2BodyId, b2ShapeId) пишутся рукописными кодеками; снапшот никогда не сериализует эти хэндлы, а восстановление пересоздаёт тела из полей компонента. Схема хранит Mirror-имя и LayoutHash каждого компонента, поэтому изменение поля отклоняется, а не читается молча неверно.

API	Контракт
GetOrAdd<T>(registry, entity)	Добавляет компонент при отсутствии и возвращает стабильную ссылку.
ecs::SnapshotWriter / SnapshotReader	Пишет и валидирует канонический образ;

	контрольная сумма — FNV-1a по payload.
Fby::Net::NetRollbackSession	Использует snapshot v4 и воспроизводит тики при опоздании ввода (раздел 10).

7. RHI, элементы отрисовки и статистика кадра

RHI — рукописный слой Vulkan/Metal. Кадры записываются в постоянные кольцевые буферы; спрайты — это 48-байтные POD; меши рисуются одним indirect-вызовом на материал/слой; linear-blend skinning — один compute-диспатч. FrameStats показывает, что реально отправил проход, и именно это рисует SceneDiagnosticsPanel.

Поле FrameStats	Значение
items / batches / drawCalls	Спрайт-элементы, батчи и отправленные вызовы.
meshItems / meshIndirectCalls	Количество MeshRenderItem и indirect-вызовов.
skinDispatches / skinRanges	Диспатчи и диапазоны compute-скиннинга.
uploadedBytes	Байты, загруженные в этом кадре.

Viewport редактора рендерит в offscreen-цель, а не в swapchain. EnsureViewportTarget() задаёт размер цели в физических пикселях; цель, заданная в точках, — обычная причина размытого Viewport на HiDPI-мониторе.

8. Материалы: кук графов, кэш пайплайнов и продакшн-путь

Авторинг материала и его компиляция находятся по разные стороны границы движка. `Render::MaterialGraph` — авторский DAG с фиксированными ёмкостями (128 узлов, 384 пина, 64 параметра), trivially copyable, хешируется без позиций в редакторе.
`Render::MaterialGraphCompiler` превращает прошедший валидацию граф в текст GLSL 450. `fby_matc` запускает этот компилятор и glslang на этапе сборки и пишет payload `.femmat`, поэтому поставленная игра не линкует glslang и не разбирает GLSL. Полное обоснование — в [ADR 0002](../../adr/0002-materials-in-engine.md).

Компонент	Заголовок	Контракт
`Render::MaterialGraph`	`Engine/RHI/MaterialGraph.hpp`	Авторский DAG; `Hash()` — идентификатор пайплайна, позиции в UI не входят
`MaterialGraphCompiler`	`Engine/RHI/MaterialGraphCompiler.hpp`	DAG в GLSL 450 и флаги контракта шейдера
`EncodeMaterialGraph` / `DecodeMaterialGraph`	`Engine/RHI/MaterialAsset.hpp`	Wire-формат `.fgraph`: явные little-endian поля, без memcpy структуры
`EncodeCookedMaterial` /	`Engine/RHI/`	Payload `.femmat`: 32-

`DecodeCookedMaterial`	MaterialAsset.hpp`	байтный заголовок, SPIR-V вершинного и фрагментного этапов, декодирование без копирования
`Render::MaterialPipelineCache`	`Engine/RHI/MaterialPipelineCache.hpp`	128 пайплайнов по хешу графа, с подсчётом ссылок, LRU-эвикцией и без обращений к куче
`Rhi::RHI::SupportsRuntimeShaderPipelines`	`Engine/RHI/RHI.hpp`	Запрос возможности: Vulkan возвращает true, Null и Metal — false

```
# Author the graph once, cook it in the build step
build/bin/fby_matc assets/materials/tinted.fgraph -o
assets/materials/tinted.femmat
```

```
// `InBytes` is the .femmat payload from the VFS or a package entry.
Render::CookedMaterialView ScopedMaterial;
if (!Render::DecodeCookedMaterial(InBytes, &ScopedMaterial))
{
    return false;
}

// One cache per device: acquire at load time, release at shutdown.
const Rhi::ShaderPipelineHandle ScopedPipeline = InMaterials.Acquire(InRhi,
ScopedMaterial);

Rhi::MeshRenderItem ScopedItem;
ScopedItem.materialPipeline =
    ScopedPipeline.IsValid() ? ScopedPipeline.id :
Rhi::kDefaultMaterialPipeline;
```

- Хеш графа — идентификатор пайплайна. Правка графа даёт новый хеш, поэтому старый пайплайн становится записью кэша без ссылок, а не меняется на месте.
- Маска контракта едет вместе с payload: сгенерированный вершинный этап, view projection во фрагменте, push-константы материала, varying world/vertex position. Живая запись, чей хеш приходит с другим контрактом, отклоняется: это другой шейдер.
- `Acquire()` добавляет ссылку, `Release()` снимает. Пайплайн с нулём ссылок остаётся резидентным, поэтому материал, берущийся раз в кадр, не пересоздаётся; он уничтожается только при LRU-эвикции, `ReleaseAll()` или `Shutdown()`.
- Кэш не аллоцирует и не форматирует текст на кадровом пути; `LastError()` возвращает фиксированный буфер ошибки.
- Бэкенды без рантайм-пайплайнов остаются на шейдерах движка. Спрашивайте `SupportsRuntimeShaderPipelines()`, а не определяйте это по невалидному хендлу.

Payload	Поля	Проверяется при загрузке
`.fgraph`	magic `FBYG`, версия,	magic, версия, счётчики

	счётчики узлов/пинов/параметров, узлы, пины, параметры	против ёмкости, точный размер, виды узлов, индексы пинов и параметров
`.femmat`	magic `FBYM`, версия, маска контракта, хеш графа, длины вершинного и фрагментного этапов	magic, версия, размер против длин, оба SPIR-V модуля, согласованность контракта

Редактор сохраняет живое редактирование поверх того же кэша: `Fby::Editor::MaterialGraphRuntime` перекомпилирует граф при смене хеша, отдаёт ошибки по вкладкам и передаёт владение пайплайном в `MaterialPipelineCache`. Поставленная сборка обходится без glslang и вызывает только `DecodeCookedMaterial` и `Acquire`.

Спрайты получают материалы через третий артефакт: `MaterialCompileOptions::domain` выбирает спрайтовый домен, где сгенерированный фрагментный этап совпадает с varying-ами `quad.vert` (`vUv`, `vColor`, `vLayer` и `vWorld`, если узел его требует), сэмплирует слой атласа элемента через `vLayer`. `Time` доступен: спрайтовый push-блок несёт часы материала по смещению 80, поэтому сгенерированный этап читает `pc.materialParams.x` так же, как меш-шейдер. `fby_matc --sprite` кукает такой вариант. `DrawOrder::MaterialBucket` помещает спрайт в бакет, `ChangeMaskRenderExtract::SetMaterialPipelines` сопоставляет бакеты с id кэшированных пайплайнов, а экстракт выдаёт один `DrawBatch` на бакет с уже разрешённым `DrawBatch::pipeline`. 64-битный ключ несёт бакет (16 слой, 8 материал, 32 глубина, 8 последовательность), поэтому материальный батч не разрывается глубиной. Узлы, которым нужна меш-проекция (`Fresnel`, `VertexPosition`), отклоняются в спрайтовом домене с понятной причиной.

9. Физика: Box2D v3, суставы, ткань и SdfCollisionSystem

PhysicsSystem владеет одним b2WorldId и отдаёт его расширениям как упакованный uint32, поэтому box2d.h не попадает в публичные заголовки. Тела зеркалируются из компонентов RigidBody2D/Collider2D в Box2D, шагают с фиксированной частотой и читаются обратно перед интерполяцией. PhysicsJointSystem добавляет верёвки и цепочки revolute-звеньев; ClothSystem интегрирует Verlet-сетку и проецирует её на те же коллайдеры Box2D.

Контракт SdfCollisionSystem:

Член	Контракт
SetField(const SdfField&)	Принимает невладеющий mdspan на row-major сетку. Отрицательные значения — твёрдое тело; память поля принадлежит рендереру/рельефу, а не системе.
Step(registry, physics, fixedDt)	Сэмплирует сетку в углах и центре тела, превращает градиент в нормаль, применяет импульс выталкивания с учётом массы, отражает внутрь направленную скорость и пишет исправленное состояние обратно. Возвращает число исправленных тел.

Update(SystemContext&)	ECS-обёртка вокруг Step(); вызов без физики ничего не делает.
Contacts()	Фиксированный массив SdfContact {entity, point, normal, penetration, impulse, sequence}.
SdfCollisionDesc	pushOut, restitution, maxImpulse, subSamples, dynamicOnly.

```
Fby::SdfCollisionDesc desc;
desc.pushOut = 0.65f;
desc.restitution = 0.05f;
desc.subSamples = 2;
sdf.Initialize(desc);
sdf.SetField(terrain.Field());
sdf.Register(graph);                    // Phase::FixedUpdate

// Viewport overlay: red vectors whose length is the penetration.
debug.Build(context, Fby::Editor::PhysicsDebugSdf, &sdf);
```

SdfCollisionSystem не входит в SystemContext. Хост, которому нужна SDF-физика, регистрирует систему явно через Register(graph) и хранит экземпляр живым; отладочный проход Viewport принимает тот же указатель.

10. NetRollbackSession: детерминированный роллбэк

Сессия связывает snapshot v4, CommandLog, Pcg32 и GameplayState в один контур роллбэка с фиксированной ёмкостью. После Initialize() аллокаций нет: одно кольцо снапшотов, одно кольцо ввода на игрока, фиксированная карта сущностей и фиксированные метаданные кадров. Колбэк симуляции — указатель на функцию плюс void*.

Метод	Контракт
Initialize(registry, schema, config, arena)	Настройка без аллокаций с ареной снапшотов, принадлежащей вызывающему.
Initialize(registry, schema, config)	Удобная перегрузка; выделяет одну арену (единственная аллокация сессии).
SetSimulation(fn, user) / SetCommandLog(log)	Привязывает чистую функцию симуляции и лог структурных команд.
Start(rng)	Фиксирует базовый тик и сохраняет RNG сессии. Обязателен до Step().
Step(localInput)	Симулирует ровно один тик с предсказанием отсутствующего удалённого ввода и фиксирует снапшот.
SubmitRemoteInput(player, frame)	Сохраняет опоздавший или текущий ввод; кадр для уже симулированного тика помечает ожидающий роллбэк.
ReplayPending(report)	Восстанавливает снапшот перед самым

	ранним исправленным тиком, восстанавливает RNG и курсор CommandLog, пересимулирует, фиксирует новые снапшоты и сообщает изменённые тики.
CompareChecksums(tick, snap, state, out)	Сравнивает удалённый отчёт с сохранённым кадром; две контрольные суммы отделяют побайтовый desync от desync игрового состояния.

```
Fby::Net::NetRollbackSession::Config cfg;
cfg.playerCount = 2;
cfg.localPlayer = 0;
cfg.rollbackTicks = 64;
session.Initialize(registry, schema, cfg, arena);
session.SetSimulation(&SimulateTick, &game);
session.Start(pcg);

while (running)
{
    session.SubmitRemoteInput(remote, received);
    session.Step(localInput);
    if (session.RollbackPending())
    {
        Fby::Net::RollbackReport report;
        session.ReplayPending(&report);
        if (report.changedTicks > 0)
        {
            LogBoy().Log(Fby::Editor::LogBoySeverity::Desync, "net",
                         "rollback {}..{} changed {}", report.fromTick,
report.toTick,
                         report.changedTicks);
        }
    }
}
```

- NetInputFrame — 24 байта целочисленных осей (int16 с фиксированной точкой). Float запрещён: битовый шаблон float не гарантированно переживает другой компилятор.
- Result перечисляет все режимы отказа (NotInitialized, BufferFull, RollbackTooOld, ChecksumMismatch, ...); ResultName() — написание для логов.
- Сессия некопируемая и владеет ареной, если использована удобная перегрузка.

11. Аудио: микшер, DSP-граф, лимитер

AudioSystem владеет устройством miniaudio и фиксированным микшером. DspGraph маршрутизирует узлы в шины, поддерживает дакинг и реверберацию и принимает команды через lock-free кольцо, поэтому игровой поток не блокирует аудиоколбэк. Limiter.hpp — финальная стадия true-peak.

- DeafMode глушит граф на корне Master и превращает звуковые триггеры в строки субтитров; это политика аудиосистемы, а не пост-эффект.
- Ogg/Opus декодируется вне аудиопотока; микшер видит только приготовленные .fpcm буферы.

12. PinkUI и WidgetCanvasSystem

PinkUI — игровой интерфейс: канвас фиксированной ёмкости, mdspan-раскладка сетки, разрешение якорей, hit-testing и упаковка в один батч. WidgetCanvasSystem превращает WidgetCanvas в FrameOutput (элементы отрисовки) без аллокаций; глифы берутся из атласа .fbyfont, сматпленного через VFS, поэтому поставочный рантайм не содержит TTF-растеризатора. Глифы существуют только в атласе: встроенного 8x8-фолбэка нет, а текстовый кадр без смонтированного массива шрифтов отклоняется, а не рисуется неверным слоем.

Член	Контракт
WidgetCanvasSystem::MountFonts(RHI&)	Обязательный этап холодного старта в главном потоке. Собирает GPU-массив из набора .fbyfont в VFS; ошибка — это PANIC в лог плюс FBY_ASSERT_MSG, а не фолбэк.
BuildFrame(context, canvas, output)	Раскладывает и отсекает один канвас, возвращая BuildStatus и заполняя FrameOutput. Текстовый канвас без актуального смонтированного атласа возвращает FontsNotMounted и ничего не пишет. Без аллокаций; безопасно вызывать из Phase::Render.
Update(context, canvases, output)	Собирает все активные канвасы в один вывод; хост отправляет результат одним батчем. Нарушение шрифтового контракта останавливает проход и ставит fontContractViolated.
WidgetCanvas::SetDisplayScale	Плотность из CameraStackSystem::DisplayScale; размеры шрифтов округляются до целых физических пикселей.
OledShiftState	Целочисленный 2-пиксельный сдвиг для защиты OLED; ApplyOledShift() привязывает его к пулу канвасов.

WidgetCanvasSystem — ещё и поверхность доступности: BlindMode читает подписи виджетов для нарративного лога, DeafMode получает виджеты субтитров, а матрицы цветокоррекции и FlashGuard применяются к цветам канваса перед упаковкой.

13. Ядро доступности (BlindMode, DeafMode, FlashGuard)

Доступность — это политика кадра, а не оверлей. BlindMode пропускает игровой проход, indirect-вызовы и диспатч скиннинга и публикует ограниченный нарративный лог. DeafMode глушит шину Master и превращает звуковые триггеры в SubtitleLines. FlashGuard сжимает резкие изменения яркости. Цветокоррекция использует опубликованные матрицы Machado 2009, а SrgbRelativeLuminance()/ContrastRatio() реализуют WCAG 2.2 для проверки при авторинге.

Элемент	Контракт
NarrativeEvent	SPSC-кольцо на 128 слотов с текстовым payload 160 байт; поток симуляции пишет, поток речи читает.
ColorVisionMatrix(deficiency)	Возвращает матрицу 3x3 для протанопии, дейтеранопии или тританопии.
ApplyColorVision(color, matrix, strength)	Применяет матрицу в линейном свете к sRGB Color32.
FlashGuard::Evaluate(luminance, dt)	Возвращает усиление для текущего кадра; большое изменение сжимается до порога.
EditorSettings::ide*	Те же политики применяются к самому окну редактора: коррекция Machado, высокий контраст WCAG, FlashGuard, навигация с клавиатуры и LogBoy как приёмник нарратива.

14. Конвейер ассетов, VFS и локализация

Ассеты адресуются относительным путём внутри VFS. Архив .fempak монтируется с приоритетом; каталог-монитор может его перекрывать — так редактор пишет loose-файл до того, как кукер вернёт его в архив. Read() возвращает span на mmap, который валиден до размонтирования или горячей перезагрузки файла.

Формат	Владелец	Назначение
.fempak	FempakCooker	Неизменяемый поставочный архив с индексом и опциональным шифрованием.
.fbyfont	FontAtlas (PinkUI)	Испечённый атлас глифов: [header][glyph records][pixels]; без TTF-растеризатора в рантайме.
.femeta	AssetDatabase	Типизированные метаданные, настройки импорта и стабильный AssetId.
.fbin	LocTool	Скомпилированная таблица

		локализации; сам интерфейс редактора — только английский.

Интерфейс редактора намеренно не локализуется: меню, заголовки панелей, ошибки и инспектор — только на английском. Локализация относится к игровому контенту и документации, где таблица .fbin и руководство EN/RU покрывают поставляемые языки.

15. Платформенные сервисы: Steam, Epic Games Store, GOG

PlatformService — мост на стороне движка; PlatformAPI.h — плоский чистый C ABI, который реализует плагин SDK магазина. Провайдер загружается динамически, резолвит только документированные точки входа и преобразует все типы магазина на границе, поэтому игра никогда не включает steam_api.h, EOSSDK или заголовки Galaxy.

Провайдер	Статус в этом дереве	Примечания
SteamProvider.cpp	Реализован	Плоский API Steamworks через таблицу функций.
EgsProvider.cpp	Реализован	Провайдер Epic Online Services через тот же C ABI.
GogProvider.cpp	Реализован (шим)	Загружает galaxy64.dll / libgalaxy64.so / libgalaxy64.dylib во время работы и резолвит плоский ABI шима fby_gog_*. Шим поставляет игра, собранная под свою версию SDK. Galaxy — библиотека C++-интерфейсов, поэтому шим обязателен. GogAchievementSystem и GogStorageSystem пишут бинарный PlatformCommandLog в Phase::PostUpdate.
fby_fontc	Реализован	Офлайн-кукер TTF -> .fbyfont (stb_truetype, RGBA8-атлас, проверяемый заголовок).

```
// Pure C ABI: the provider is a shared library, not a C++ dependency.
typedef struct FbyPlatformApi {
    uint32_t version;
    void* user;
    bool (*init)(void* user);
    void (*shutdown)(void* user);
    void (*unlockAchievement)(void* user, const char* id);
```

```
    bool (*writeCloudFile)(void* user, const char* name, const uint8_t* data,
uint32_t size);
} FbyPlatformApi;
```

16. Редактор: дашборд, шрифты, LogBoy, Scene Diagnostics

Дашборд — единственная поверхность редактора, существующая до открытия проекта. Он устанавливает набор шрифтов редактора до первого кадра, поэтому первое окно уже отрисовано Inter с учётом физического DPI. Новый проект создаётся в Projects/ в корне движка, путь определяет DefaultProjectsRoot() — никогда рабочий каталог процесса и никогда каталог установки.

Компонент	Контракт
EditorFonts	Загружает Inter и JetBrains Mono из mmap VFS. Субпиксельные продвижения (PixelSnapH=false), авто-оверсэмплинг, без bitmap-fallback, динамический атлас печётся под плотность пикселей вьюпорта. Устаревшие бэкенды получают RasterizerDensity + обратный FontGlobalScale.
ProjectDashboard	Initialize(paths, config, visible, window, vfs, fonts, uiScale): устанавливает типографику, определяет расположение Projects/ по умолчанию и создаёт его.
LogBoySystem / LogBoyPanel	Фиксированное кольцо на 4096 записей с текстом 192 байта, уровнями Trace/Info/Warning/Critical/Desync, FNV-1a хэшами систем и фильтрацией по цвету и хэшу.
SceneDiagnosticsPanel	Столбцы и тепловая карта занятости потоков, байты FrameArena, количество MeshRenderItem, indirect-вызовы, draw calls и точное время симуляции в микросекундах с avg/p99/peak.
PhysicsDebugVisualizer	Геометрия линий и точек в фиксированных буферах для тел Box2D, верёвок, ткани и нормалей SDF, рисуется поверх Viewport через карту мир→экран, заданную вызывающим.

Атлас .fbyfont — испечённый ассет PinkUI и не может питать векторный растеризатор ImGui. Поэтому редактор загружает TTF Inter из того же монтирования VFS; заголовок атласа всё равно валидируется при старте, чтобы испорченный атлас игрового UI обнаруживал редактор, а не игрок.

17. Пошагово: игра с пустой папки

Этот пример создаёт играбельный проект со спрайтом, физикой, вводом с клавиатуры, HUD на PinkUI и детерминированной rollback-сессией. Каждый шаг соответствует реальному заголовку из индекса.

- 1. Запустите редактор и создайте проект через New Project. Дашборд создаёт <engineRoot>/Projects/<name> с assets/{textures,audio,scenes}, configs/ и project.femproj.
- 2. Импортируйте спрайт в assets/textures. Наблюдатель импортирует файл, пишет .femeta и готовит слой текстурного массива .ftex.
- 3. Создайте assets/scenes/main.toml с одним SpriteRenderer, одним Transform2D и одним RigidBody2D + Collider2D.
- 4. Зарегистрируйте системы: ввод в PreUpdate, физика в FixedUpdate, движение в Update, PinkUI в Render.
- 5. Дайте камере CameraStackSystem и вызовите SetDisplayScale() из SDL_GetWindowDisplayScale, чтобы глифы UI были чёткими на HiDPI.
- 6. Подключите AccessibilityKernel из настроек проекта; BlindMode и DeafMode читаются из project.femproj, а не зашиты в код.
- 7. Создайте NetRollbackSession на матч, привяжите чистый колбэк SimulateTick и подавайте значения NetInputFrame.
- 8. Логируйте через LogBoy: уровень + хэш системы + сообщение в фиксированном формате. Откройте панель LogBoy и фильтруйте по Desync при тестировании сети.
- 9. Включите Physics Debug Draw во Viewport при настройке коллайдеров и ткани.
- 10. Соберите архив через FempakCooker, прогоните ctest и стилевой гейт, затем поставляйте цель только с движком.

```
// 4-7 in code: systems, accessibility and rollback for one match.
Fby::Ecs::Schedule schedule;
schedule.Add(Fby::Ecs::Phase::PreUpdate, "input", &InputSystem);
schedule.Add(Fby::Ecs::Phase::FixedUpdate, "physics", &PhysicsStepSystem,
&physics);
schedule.Add(Fby::Ecs::Phase::Update, "movement", &MovementSystem);
schedule.Add(Fby::Ecs::Phase::Render, "ui", &UiSystem, &ui);

Fby::AccessibilityKernel kernel;
kernel.Configure(settings);              // blind/deaf/colour/flash

Fby::Net::NetRollbackSession session;
if (session.Initialize(registry, schema, cfg, arena) !=
Fby::Net::NetRollbackSession::Result::Ok)
{
    LogBoy().Log(Fby::Editor::LogBoySeverity::Critical, "net", "rollback init
failed");
}
```

18. Тесты, стилевой гейт и чек-лист релиза

Набор намеренно не использует фреймворк: каждый тест — исполняемый файл с main(), возвращающим ненулевой код при ошибке, поэтому одинаково работает на headless CI-

агенте, консольном devkit и рабочей станции разработчика. На релиз-кандидате проходит весь набор.

```
ctest --test-dir build --output-on-failure          # the full suite
python Source/Tools/Dev/CheckStyle.py               # style gate
python Documentation/Tools/GenerateAPIReference.py  # this manual
python Documentation/Tools/GenerateDocumentation.py
python Documentation/Tools/GenerateApplicationGuide.py
```

- Релизная сборка включает утверждения и один раз прогоняет весь набор с FBY_ENABLE_ASSERTS=1, потому что на RC существовали две скрытые ошибки компиляции debug-сборки в JobSystem.
- POSIX-ветка WriteFileAtomically() и LogLine() должна компилироваться на Linux/macOS в CI; Windows-only сборка их не видела.
- Стилевой гейт должен сообщать ноль ошибок, прежде чем коммит может считаться релизным.

19. Известные пробелы и рекомендуемая следующая работа

Живой бэклог с файлами, строками и статусом каждого пункта — Documentation/ROADMAP.md. Ранее перечисленные здесь пункты (сборка GOG-шима, канвас со смешением текста и собственного массива изображений, std::function в наблюдателях/диалогах окна, озвучивание подписи сфокусированного виджета в BlindMode, время симуляции в SceneDiagnostics и контроль фиксированного шага) отслеживаются там; эта глава не держит вторую копию.

20. GOG Galaxy: провайдер, ABI шима и системы PostUpdate

GOG Galaxy поставляется как библиотека C++-интерфейсов, а не плоский C API. Поэтому движок сохраняет ту же архитектуру, что для Steam и Epic: разделяемая библиотека провайдера говорит только через PlatformAPI.h, а тонкий шим Galaxy (собирается игрой под конкретную версию SDK) экспортирует плоский C-интерфейс, который провайдер резолвит через dlopen/LoadLibrary во время работы. Ни один заголовок Galaxy не попадает в движок, а обновление SDK заменяет шим без пересборки игры.

Экспорт шима	Соответствие
fby_gog_initialize(clientId, clientSecret)	Galaxy::Init с регистрацией User/Listener.
fby_gog_process_data()	Galaxy::ProcessData, вызывается один раз за Pump().
fby_gog_unlock_achievement(name)	IAchievements::SetAchievement + StoreAchievements.
fby_gog_write_cloud/read_cloud/ delete_cloud	Файловый API IStorage; BufferTooSmall сообщает требуемый размер.
fby_gog_pending_cloud_writes()	Незавершённые listener'ы записи Galaxy.

```
// The provider resolves the shim once, then every call is a pointer hop.
bool EnsureLibrary() noexcept
{
    GState.library = OpenGalaxyLibrary();          //
galaxy64.dll / .so / .dylib
    if (GState.library == nullptr)
    {
        return false;                              // honest NotAvailable
    }
    return ResolveFlat(GState.library, "fby_gog_unlock_achievement",
                       GState.api.unlockAchievement);
}

// Game side: PostUpdate turns components into logged commands.
Fby::Platform::GogAchievementSystem achievements;
achievements.Initialize(platform, commandLog, 1);
Fby::Platform::GogAchievementSystem::Register(graph, achievements);

Fby::Platform::GogStorageSystem storage;
storage.Initialize(platform, commandLog, 1);
std::span<const std::byte> save = SerializeSlot(slot);
storage.StageWrite(0x51u, "slot0.sav", save);
Fby::Platform::GogStorageSystem::Register(graph, storage);
```

Обе системы работают в Phase::PostUpdate. Они обходят ограниченный view (64 сущности), детерминированно сортируют по id и сущности, добавляют одну байт-точную запись в PlatformCommandLog и ставят в очередь одну PlatformCommand фиксированного размера. Только PlatformService::Pump() вызывает шим, поэтому колбэк Galaxy не может выполниться в потоке симуляции. Если шима нет, провайдер возвращает NotAvailable, и лог сохраняет намерение вместо имитации разблокировки.

21. Шрифтовой конвейер: fby_fontc, .fbyfont и стабильный буфер

Pink UI никогда не растеризует TTF в рантайме. fby_fontc печёт векторные контуры в формат атласа движка, кукер упаковывает блоб в .fempak, а VFS монтирует его по требованию. Только этот инструмент линкует stb_truetype.

```
fby_fontc --input assets/fonts/Inter-Regular.ttf \
          --output assets/fonts/Inter.fbyfont \
          --size 32 --font-id 0 --ranges latin,cyrillic --verify

fby_fempak --input assets --output assets.fempak
```

Область	Содержимое
Заголовок	48 байт: magic FBYFNT01, версия, число глифов, размер атласа, формат, смещения, высота строки, ascent/descent, id шрифта.
Таблица глифов	40 байт на глиф: кодовая точка, нормализованные UV, advance, bearings,

	размер квада. Отсортирована по возрастанию для бинарного поиска в FontAtlas::Glyph().
Пиксели	RGBA8: белый RGB и маска покрытия в A, поэтому тинт подписи умножается корректно.

Типографика редактора — другой потребитель: ImGui нужны векторные контуры, чтобы печь глифы под произвольный DPI, и он не читает .fbyfont. EditorFonts копирует байты TTF из VFS в собственный буфер до передачи указателя ImGui, поэтому hot reload, размонтирующий файл, не оставит атлас ссылаться на освобождённые страницы. Заголовок атласа всё равно валидируется при старте — так редактор ловит сломанный игровой шрифт.

22. Путь отрисовки ткани: от вершин Verlet до одного indirect-вызова

ClothSystem интегрирует Verlet-сетку и пишет позиции MeshVertex, которые уже финальны. Рендерер не должен их скинить: при boneCount == 0 диспатч скиннинга умножил бы каждую вершину на дефолт стека и схлопнул ткань в точку. Поэтому в движке ровно один помощник классификации и один поставщик данных:

```
// Mesh.hpp: one predicate, used by VulkanRHI and any future backend.
inline bool IsUnskinnedMeshItem(const MeshRenderItem& item) noexcept
{
    return (item.Flags & MeshItemUnskinned) != 0 || item.boneCount == 0;
}

// VulkanRHI: unskinned vertices bypass the compute dispatch entirely.
const bool unskinned = !isStatic && IsUnskinnedMeshItem(item);
memcpy(targetVertexRing + totalSkinned + unskinnedCursor,
item.vertices.data(), ...);
record.unskinned = true;   // mesh.vert reads binding 3, not the skinned SSBO

// App: ClothRenderSystem is the producer the path was missing.
Fby::ClothRenderSystem clothRender;
clothRender.Initialize(&clothSystem);
Fby::ClothRenderSystem::Register(graph, clothRender);   // Phase::PostUpdate
extract.Build(registry, camera, clear, textureArray, clothRender.Items());
```

Мост заполняет фиксированный массив на 64 элемента, принудительно ставит MeshItemUnskinned и сортирует по (sortLayer, depth, ClothId), поэтому порядок отрисовки детерминирован без аллокаций. Регрессионный тест проверяет, что сетка по-прежнему занимает заданные 64x64 пикселя — ровно симптом схлопывания, — и что элемент с нулём костей классифицируется как unskinned, даже если флаг потерян.

23. Атмосфера: погодный контекст, fBm-ветер и GPU-дождь с брызгами по SDF

WeatherContextComponent — единственный авторский источник констант атмосферы и ветра. WeatherSystem раз в кадр вычисляет вектор с модуляцией порывами через fBm-шум и записывает его обратно в компонент и в std140-блок WeatherWindField. Ткань, вершины растений и дождь читают один и тот же вектор, поэтому нет расхождений в направлении ветра.

```
Fby::Weather::WeatherContextComponent weather;
weather.atmosphereDensity = 1.35f;   // denser alien air
weather.airViscosity = 1.1e-5f;
weather.gravityScale = 0.82f;
weather.windSpeed = 26.0f;
weather.windDirectionX = 0.8f;
weather.windDirectionY = -0.6f;
weather.turbulence = 0.55f;
weather.rainIntensity = 0.85f;
weather.Flags |= Fby::Weather::WeatherRain | Fby::Weather::WeatherWind;

// WeatherSystem -> ClothSystem -> shader block, one resolved vector.
cloth.SetAmbientWind({weather.resolvedWindX, weather.resolvedWindY});
shaders.SetWeatherBlock(WeatherWindFieldFrom(weather));
```

WeatherPreconciliatedSystem не хранит частицы на CPU. Он загружает один блок WeatherGpuConstants за кадр и записывает четыре прохода графа: интеграция капель (weather_rain.comp), старение брызг (weather_rain_splash.comp), отрисовка линий (weather_rain.vert/frag) и отрисовка спрайтов брызг. Compute-шейдер дождя интегрирует ветер, сэмплирует глобальную SDF-карту рельефа и добавляет брызгу через atomicAdd, когда капля пересекает неявную поверхность.

```
// weather_rain.comp: one invocation per droplet, 262144 drops per dispatch.
const vec2 previous = particle.state0.xy;
particle.state0.xy += velocity * dt;

const vec2 uv = (particle.state0.xy - weather.fieldTransform.xy) / fieldSize;
const float surface = texture(uSdf, uv).r;          // negative = solid
if (surface <= 0.0 && previousSurface > 0.0)
{
    const uint slot = atomicAdd(splashBuffer.splashCount, 1u);
    splashBuffer.splashes[slot].state0 = vec4(particle.state0.xy, 0.0, 1.0);
    splashBuffer.splashes[slot].color = particle.color;
    spawnParticle(particle, randomState);             // respawn at the top
}
particleBuffer.particles[particleIndex] = particle;
```

Metal-версия содержит те же константы и ту же арифметику брызг; MSL-исходник встраивается в движок при сборке. Стоимость бури на CPU — одна загрузка констант плюс записи графа, независимо от числа капель.

24. ECS через композицию: 2D-шутер без наследования

Player и Enemy — не классы. Это один тип сущности с разными наборами компонентов, а набор работающих систем определяется данными. Этот раздел строит канонический шутер из четырёх компонентов и трёх систем; чтобы добавить босса, нужны компоненты и строка спавна, а не новый подкласс.

```
struct HealthComponent   { float health = 100.0f; float maxHealth = 100.0f;
float invuln = 0.0f; };
struct WeaponComponent   { float damage = 10.0f; float cooldown = 0.0f; float
interval = 0.12f;
                           float projectileSpeed = 900.0f; std::uint32_t
archetype = 0; };
struct InputComponent    { std::int16_t moveX = 0; std::int16_t moveY = 0;
std::uint32_t buttons = 0; };
struct AIBehaviorComponent { std::uint32_t behaviour = 0; float aggroRadius =
400.0f;
                             float fireRange = 320.0f; std::uint32_t targetNetId
= 0; };

// Player: Transform + PhysicsBodyDesc + Health + Weapon + Input
// Enemy : Transform + PhysicsBodyDesc + Health + Weapon + AIBehavior
// Boss  : same as Enemy + phases[] + WeaponArray; no new class.
```

Система	Фаза	Читает / пишет
InputSystem	PreUpdate	Читает InputComponent, пишет его из пакета ввода тика.
AISystem	Update	Читает AIBehavior + Health, пишет InputComponent (ИИ создаёт ввод, а не скорость).
WeaponSystem	FixedUpdate	Читает Weapon + Input, спавнит снаряды через CommandBuffer.
DamageSystem	PostUpdate	Читает события коллизий, уменьшает Health, ставит despawn в очередь.

```
void SpawnEnemy(Fby::Ecs::Registry& registry, const Fby::CharacterDesc& desc,
                const Fby::PhysicsBodyDesc& body, Fby::Vector2D at)
{
    const Fby::Entity enemy = registry.create();
    registry.emplace<Fby::Transform2D>(enemy, Fby::Transform2D{at, {}, {1.0f,
1.0f}, 0.0f});
    registry.emplace<Fby::RigidBody2D>(enemy, Fby::RigidBody2D{.Position = at});
    registry.emplace<Fby::Collider2D>(enemy, Fby::Collider2D{body.halfExtents});
    registry.emplace<HealthComponent>(enemy, HealthComponent{desc.maxHealth,
desc.maxHealth, 0.0f});
    registry.emplace<WeaponComponent>(enemy,
```

```
        WeaponComponent{desc.damage, 0.0f, desc.fireInterval,
desc.projectileSpeed, desc.weaponArchetype});
    registry.emplace<AIBehaviorComponent>(enemy, AIBehaviorComponent{0, 420.0f,
300.0f, 0});
}
```

Детерминизм следует из набора компонентов: те же seed, целочисленный ввод и системы дают ту же контрольную сумму снапшота. Боссу с фазами достаточно добавить PhaseComponent и систему в той же фазе; Player, Enemy, код оружия и сетевой код не меняются.

25. Multi-Draw Indirect и кукинг текстур в продакшене

Спрайты и меши идут двумя разными путями через один FramePacket. Спрайты — 48-байтные RenderItem, отсортированные в батчи; меши — MeshRenderItem, из которых получается одна запись MeshDrawData и одна indirect-команда на элемент, сгруппированные по (textureArray, layer, materialPipeline). GPU не видит привязку дескриптора на объект на этом пути.

```
Fby::Rhi::MeshRenderItem item;
item.vertices   = skinVertices;              // or MeshSystem::WriteMesh()
item.indices    = skinIndices;
item.boneOffset = paletteOffset;
item.boneCount  = paletteSize;               // 0 => unskinned bind pose
item.textureArray = atlas.Handle().id;       // one binding for the scene
item.Layer        = sprite.Layer;
item.model        = Fby::Mat4::Identity();

packet.meshItems = renderExtract.MeshItems();
packet.boneMatrices = renderExtract.BonePalette();
rhi.Submit(packet);                                // one indirect call per group
```

Бюджет	Что измерять
MeshDrawData	80 байт на draw, загружается раз в кадр; firstInstance выбирает запись.
Палитра костей	Один непрерывный блок Mat4 на кадр; никогда не на draw.
Текстурный массив	Одна привязка sampler2DArray на всю сцену; кукер выбирает BC7 (десктоп) или ASTC (мобильные).
SceneDiagnostics	Следите за MeshRenderItem, indirect-вызовами, draw calls и кадровой ареной в одной панели.

```
# Texture cooking: source PNG -> .ftex (BC7/ASTC mips) -> .fempak
fby_fempak --input assets --output assets.fempak --format bc7-srgb
```

```
fby_fontc --input assets/fonts/Inter-Regular.ttf --output
assets/fonts/Inter.fbyfont

# Verify the shipping pack through the same VFS the game uses
ctest --test-dir build -R data_driven --output-on-failure
```

Панель Project Settings редактирует формат кукинга и матрицу коллизий транзакционно; оба значения сохраняются в project.femproj и применяются к живым b2Shape-фильтрам при коммите. Проект, меняющий цель с десктопа на мобильные, сохраняет ту же сцену и перекукивает атлас.

26. Структура репозитория, цели CMake и гейты

Дерево — часть контракта. CheckStyle.py теперь падает, если единица трансляции появляется прямо в Source/Engine или Source/Editor, если существует неожиданный каталог модуля, или если лог сборки, объектный файл, скриншот или Python-кэш закоммичены вне одноразового дерева сборки. Набор модулей фиксирован: Core, RHI, ECS, Physics, Audio, Network, Assets, Script, UI, Video плюс полноценные Weather и Platform.

Цель	Вид	Назначение
fby_engine	статическая библиотека	Рантайм; без ImGui, TOML-писателя и SDK магазина.
fby_editor / fby_editor_app	библиотека + исполняемый файл	Панели ImGui/ImPlot и IDE.
fby_fontc	инструмент	TTF -> атлас .fbyfont; единственный потребитель stb_truetype.
fby_fempak	инструмент	Кукер ассетов и писатель архива.
fby_platform_steam/egs/gog	разделяемые библиотеки	Провайдеры магазинов за C ABI.

```
python Source/Tools/Dev/CheckStyle.py                # style + layout + hygiene
cmake --build build --target fby_editor fby_fontc fby_platform_gog
ctest --test-dir build --output-on-failure        # the full suite
python Documentation/Tools/GenerateCompleteReference.py
```

Шейдеры остаются в Source/Engine/RHI/Shaders, потому что движок встраивает их SPIR-V на этапе конфигурации; этот каталог — модуль шейдеров, а не свободная папка ассетов. Перенос в верхнеуровневый Shaders/ потребовал бы менять шаг встраивания и все пути шейдеров без выгоды для поставки.

27. Контракт потоков: RHI, главный цикл ОС и TTS

У трёх подсистем жёсткая привязка к потоку: командный поток RHI, цикл окна/ImGui и системная речь. Симуляционные системы могут исполняться на воркерах JobSystem, но воркеры никогда не вызывают RHI, не трогают окно и не создают AVSpeechSynthesizer или ISpVoice. Они только пишут в lock-free очереди и логи фиксированного размера.

Точка входа	Поток	Правило
VulkanRHI::DrawPacket	главный поток рендера	Пишет vkCmdBindDescriptorSets внутри цикла батчей; батч несёт полные 32-битные id пайплайна и текстурного массива, 0 наследует массив пакета.
WidgetCanvasSystem::MountFonts	холодный путь, главный поток	Обязательный этап UI-хоста. Создаёт GPU-массив и может вызвать WaitIdle. Отсутствующий набор, пустой набор или неудачный монтаж пишут PANIC и вызывают FBY_ASSERT_MSG: запасного пути нет.
WidgetCanvasSystem::BuildFrame	главный поток рендера	Читает FontAtlasSet::IsCurrent() и 32-битный id массива. Текст без смонтированного массива возвращает BuildStatus::FontsNotMounted и ничего не пишет; никакого создания GPU-ресурсов, WaitIdle, перемапливания VFS и аллокаций после Add().
FontAtlasSet::MountToTextureArray	холодный путь	Проверяет maxTextureLayers устройства до выделения GPU-памяти, публикует новый массив только после сборки всех слоёв и вызывает WaitIdle перед уничтожением предыдущего массива (никакого use-after-free на GPU).
Горячая перезагрузка EditorFonts	главный поток, до NewFrame	DetectFontChanges() -> RHI::WaitIdle() -> ReloadFonts(). PollForFontChanges() удалён: совмещённый помощник уничтожал атлас без

		барьера.
AccessibilityKernel::Update	симуляция / воркер	Строит Narrative Tree и пишет NarrativeEvents. Никогда не вызывает SAPI или AVSpeech.
AccessibilityKernel::SetBlindMode / SetTtsBackend	главный поток ОС (с проверкой)	Сначала RequireMainThread(): вызов из воркера логируется, отбрасывается и вызывает FBY_ASSERT_MSG в assert-сборке. Включение BlindMode может подключить нативный бэкенд и выбрать голос.
AccessibilityKernel::SetNarrativeLocale	любой поток	Меняет язык Narrative Tree. Вызов из воркера лишь запоминает отложенный язык TTS; setVoice() применяется в FlushSpeech() в потоке-владельце.
AccessibilityKernel::FlushSpeech	главный поток ОС	Вызывается из EditorSystem::Tick() / главного цикла SDL. Применяет отложенную локаль, затем дренирует лог и вызывает speak-колбэк бэкенда. Вызов из воркера возвращается до обращения к логу.
SAPI 5 (Windows)	главный STA-поток	CoInitializeEx(COINIT_APARTMENTTHREADED) парен с CoUninitialize. SetVoice перебирает все установленные токены и оценивает их по BCP-47 тегу; при отсутствии совпадения используется системный голос по умолчанию.
AVSpeechSynthesizer (macOS)	главный run loop	Initialize/Speak/SetVoice проверяют [NSThread isMainThread]. SetVoice выбирает лучший установленный язык голоса и откатывается к системному по умолчанию.
Воркеры JobSystem	любой воркер	Могут исполнять ECS-системы и писать в lock-free кольца; не могут вызывать

		RHI, окно, перемапливание VFS или TTS.

```
// Cold path: app init / scene load, OS main thread. Creates the GPU array once.
ui::WidgetCanvasSystem scopedUi{&fonts};
if (!scopedUi.MountFonts(*gpu))
{
    // Panic already logged; an assert build never reaches this line.
    return Shutdown(ExitCode::UiFontsUnavailable);
}

// Hot path: Render phase, main thread. No GPU creation, no WaitIdle.
ui::FrameOutput out;
const ui::BuildStatus status = ui::WidgetCanvasSystem::BuildFrame(ctx, canvas,
out);
if (status == ui::BuildStatus::FontsNotMounted) { /* refuse the whole UI pass */
}
// out.batches[0].textureArray is the 32-bit font array id and out.items carry
the
// real layer (0 = white, 1..N = fonts); DrawPacket rebinds per batch.

// Font hot reload: the barrier is deliberate and one-shot.
if (fonts.DetectFontChanges())
{
    gpu->WaitIdle();
    fonts.ReloadFonts(ImGui::GetIO());
}

// Speech: Simulation never flushes. The editor/Main loop owns it.
void EditorSystem::Tick(ecs::SystemContext& ctx)
{
    if (ctx.accessibility) { ctx.accessibility->FlushSpeech(); }  // main thread
only
    // ... ImGui frame ...
}
```

Правило: если операция может ждать GPU, создавать VkImage, закрывать mmap VFS или обращаться к COM/Objective-C UI-объекту, она принадлежит холодному пути или главному потоку ОС. Горячие пути рендера и симуляции только читают неизменяемое состояние и пишут в очереди. Debug-сборка превращает каждую такую границу в FBY_ASSERT_MSG; release-сборка логирует и отбрасывает небезопасный вызов, а не выполняет его.

28. 32-битный конвейер DrawBatch: конец потолка в 65 536 текстур

DrawBatch — единственное место, где id текстурного массива и id шейдерного пайплайна пересекают границу render-extract. До аудита RC-2 оба поля были std::uint16_t, тогда как все RHI-хэндлы (TextureArrayHandle, ShaderPipelineHandle) — std::uint32_t. Результат — тихое совпадение id: 65 537-й текстурный массив или пайплайн обрезался до чужого младшего id, и батч сэмплировал не тот ресурс — без ошибки, без assert и без следа в логе. Исправление

расширяет оба поля до std::uint32_t и переставляет их так, что структура по-прежнему ровно 16 байт.

Смещение	Поле	Тип	Контракт
0	firstItem	uint32_t	Первый RenderItem серии; индекс в span элементов кадра.
4	itemCount	uint32_t	Число непрерывных элементов в серии.
8	pipeline	uint32_t	ShaderPipelineHandle ::id, 0 = пайплайн квадов. Без сужения до uint16.
12	textureArray	uint32_t	TextureArrayHandle::i d, 0 = наследовать FramePacket::texture Array (id 0 по контракту — встроенный атлас бэкенда).

```
// RenderItem.hpp - 16 bytes, no padding, four batches per 64-byte cache line.
struct DrawBatch
{
    std::uint32_t firstItem = 0;
    std::uint32_t itemCount = 0;
    std::uint32_t pipeline = 0;      // ShaderPipelineHandle::id
    std::uint32_t textureArray = 0;  // TextureArrayHandle::id, 0 = inherit
};
static_assert(sizeof(DrawBatch) == 16);
static_assert(alignof(DrawBatch) == 4);
static_assert(offsetof(DrawBatch, textureArray) == 12);
static_assert(std::is_trivially_copyable_v<DrawBatch>);

// VulkanRHI::DrawPacket - the id is consumed at full width.
const std::uint32_t requested =
    batch.textureArray != 0 ? batch.textureArray : packet.textureArray;
if (requested != boundArray) { BindDescriptorFor(requested); boundArray =
requested; }

// Pink UI writes the same 32-bit id: the old "<= 0xFFFF" clamp is gone.
Output.batches[0] = rhi::DrawBatch{start, count, canvas.pipeline, fontArray.id};
```

- На пути не остаётся ни одного сужающего приведения: поиск 0xFFFF в батче, WidgetCanvas и цикле батчей Vulkan ничего не находит.
- WidgetCanvas::textureArray и WidgetCanvas::pipeline тоже 32-битные; SetTextureArray и SetPipeline принимают std::uint32_t.

- MeshRenderItem::textureArray уже был std::uint32_t; static_assert'ы теперь фиксируют его ширину, чтобы будущее «уплотнение» не вернуло потолок на mesh-путь.
- Хост, индексирующий собственную таблицу дескрипторов по id из батча, обязан его проверять: движок передаёт id, а бэкенд для неизвестного id логирует ошибку и использует встроенный атлас вместо разыменования.

Почему структура не выросла: два бывших uint16_t поля плюс 4-байтный паддинг стали двумя uint32_t полями. Батч по-прежнему 16 байт, отсортированный массив батчей сохраняет плотность, а четыре подряд идущих батча делят одну кэш-линию. Расширение id ресурса не должно стоить промаха кэша в покадровом цикле батчей.

29. Атомарный хот-релоад шрифтов VFS и строгий контракт MountFonts

Текст Pink UI существует только в атласе. mmap VFS поставляет приготовленные блобы .fbyfont, холодный путь упаковывает их в один GPU-массив (слой 0 — белый, атласы шрифтов 1..N), а горячий путь только читает этот массив. Аудит RC-2 нашёл в этой цепочке две дыры: ленивый монтаж, позволявший первому кадру тихо откатиться на 8x8-слой, и повторный монтаж, уничтожавший предыдущий массив, пока кадр ещё мог его сэмплировать. Оба места теперь — жёсткие контракты.

Этап	Поток / момент	Правило
FontAtlasSet::Add / LoadFromVfs	холодный путь	Сразу проверяет заголовок .fbyfont и общий размер пикселей, растит scratch-буфер вне кадра и увеличивает Generation набора.
WidgetCanvasSystem::MountFonts	холодный старт, главный поток	Обязателен. Пустой/нулевой/неудачный монтаж -> PANIC в LogBoy + FBY_ASSERT_MSG (debug). Перемонтирует набор, если он не актуален.
FontAtlasSet::MountToTextureArray	холодный путь	Проверяет maxTextureLayers до выделения, собирает все слои, публикует атомарно и вызывает WaitIdle перед освобождением предыдущего массива.
FontAtlasSet::IsCurrent	чтение с горячего пути	MountedGeneration == Generation. Добавление атласа инвалидирует монтаж; следующий кадр отклоняется, пока MountFonts() не выполнится снова.
WidgetCanvasSystem::BuildFrame	главный поток рендера	Текст с отсутствующим/устаревшим/

		неактуальным набором, неверным индексом атласа или чужим массивом изображений возвращает FontsNotMounted и ничего не пишет. Без монтажа, без WaitIdle, без аллокаций.
EditorFonts::DetectFontChanges	главный поток, между кадрами	Только обнаружение. Затем хост вызывает RHI::WaitIdle() и ReloadFonts(). PollForFontChanges() удалён, потому что пропускал барьер.

```
// 1. Cold start: load -> mount -> build. Order is mandatory.
ui::FontAtlasSet fonts;
if (!fonts.LoadFromVfs(vfs, "fonts/Inter.fbyfont")) { return Fail("atlas"); }
ui::WidgetCanvasSystem ui{&fonts};
if (!ui.MountFonts(*gpu)) { return Fail("font array"); }  // PANIC + assert in
debug

// 2. Hot frame: atlas-only text, full 32-bit array id.
ui::FrameOutput out{items, keys, batches, 0};
if (ui::WidgetCanvasSystem::BuildFrame(ctx, canvas, out) ==
ui::BuildStatus::FontsNotMounted)
{
    frame.fontContractViolated = true;  // release: draw no text, never a bitmap
}

// 3. Adding a locale atlas at runtime invalidates the mount.
fonts.LoadFromVfs(vfs, "fonts/NotoSansJP.fbyfont");
if (!fonts.IsCurrent()) { (void)ui.MountFonts(*gpu); }  // remount, old array
waits idle

// 4. Editor ImGui typography: the only supported hot-reload chain.
if (editorFonts.DetectFontChanges())
{
    gpu->WaitIdle();
    editorFonts.ReloadFonts(ImGui::GetIO());  // between frames, before
NewFrame()
}
```

- Владение VFS: EditorFonts копирует каждый TTF в стабильный буфер до того, как его увидит ImGui, поэтому перемапливание VFS не может оставить атлас указывающим в размонтированную страницу.
- Владение GPU: FontAtlasSet ждёт устройство перед уничтожением текстурного массива, потому что предыдущий кадр может ещё сэмплировать его descriptor set.
- Политика отказа: отсутствующий атлас заметен. 8x8-фолбэка нет ни в одной сборке; release-сборка отклоняет текстовый кадр и ставит FrameOutput::fontContractViolated.

Известное ограничение, которое теперь проверяется, а не скрывается: канвас, смешивающий текст с собственным массивом изображений, не может быть одним батчем, потому что один батч сэмплирует ровно один массив. BuildFrame отклоняет его с FontsNotMounted. Дальнейшее исправление — разбиение батчей по массивам (отсортированные серии элементов, сгруппированные по массиву), что требует контракта FrameOutput с несколькими батчами; запланировано после Release.

30. TTS: контракт главного потока и локализованный выбор голоса

Речь — единственная поверхность доступности, которую нельзя симулировать на воркере. SAPI 5 создаёт COM-объект с apartment-моделью, AVSpeechSynthesizer владеет объектом главного run loop, и ни один из них не терпит создания или управления из потока JobSystem. Ядро теперь обеспечивает это тремя слоями: RequireMainThread() на каждой точке входа, FBY_ASSERT_MSG в assert-сборке и release-фолбэк, который логирует и отбрасывает небезопасный вызов. Воркер симуляции по-прежнему может сменить язык повествования; он лишь запоминает отложенный язык, который FlushSpeech() применяет в потоке-владельце перед следующей фразой.

Точка входа	Поток	Вызов из воркера
SapiTts::Initialize / AvSpeechTts::Initialize	главный поток ОС / главный run loop	Возвращает false после RequireMainThread(); assert-сборка падает.
SapiTts::SetVoice / AvSpeechTts::SetVoice	главный поток ОС / главный run loop	Отбрасывается с записью в лог; вызова COM/Objective-C не происходит.
SapiTts::Speak / Stop / SetRate	главный поток ОС	Отбрасывается; SAPI остаётся рабочим для потока-владельца.
AccessibilityKernel::SetBlindMode	главный поток ОС	Возврат до изменения настроек и подключения бэкенда.
AccessibilityKernel::SetTtsBackend	главный поток ОС	Возврат; предыдущий бэкенд остаётся установленным.
AccessibilityKernel::SetNarrativeLocale	любой поток	Запоминает отложенный язык; setVoice() выполняется позже в FlushSpeech().
AccessibilityKernel::FlushSpeech	главный поток ОС	Возврат до обращения к логу повествования, поэтому ни одно событие не крадётся.
NativeTtsBackend	главный поток ОС	Возвращает nullptr; COM/AVFoundation никогда не инициализируется на воркере.

Выбор голоса — это поиск по локали, а не фильтр токенов. Старая реализация сопоставляла только первую букву тега с Language=409/419 и молча возвращалась для всего остального, поэтому русское повествование на английской ОС сохраняло английский голос (или после неудачного фильтра — вообще тот, что уже был). Теперь SetVoice() разбирает основной язык BCP-47, разрешает полный тег через LocaleNameToLCID, перебирает все установленные токены SAPI и оценивает их.

Оценка	Значение	Пример
4	Точное совпадение LCID	запрошен ru-RU, установлен ru-RU
3	Тот же язык и тот же субъязык/регион	запрошен ru-RU, установлен ru (0x0419)
2	Тот же основной язык, другой регион	запрошен en-GB, установлен en-US
0	Общего языка нет -> системный голос по умолчанию	запрошен ru-RU на ОС только с английским

```
// AccessibilityKernel::SetBlindMode - main thread only.
if (!RequireMainThread("AccessibilityKernel::SetBlindMode")) { return; }
MTtsBackend = NativeTtsBackend();   // SAPI COM / AVSpeech created here
ApplyTtsLanguage();                // setVoice on the owner thread

// A worker that changes the locale never reaches the backend.
void AccessibilityKernel::SetNarrativeLocale(std::string_view locale) noexcept
{
    // ... update MTtsLanguage ...
    if (IsMainThread()) { ApplyTtsLanguage(); } else { MPendingVoiceLanguage =
true; }
}

// The owner thread applies it before draining the log.
void AccessibilityKernel::FlushSpeech() noexcept
{
    if (std::this_thread::get_id() != MMainThread) { return; }  // drop, do not
steal
    if (MPendingVoiceLanguage) { ApplyTtsLanguage(); }
    // ... speak NarrativeEvents ...
}

// SAPI fallback: score 4/3/2, else the system default. Never silent.
LogWarn("tts", "no installed voice matches '{}'; falling back to LCID 0x{:04X}",
        tag, defaultLcid);
```

- Реализация SAPI больше не включает sphelper.h: она использует ISpObjectTokenCategory напрямую, поэтому сборка движка не зависит от компонента ATL (машина без atlbase.h всё равно собирается).

- AVSpeech перебирает [AVSpeechSynthesisVoice speechVoices], оценивает префикс языка и сохраняет лучший установленный язык; системный по умолчанию — фолбэк.
- Каждое решение логируется один раз с запрошенным тегом, выбранным LCID/языком и признаком фолбэка, поэтому в тикете поддержки видно, что игрок реально слышал.

Debug-сборки компилируют FBY_ASSERT_MSG в каждую перечисленную проверку. Вызов не из того потока останавливается в отладчике с точным именем функции вместо плавающего отказа COM или тихой порчи run loop. Release-сборки сохраняют семантику проверки, но не abort: они логируют и отказывают в вызове.

31. Геймплей на состояниях: FemAnimGraph, ИИ StateTree и холст Animation Graph

Высокоуровневый геймплейный слой следует тому же правилу, что и рендерер: поведение — это функция над простыми данными, а не граф объектов. FemAnimGraph — плоский автомат анимаций, FemStateTree — плоское дерево состояний ИИ. Ни у одного нет базового класса, виртуального Evaluate(), аллокации на узел или колбэка std::function. Сущность владеет POD-компонентом фиксированного размера; система владеет общими таблицами (клипами анимации и массивами узлов/переходов ИИ) и обходит только активный путь.

Почему не Behaviour Tree: BT переобходит иерархию с корня каждый тик, поэтому цена простаивающего NPC пропорциональна размеру дерева. State Tree хранит активный путь и переоценивает одну проверку; переходы — плоский массив, индексируемый триггерами. Та же логика у AnimGraph: условия переходов — это битовые тесты и сравнения float на компоненте, а дорогая часть — поза костей — смешивается один раз за тик, а не на каждый узел графа.

Поле AnimStateComponent	Тип	Назначение
CurrentState	uint64_t	Непрозрачный логический id состояния. 0 = rest-поза; нумерация принадлежит геймплею.
PreviousState	uint64_t	Исходная поза активного кросс-фейда.
StateTime	float	Секунды в текущем состоянии; сбрасывается при каждом переходе.
TransitionCount	uint32_t	Диагностика. Размещено по смещению 20, чтобы заполнить паддинг перед массивом переходов с выравниванием 8.
Transitions[8]	AnimTransition	Авторские переходы, перебираются по порядку; побеждает первое совпадение. Инертны при Conditions ==

		AnimConditionNone.
Clips[8] / ClipCount	AnimClip	Таблица клипов сущности: ключи костей плюс диапазон кадров SpriteAnimation для того же состояния.
Floats[8]	float	Float-параметры (скорость, здоровье, прицел), читаемые условиями переходов.
RestPose[128]	AnimRestBone	Снимается один раз со скелета; клип описывает только те кости, которые двигает.
BlendDuration / BlendTime / BlendWeight	float	Тайминг кросс-фейда и сглаженный вес, который применяет мост SkeletalAnimation.

Флаг условия	Проверка
AnimConditionStateTimeAtLeast	StateTime >= StateTimeThreshold
AnimConditionStateTimeAtMost	StateTime <= StateTimeThreshold
AnimConditionTagsAny	(GameplayState.Bits & TagsAny) != 0
AnimConditionTagsAll	(GameplayState.Bits & TagsAll) == TagsAll
AnimConditionTagsNone	(GameplayState.Bits & TagsAny) == 0
AnimConditionFloatAtLeast / AtMost	Floats[FloatParam] сравнивается с FloatThreshold; не-конечное значение даёт false.

```
// AnimGraph.hpp - the component prefix is stable and visible in an inspector.
struct AnimStateComponent
{
    std::uint64_t CurrentState = 0;   // 0  logical state, 0 = rest
    std::uint64_t PreviousState = 0;  // 8  cross-fade source
    float StateTime = 0.0f;           // 16 seconds in state
    std::uint32_t TransitionCount = 0;// 20 diagnostics / pad
    std::array<AnimTransition, 8> Transitions{};  // 24
    std::array<AnimClip, 8> Clips{};
    std::array<float, 8> Floats{};
    std::array<AnimRestBone, kMaxBones> RestPose{};
    float BlendDuration = 0.0f, BlendTime = 0.0f, BlendWeight = 1.0f;
    // ...
};

// Once per fixed tick: conditions -> state -> two bridges.
for (auto [entity, state] : registry.view<AnimStateComponent>().each())
{
```

```
    state.StateTime += dt;
    const std::uint64_t tags = gameplayBits(registry, entity);
    for (auto& t : state.Transitions)
    {
        if (t.FromState != kAnimAnyState && t.FromState != state.CurrentState)
continue;
        if (!AnimTransitionMatches(t, state, tags)) continue;
        state.PreviousState = state.CurrentState;
        state.CurrentState = t.ToState;
        state.StateTime = 0.0f;
        state.BlendDuration = t.BlendDuration;
        break;   // first match wins, at most 8 comparisons
    }
    // SkeletalAnimation: blend local TRS, then one hierarchy pass.
    if (auto* skeleton = registry.try_get<SkeletonComponent>(entity))
        AnimGraphApplySkeletal(state, *skeleton);
    // SpriteAnimation: only on a state change, so the frame advance is not
fought.
    if (changed) AnimGraphApplySpriteClip(state, spriteAnimation, grid,
spritePtr);
}
```

Мост SkeletalAnimation смешивает в пространстве локальных TRS — кратчайшая дуга поворота, линейные позиция и масштаб — и затем вызывает SkeletalAnimation::Evaluate() ровно один раз. Этот единственный проход по иерархии перестраивает globalMatrices и палитру Mat4 skinningMatrices, которую потребляет GPU. Лерп двух готовых палитр Mat4 был бы дешевле на один проход, но даёт сдвиг (shear) на вращающейся цепочке родитель/ребёнок и не является жёстким преобразованием; локальный блендинг сохраняет каждую кость жёсткой. Мост SpriteAnimation — обратный компромисс: он мгновенный по замыслу (startFrame, frameCount, длительность, currentFrame = 0, UV применяется в том же тике), потому что у флипбука нет интерполируемой позы.

Поле StateTreeComponent	Тип	Назначение
ActiveStates[4]	uint16_t	Плотный путь от корня: [0] корень, последний = активный лист, свободные = kStateTreeNone. State Tree никогда не хранит весь список узлов на сущность.
CurrentLeaf	uint32_t	Индекс активного листа (совпадает с ActiveStates[ActiveDepth - 1]).
ActiveDepth	uint32_t	1..4; проверяется StateTreeAddNode, поэтому BuildPath не может выйти за границы.
Triggers	uint64_t	Ожидающие триггеры изменений. 0 -> таблица переходов в этом тике не

		трогается вообще. Бит 63 зарезервирован под внутреннее событие «guard листа провален».
Tags	uint64_t	Снимок GameplayState.Bits на этот тик; все оценщики видят одну маску.
Target / TargetPosition	Entity / Vector2D	Blackboard: живая цель-сущность плюс закэшированная позиция для точек патруля.
Floats[4]	float	Float-blackboard для FloatBelow / FloatAbove (здоровье, угроза, ...).
StateTime / Ticks	float / uint32_t	Время в текущем листе и число фиксированных тиков с момента входа; сбрасываются BuildPath.
TreeId	uint32_t	Выбирает StateTreeDefinition, принадлежащее системе; много сущностей делят одно дерево.

Плоская таблица	Содержимое	Цена за тик
StateTreeDefinition::Nodes[64]	Тип Root/Selector/Leaf, родитель, интрузивный список детей, глубина, оценщик, задача.	В устойчивом состоянии не обходится.
StateTreeDefinition::Transitions[32]	FromLeaf, ToNode, TriggerMask, ConsumeMask, RequireTagsAll, ForbidTagsAny.	Перебирается только при Triggers != 0.
StateTreeEvaluator	Плоский enum + uint64-маска тегов + float-порог + индекс параметра, разбирается одной функцией.	Один вызов для активного листа.
StateTreeTask	None / MoveToTarget / Stop / AttackTarget / SetTag / ClearTag.	Один switch; перемещение и атака идут через CommandBuffer.

```
// Authoring: a fixed table, no allocations, depth enforced at AddNode().
StateTreeDefinition tree; tree.TreeId = 7;
const auto root  = StateTreeAddNode(tree, StateTreeNodeKind::Root);
const auto group = StateTreeAddNode(tree, StateTreeNodeKind::Selector, root);
const auto attack = StateTreeAddNode(tree, StateTreeNodeKind::Leaf, group);
```

```
const auto move   = StateTreeAddNode(tree, StateTreeNodeKind::Leaf, group);
StateTreeSetLeaf(tree, attack,
                 StateTreeEvaluator{StateTreeEvaluatorKind::TargetWithinRadius,
0, 64.0f, 0},
                 StateTreeTask{StateTreeTaskKind::AttackTarget, 0, 0.0f, 48.0f,
0});
StateTreeSetLeaf(tree, move,
                 StateTreeEvaluator{StateTreeEvaluatorKind::AlwaysTrue, 0, 0.0f,
0},
                 StateTreeTask{StateTreeTaskKind::MoveToTarget, 0, 120.0f, 0.0f,
0});
StateTreeTransition spotted{};
spotted.FromLeaf = move;
spotted.ToNode = attack;
spotted.TriggerMask = 1ull << 0;   // "player spotted"
spotted.ConsumeMask = 1ull << 0;
StateTreeAddTransition(tree, spotted);

// Runtime: the path is data, the transition table is consulted only on a
trigger.
void StateTreeSystem::Run(ecs::SystemContext& ctx) const noexcept
{
    for (auto [entity, state] : ctx.registry.view<StateTreeComponent>().each())
    {
        const auto* tree = Tree(state.TreeId);
        if (!state.Active || tree == nullptr) continue;
        state.Tags = gameplayBits(ctx.registry, entity);
        ++state.Ticks; state.StateTime += ctx.dt;
        if (!state.Initialized) { BuildPath(*tree, SelectLeaf(...), state); }
        else if (!GuardPasses(state, *tree)) state.Triggers |=
kStateTreeTriggerLeafFailed;
        if (state.Triggers != 0) TryTransitions(*tree, state);  // flat, <= 32
entries
        RunTask(ctx, entity, state, *tree);  // emits AICommandComponent
    }
}
```

- Ни наследования, ни виртуального вызова: StateTreeNodeKind, StateTreeEvaluatorKind и StateTreeTaskKind — enum, разбираемые в одной единице трансляции. Плагин добавляет оценщик расширением enum и switch, а не подклассом узла.
- Никакого переобхода дерева за кадр: без ожидающего триггера система читает один guard и выполняет одну задачу. Тик Behaviour Tree затронул бы каждую ветвь иерархии.
- Никаких рантайм-аллокаций: узлы, переходы, активный путь и payload команд — фиксированные массивы; CommandBuffer копирует крошечный AICommandComponent в кадровую арену, а Play() применяет его после фиксированных шагов.
- Детерминизм: Phase::FixedUpdate + фиксированный dt + воспроизведение команд в порядке вставки. Сетевой rollback воспроизводит тот же путь ActiveStates из тех же триггеров.

Рантайм-таблицы выше создаются в AnimationGraphCanvasPanel — ImNodes-редакторе из Source/Editor/Panels. Панель намеренно не является второй рантайм-моделью: её документ — trivially copyable AnimationGraphDocument, зарегистрированный как один external-buffer CommandStack. Узел клипа компилируется в AnimClip, узел float-параметра — в AnimStateComponent::Floats, прямая связь клип-клип — в AnimTransition, а blend-узел — в AnimTransition кросс-фейда вида Idle -> Attack. Рантайм никогда не видит ImNodes, позиции, имена и редакторскую топологию блендинга.

Узел редактора	Пины	Во что компилируется
Clip	In, Out	AnimClip (State, Keys[8], SpriteAnimation start/count/duration/loop/play).
Float Parameter	Float out	AnimStateComponent::Floats[ParameterIndex] = ParameterValue; индекс также используется условиями AnimConditionFloatAtLeast/At Most.
Blend	Src, Mod, Out	Один AnimTransition из исходного клипа в целевой; BlendDuration и битовая маска условий хранятся в blend-узле, необязательная связь Mod задаёт FloatParam.

Поле AnimationGraphDocument	Тип / ёмкость	Назначение
GraphId	uint32_t	Идентификатор для блока TOML сцены и привязки PIE.
NodeCount / Nodes[24]	uint16_t / AnimationGraphNode	Фиксированный POD-узел: kind, enabled, name[32], position, AnimClip, payload AnimTransition для blend, индекс/значение параметра.
LinkCount / Links[32]	uint16_t / AnimationGraphLink	Kind, концы, пины и payload прямого AnimTransition. Четыре вида связи: Transition, BlendSource, BlendParameter и BlendTarget.

```
// CompileGraph(): fixed storage, one flat AnimStateComponent, no allocation.
AnimStateComponent out;
std::array<bool, kMaxAnimParams> paramUsed{};
std::array<std::uint64_t, kMaxAnimClips> stateIds{};
```

```
std::uint16_t clipCount = 0, transitionCount = 0;
for (node : document.Nodes)
    if (node.Kind == Clip)
    {
        if (node.Clip.State == 0 || duplicate(stateIds, node.Clip.State))
reject();
        stateIds[clipCount] = node.Clip.State;
        out.Clips[clipCount++] = node.Clip;
    }
    else if (node.Kind == FloatParameter)
        out.Floats[node.ParameterIndex] = node.ParameterValue;
// Isolated-node check: every node needs at least one incident link.
for (node : document.Nodes) if (degree[node] == 0) reject("isolated");
// Direct links become transitions; each Blend needs one source, one target
// and at most one parameter binding.
for (link : document.Links)
    if (link.Kind == Transition && conditionsValid(link.Transition))
        out.Transitions[transitionCount++] = link.Transition;
for (blend : document.Nodes)
    if (blend.Kind == Blend)
        out.Transitions[transitionCount++] = makeTransition(blend, source,
target);
for (i = transitionCount; i < 8; ++i)
    out.Transitions[i].Enabled = false;   // inert slots, not garbage
out.ClipCount = clipCount;
out.TransitionCount = 0;                  // runtime diagnostic, not an authored
count
```

Каждый жест — одна транзакция CommandStack. Документ является одним external buffer, поэтому перетаскивание узла записывает только изменившиеся байты позиции, а переключение условия — только изменившееся слово условий. Колбэки ImNodes никогда не вызывают Record() напрямую: DrawCanvas сообщает об изменении, ProcessCanvasEvents открывает BeginEdit(), метод правки меняет документ, RecordEdit() снимает новые байты, CommitEdit() фиксирует одну группу undo. Ctrl+Z/Ctrl+Y обрабатывает EditorSystem: сначала завершает открытую транзакцию холста, затем просит CommandStack применить обратную дельту. Правки инспектора используют Mirror-хелперы AnimationGraphTransitionFieldsEqual()/_ClipFieldsEqual(); оба сравнивают только диапазоны байтов FieldDesc через reflect::FieldBytesEqual(), поэтому паддинг не создаёт пустую undo-команду.

Бит условия	Контрол инспектора	Рантайм-проверка
AnimConditionStateTimeAtLeast	Чекбокс StateTimeAtLeast + StateTimeThreshold	StateTime >= threshold
AnimConditionTagsAll	Чекбокс TagsAll + маска TagsAll	(tags & TagsAll) == TagsAll; пустая маска даёт false
AnimConditionFloatAtMost	Чекбокс FloatAtMost + FloatParam + FloatThreshold	Floats[FloatParam] <= FloatThreshold

```
# Appended by "Compile Animations"; EditorSystem re-appends it after Ctrl+S.
[anim_graph]
version = 1
graph_id = 12
node_count = 4
link_count = 4

[[anim_graph.node]]
kind = 0                        # Clip
name = "Idle"
position = [-260.0, 40.0]
clip_state = "0x0000000000000001"
clip_key_count = 1
clip_sprite_count = 4
[[anim_graph.node.key]]
bone = 1
rotation = 0.5
position = [0.0, 0.0]
scale = 1.0

[[anim_graph.node]]
kind = 2                        # Blend
blend_duration = 0.2
blend_conditions = 64           # FloatAtMost/AtLeast family
blend_float_param = 0

[[anim_graph.link]]
kind = 0                        # Transition
from = 1
to = 0
t_conditions = 1                # StateTimeAtLeast
t_state_time = 1.0
t_duration = 0.15
```

PIE-дебаг — это read-only проекция живого компонента обратно на авторский документ. RefreshPIEState() находит выбранную сущность (EditorSelected), читает AnimStateComponent::CurrentState/PreviousState и сопоставляет эти 64-битные id с узлами Clip; LastTransition отображается через таблицу transition-to-link компилятора в точную авторскую связь. Текущий узел рисуется пульсирующим неоновым контуром и ProgressBar для BlendWeight, PreviousState — цветом источника кросс-фейда, а BlendTime/BlendWeight показываются и на холсте, и в Mirror-инспекторе. Debug-draw ничего не аллоцирует: он только читает компонент и пишет команды отрисовки ImGui.

Обе системы не зависят от рендера: AnimStateComponent пишет только данные скелета/SpriteAnimation, а StateTreeComponent — только намерения ИИ. Metal-бэкенд теперь потребляет тот же 32-битный контракт DrawBatch, что и Vulkan — per-batch setFragmentTexture: и смещение вершинного буфера вместо baseInstance — поэтому батч, который производит кадр State Tree или AnimGraph, выглядит одинаково на обоих API.

32. Продвинутые AAA DOD-конвейеры: виртуальные текстуры, маски изменений компонентов, сетки релевантности и аудио-шины

Глава 31 описывает четыре конвейера, добавленные в сборку Release. Их объединяет одно ограничение: кадр не должен выделять память, обходить граф сцены и загружать ресурс, который не изменился. VirtualTextureSystem превращает мир в VFS-отображённые файлы чанков и фиксированный texture array 2048; ChangeMaskRenderExtract превращает 64-битное слово на сущность в слитые диапазоны SSBO; RelevanceGrid превращает пространственный mdspan в фильтр снапшотов для игрока; AudioZoneSystem превращает объём Box2D-сенсора в lock-free ramp параметров шины. У каждой подсистемы фиксированная раскладка данных и headless-тест приёмки.

Конвейер	Изменяемое состояние	Ограничено	Аллокация за кадр
VirtualTextureSystem	Таблица резидентности страниц + отображения VFS	layerCount x 32 x 32 страниц	0
ChangeMaskRenderExtract	Стабильные слоты сущностей + кэш сортировки	Параметр шаблона Capacity	0
RelevanceGrid	mdspan-ячейки + один 64-битный bitset на план	cellsX x cellsY <= 4096	0
AudioZoneSystem	Одна запись контекста на слушателя	kMaxZones = 64 авторизованных зон	0

31.1 Виртуальные текстуры. Адресное пространство мира делится на тайлы 64x64. Четыре тайла по оси образуют один chunk-файл 256x256 RGBA8, поэтому одно отображение VFS даёт шестнадцать загрузок тайлов. GPU владеет фиксированным texture array 2048x2048 с N слоями; слой — это пул из 32x32 страниц, и каждая страница — один тайл. Резидентный объём поэтому строго равен

```
// Constant VRAM, independent of the camera path or the world size.
constexpr std::uint64_t kPageBytes = 2048ull * 2048ull * 4ull;
const std::uint64_t Budget = layerCount * kPageBytes;

// Residency is a two-pass clock over a fixed page table.
void VirtualTextureSystem::Update(const VirtualTextureCamera& InCamera) noexcept
{
    ++MFrame;
    const float HalfX = InCamera.ViewSize.X * 0.5f / InCamera.Zoom;
    const float HalfY = InCamera.ViewSize.Y * 0.5f / InCamera.Zoom;
    const float X = std::fabs(std::cos(InCamera.Rotation));
    const float Y = std::fabs(std::sin(InCamera.Rotation));
    const float ExtentX = HalfX * X + HalfY * Y;   // rotated AABB
    const float ExtentY = HalfX * Y + HalfY * X;
```

```
    for (std::int64_t Ty = MinY; Ty <= MaxY; ++Ty)
        for (std::int64_t Tx = MinX; Tx <= MaxX; ++Tx)
        {
            const std::uint32_t Layer = LayerForTile(Tx, Ty);
            const std::uint32_t Hash = FindHash(Tx, Ty);
            if (Hash != kHashCapacity)
            {
                MSlots[Layer * 1024u + MHash[Hash].slot].lastUsedFrame = MFrame;
                continue;
            }
            std::uint32_t Slot = 0;
            if (LoadTile(Tx, Ty, Layer, &Slot))   // mmap chunk, upload one
64x64 tile
            {
                MSlots[Layer * 1024u + Slot].lastUsedFrame = MFrame;
                ++MStats.uploads;               // one vkCmdCopyBufferToImage
            }
        }
}
```

Поле заголовка чанка	Смещение	Значение	Проверка
magic	0	FBYVTC01	Отклоняется до индексации отображения
version	8	1	Будущий формат отклоняется
tileSize	12	64	Должен совпадать с дескриптором
chunkTiles	16	4	Должен совпадать с дескриптором
format	20	0 = RGBA8	Без декодирования в рантайме
pixels	32	256 x 256 x 4	Точная проверка размера перед subspan

- Вытеснение — часовой алгоритм на слой. Страница, у которой lastUsedFrame равен текущему кадру, видима и никогда не становится жертвой; если все 1024 страницы пула видимы, запрос считается spill вместо порчи тайла, находящегося на экране.
- Поиск резидентности — открытая адресация с tombstone. Попадание стоит одного зонда и записи lastUsedFrame; отображение чанка никогда не перечитывается для резидентного тайла.
- Сам VirtualTextureSystem занимает около 2 МиБ (таблица страниц и хеш на 65536 записей). Это стартовый объект, один раз выделяемый в куче, а не стековая переменная кадровой функции.

31.2 Маски изменений компонентов. Видимая для рендера сущность несёт ComponentChangeMask рядом с Transform2D/Sprite2D. Система, меняющая данные, вызывает change::MarkTransform(), MarkSprite() или MarkDrawOrder(); отметка — один атомарный fetch_or по 64-битному слову. Extract читает слово, потребляет его через fetch_and(0) и пересчитывает RenderItem только если присутствовал один из трёх рендер-битов. Сущность с нулевой маской стоит одного поиска указателя и одной relaxed-загрузки — без чтения Transform, копирования Sprite и вычисления RenderKey.

Бит	Имя	Потребитель	Эффект
0	Transform2D	ChangeMaskRenderExtract	Пересчёт position/scale/rotation; ключ не меняется
1	Sprite2D	ChangeMaskRenderExtract	Пересчёт UV/size/colour/layer
2	DrawOrder	ChangeMaskRenderExtract	RenderKey может измениться: структурная пересборка
3-9	Velocity/ GameplayState/ Animation/Audio/ Physics/Lifetime/ Custom	Игровые системы	Потребляется владельцем; GPU ничего не видит
16-63	GameBitBase + n	Игровой код	Зарезервировано под алиасы проекта

```
struct ComponentChangeMask          // 16 bytes, align 8
{
    ChangeMask   Bits      = 0;     // offset 0, atomic_ref target
    std::uint32_t LastFrame = 0;    // offset 8, diagnostic only
    std::uint32_t Reserved  = 0;    // offset 12, zero
};
static_assert(sizeof(ComponentChangeMask) == 16);

// A system marks after the write, not before: the extract must never see a bit
// for a value it has not finished writing.
transform.Position = NewPosition;
Fby::Change::MarkTransform(registry, entity, frameIndex);

// Extract: static items fall through after one relaxed load.
for (auto [e, mask, transform, sprite] : view.each()) {
    const ChangeMask Bits = change::Peek(mask);
    if (Bits == 0 && SlotKnown(e)) { ++stats.skippedStatic; continue; }
    const ChangeMask Hit = change::Consume(mask);
    if ((Hit & kRenderBits) == 0) { continue; }
    BuildItem(registry, e, &items[slot], &keys[slot]);
}
```

```
// Output-relative dirty ranges coalesced in one sorted pass.
struct DirtyRange { std::uint32_t first; std::uint32_t count; };
static_assert(sizeof(DirtyRange) == 8);

// Transform-only edit: key unchanged, order unchanged, patch one output item.
MOutputItems[MOutputIndex[slot]] = MItems[slot];
MChangedScratch[MChangedCount++] = MOutputIndex[slot];
// ... sort + coalesce into 1..N ranges; the backend uploads only those bytes.

// DrawOrder edit: the item can move in the sort. The safe upload is the whole
// sorted range, because firstItem in every DrawBatch may have shifted.
if (OldKey != NewKey) { RebuildOutput(); return DirtyRange{0, MOutputCount}; }
```

FillPacket() публикует те же диапазоны как FramePacket::instanceRanges и выставляет instanceRangesAreDeltas. VulkanRHI копирует только эти диапазоны в постоянный срез instance-буфера кадра после того, как срез получил один полный образ; первое использование, resize или структурная пересборка откатываются к полному копированию. MetalRHI сохраняет полное копирование как fallback, поэтому контракт пакета переносим, а оптимизация явная.

- Без аллокаций: каждый слот, ключ, батч, диапазон и бакет поиска живут внутри объекта, параметризованного Capacity. Значение по умолчанию 4096 покрывает большую 2D-сцену; стриминговый мир инстанцирует ChangeMaskRenderExtract<1u << 18> без изменения API.
- Extract отдаёт оба представления: Items()/Keys()/Batches() заполняют существующий FramePacket, а DirtyRanges() — план загрузки для бэкенда с постоянным instance SSBO. Кадр без изменений и без структурных правок даёт пустой список диапазонов.
- Смена слоя намеренно считается структурной, даже если float-глубина не изменилась: батчинг группирует по старшим 16 битам RenderKey, поэтому стабильный порядок — условие корректности, а не оптимизация.

31.3 Сетка сетевой релевантности. NetRollbackSession владеет стабильной картой NetEntityId; RelevanceGrid владеет mdspan ячеек. Сетка пересобирается раз в тик: сущности сортируются по NetEntityId, позиция Transform2D выбирает ячейку, и каждая ячейка добавляет фиксированное число id. План для игрока обходит только квадрат ячеек внутри farRadius. Текущая и соседние ячейки — немедленные; ближнее кольцо ограничено nearInterval; дальнее — farInterval. Фаза равна NetEntityId::value % interval, поэтому дальние объекты не обновляются все в один тик.

Полоса	Расстояние Чебышёва	Частота	Типичный payload
Немедленная	0..neighborRadius (по умолчанию 1)	каждый тик	игрок, ближние актёры, двери
Ближняя	neighborRadius+1..nearRadius	каждые nearInterval тиков	средние объекты
Дальняя	nearRadius+1..farRadius	каждые farInterval тиков	фоновые движущиеся объекты, якоря VFX

Внешняя	> farRadius	никогда	нет в плане

```
// The cell grid is an mdspan over one flat vector<RelevanceCell>.
std::mdspan<RelevanceCell, std::dextents<std::size_t, 2>>
    Grid(MCells.data(), MConfig.cellsY, MConfig.cellsX);

// Rebuild once per tick, deterministically sorted by NetEntityId.
InEntities.ForEach([&](NetEntityId Net, Entity Local) noexcept {
    MSortScratch[Count++] = SortableEntity{Net.Value, Local};
});
std::sort(MSortScratch.begin(), MSortScratch.begin() + Count,
          [](const auto& A, const auto& B) { return A.id < B.id; });
for (const SortableEntity& E : span(MSortScratch.data(), Count)) {
    const Transform2D& T = InRegistry.get<Transform2D>(E.local);
    Grid[CellY(T.Position), CellX(T.Position)].Append(NetEntityId{E.id});
}

// Plan: walk only cells inside farRadius; the bitset is O(1) membership.
const RelevanceBand Band = BandFor(std::abs(Cx - PlayerCx), std::abs(Cy -
PlayerCy));
const std::uint32_t Interval = Band == RelevanceBand::Immediate ? 1u
    : (Band == RelevanceBand::Near ? Config.nearInterval : Config.farInterval);
if (!IsDue(Interval, Id, Tick)) { continue; }
OutPlan.items[OutPlan.count++] = RelevanceEntity{Id, Cx, Cy, Band};
OutPlan.Set(Id);
```

```
// Filtering is a record copy, not a registry walk. The output is a valid
// Snapshot v4 delta against the same base: checksum and counts are rebuilt.
for (std::uint32_t R = 0; R < View.header.entityCount; ++R) {
    const FilteredRecordHeader& Rec = ReadRecord(View.payload, Offset);
    const bool Removed = (Rec.Flags & SnapshotEntityRemoved) != 0;
    const NetEntityId Net = MEntities.IdOf(Rec.entity);
    const bool Keep = Removed ? InKeepRemovals : InPlan.Contains(Net);
    if (Keep) { CopyRecord(Rec, Out); ++Out.header.entityCount; }
}
Out.header.checksum = Fnv1a(OutPayload);
// Client: RestoreDelta(base, filtered) - each record still restores the
// component bytes that the server captured for that tick.
```

Поле RelevancePlan	Размер	Назначение
player, tick	16 Б	Идентифицирует представление для отправки
items[4096]	64 КиБ	Отсортированные записи RelevanceEntity
bits[64]	512 Б	Один бит на NetEntityId для O(1)-фильтрации
счётчики immediate/near/far	12 Б	Диагностика и бюджет

		трафика

31.4 Иерархические аудио-шины. AudioZoneComponent — авторизованный объём Box2D-сенсора: AABB, скопированный из формы, стабильный zoneId, приоритет, маска шин и два целевых параметра ReverbGain и LowPassHz. AudioContextComponent — разрешённое состояние слушателя. AudioZoneSystem выбирает зону с наибольшим приоритетом, интерполирует контекст в игровом времени однополюсным ramp и отправляет не более трёх фиксированных команд (Master, SFX, Music). Затем аудиопоток плавно меняет коэффициент по сэмплам в ApplyFilter и gain возврата реверба в ApplyReverb; ни одна из функций не аллоцирует, не блокирует и не обращается к ECS.

Шина	Флаг	Параметр зоны	Стадия аудиопотока
Master	бит 0	ReverbGain, LowPassHz	ApplyFilter -> ApplyReverb -> ApplyBusGain
SFX	бит 1	ReverbGain, LowPassHz	Та же цепочка, независимо
Music	бит 2	ReverbGain, LowPassHz	Та же цепочка, независимо
Cave (пример)	Master+SFX	0.45, 900 Гц	Затухание контекста 0.1 с, затем ramp по сэмплам
Underwater (пример)	Все	0.20, 450 Гц	Приоритет 9 побеждает Cave

```
struct AudioZoneComponent                  // authored, trivially copyable
{
    Vector2D min, max;                    // Box2D sensor AABB, world units
    std::uint32_t zoneId, kind, priority, busMask, flags;
    float reverbGain;                     // 0..4, return gain
    float lowPassHz;                      // 20..20000 one-pole cutoff
    float fadeSeconds;                    // context cross-fade time
};

struct AudioContextComponent              // resolved per listener
{
    std::uint32_t activeZoneId, previousZoneId, activeKind, flags;
    float blend, reverbGain, lowPassHz;
    float targetReverbGain, targetLowPassHz, fadeSeconds;
    Vector2D listenerPosition;
    float lastSentReverbGain, lastSentLowPassHz;
    std::uint32_t lastBusMask;
};

// Zone system: retarget and ramp the game-side context.
const float Alpha = 1.0f - std::exp(-dt / Fade);
```

```
Context.reverbGain += (Context.targetReverbGain - Context.reverbGain) * Alpha;
Context.lowPassHz  += (Context.targetLowPassHz  - Context.lowPassHz)  * Alpha;
Context.blend = Saturate(Context.blend + dt / Fade);
Audio.SetBusZoneParameters(AudioBus::Sfx, Context.reverbGain, Context.lowPassHz,
Fade);
```

```
// Audio thread: coefficient and reverb gain are ramped per sample.
const float Start = ScopedCoefficient(CurrentCutoff);
const float End   = ScopedCoefficient(TargetCutoff);
for (std::uint32_t F = 0; F < Frames; ++F) {
    const float C = Start + (End - Start) * T(F);
    State[0] += C * (Buffer[F * 2 + 0] - State[0]);
    State[1] += C * (Buffer[F * 2 + 1] - State[1]);
    Buffer[F * 2 + 0] = State[0];
    Buffer[F * 2 + 1] = State[1];
}
// Reverb return: value = wet * wetParams * zoneGainRamp(F).
// Command ring: SetBusFilter, fixed size, no lock on the producer side.
```

Четыре конвейера покрыты headless-тестом advanced_pipelines. Он монтирует реальный каталог, пишет реальные .fvtc-чанки и проверяет, что повторный вид даёт ноль загрузок; проверяет, что статичная сущность даёт пустой dirty range; строит реальную Snapshot v4 дельту и проверяет, что FilterDeltaForPlayer сохраняет одну из двух записей; и прогоняет underwater-зону с приоритетом 9 до 450 Гц и обратно до открытого фильтра. GPU и аудиоустройство не открываются, поэтому тест работает на headless CI-агенте.

33. Продакшн-дистрибуция: шифрование, сжатие LZMA и чек-лист релиза Steam/EGS/GOG

Глава 32 закрывает разрыв между изолированными консольными утилитами и редактором. До неё в дереве были fby_fempak (cook, secure, list, extract), FemArchiver (LZMA/FbyLZ и ChaCha20-Poly1305/AES-256-GCM), FemCrypto, AotBuilder и кнопка 'Cook AOT' в отдельной панели, но не было одной операции, запускающей всё это как релиз. Класс ReleaseCooker теперь владеет этой операцией: одна кнопка Build & Pack Game запускает native-плагин логики, упорядоченный кук ассетов, детерминированный .fempak и зашифрованный .femarc, а затем публикует прогресс в SceneDiagnosticsPanel.

Стадия	Команда / класс	Пишет	Контракт ошибки
Ключ	ReleaseCooker::PrepareKey	.fbyrelease.seed, release.key.tmp, ReleaseKey.generated.hpp	Нет открытого ключа в project.femproj или editor.ini
AOT	script::AotBuilder + forceIncludeHeader	logic_graph.cpp / .dll	Пропускается без LogicGraph, без имитации
Кук	fby_fempak cook --	assets.fempak +	Ненулевой код

	progress	cook.progress	дочернего процесса валит сборку
Защита	fby_fempak secure --key-file --progress	game.femarc	Ошибка AEAD-тега или отсутствие ключа отклоняются
Прогресс	CookProgressRecord -> ReleaseCookProgress	cook.progress / secure.progress	Разорванная запись отклоняется двойным чтением

31.1 ReleaseCooker. Класс находится в Source/Editor/Infrastructure/ReleaseCooker.hpp и связывает только уже существующие сервисы редактора. Start() проверяет пути, создаёт дерево релиза, копирует POD LogicGraph и запускает один std::jthread. Worker вызывает AotBuilder, затем кукер и secure-шаг, а поток редактора раз в кадр вызывает Progress(). Копия для UI — фиксированный POD под коротким mutex; worker никогда не вызывает ImGui, а редактор не блокируется на компиляторе или компрессоре.

```
// One click from the toolbar: Build menu or the Viewport button.
ReleaseCookConfig Config = ReleaseCooker::FromProject(
    projectConfig.Paths(), projectConfig.Project(), EngineRoot, FempakPath);
Config.compileAot = visualScript.Initialized() && visualScript.Graph().nodeCount
!= 0;
char Error[256]{};
if (!releaseCooker.Start(Config, Graph, Error, sizeof(Error)))
{
    SetStatus(Error);
}

// SceneDiagnosticsPanel every frame. No allocation while drawing.
releaseCooker.Poll();
sceneDiagnostics.SetReleaseProgress(releaseCooker.Progress());

// The worker owns the sequence, in this order:
//   PrepareKey -> AotBuilder -> fby_fempak cook -> fby_fempak secure
```

32.2 Запекание ключа. Раньше secure требовал --key или --key-env, и в редакторе не было вызывающего. Теперь ReleaseCooker читает проектный 32-байтный seed из .fbyrelease.seed (создаётся через SecureRng при отсутствии), пишет build-scoped бинарный ключевой файл для fby_fempak и force-include'ит сгенерированный заголовок в AOT-единицу трансляции. Заголовок экспортирует C-ABI функцию, восстанавливающую ключ из двух XOR-массивов; ключ находится в плагине, а в проектном файле нет пароля. Seed — единственный постоянный секрет; его удаление меняет release-ключ на следующей сборке.

```
// Generated into build/release/ReleaseKey.generated.hpp and force-included
// into the AOT plugin (clang-cl /FI, clang/g++ -include).
FBY_RELEASE_KEY_EXPORT bool fby_release_archive_key(std::uint8_t* OutKey,
                                                    std::uint32_t InCapacity)
noexcept
{
    if (OutKey == nullptr || InCapacity < 32u) return false;
```

```
    constexpr std::uint8_t kMasked[32] = { /* key ^ mask */ };
    constexpr std::uint8_t kMask[32]   = { /* build mask */ };
    for (std::uint32_t i = 0; i < 32u; ++i)
        OutKey[i] = static_cast<std::uint8_t>(kMasked[i] ^ kMask[i]);
    return true;
}

// A launcher can resolve the same plugin and call the export before
// opening game.femarc. No password is read from ini or the environment.
```

Файл	Время жизни	Содержит	При успехе
.fbyrelease.seed	Постоянный секрет проекта	32 случайных байта	Хранится, права задаёт хост
release.key.tmp	Только одна сборка	Открытый AEAD-ключ для fby_fempak	Перезаписывается нулями и удаляется
ReleaseKey.generated.hpp	Только компиляция	Маскированный ключ + C-ABI экспорт	Удаляется
archives/ game.femarc	Поставляемый артефакт	LZMA/FbyLZ + ChaCha20/AES тело	Хранится

32.2a Project-local дерево вывода. Проект, созданный дашбордом, владеет всеми своими артефактами: промежуточные файлы сборки и release-ключ лежат в <root>/build/release, канонический архив — <root>/archives/game.femarc, а staging для магазинов — <root>/stores/{steam,epic,gog}. После успешной сборки ReleaseCooker копирует архив и AOT-плагин в каждый store-каталог и пишет манифест store.txt (project, archive, plugin, cipher, key_baked). Перенос или удаление каталога проекта переносит сборку, архив и все store-копии вместе; ничего не пишется в дерево движка или рабочий каталог процесса.

32.3 Протокол прогресса. fby_fempak пишет 424-байтную CookProgressRecord в --progress. Запись — POD: стадия, completed/total, текущий ассет, raw/stored байты, микросекунды, байты/сек, шифр и флаги. Writer ищет нулевое смещение, пишет всю запись и делает flush после каждого обновления; ReleaseCooker читает её дважды и принимает только совпадающую последовательность, поэтому частично записанная запись не даёт ложные 90%. ArchiveBuildOptions несёт обычный function-pointer колбэк прогресса, не std::function, поэтому FemArchiver сообщает сжатые байты по записям внутри noexcept-сборки.

```
struct CookProgressRecord                       // 424 bytes, ABI-stable
{
    char magic[8];                              // FBYCOOK1
    std::uint32_t version, sequence;
    std::uint32_t stage, completed, total, bytesPerSecond;
    std::uint32_t cipher, flags, reserved;
    std::uint64_t rawBytes, storedBytes, elapsedMicroseconds;
    char asset[192];
    char message[160];
};

// SceneDiagnosticsPanel: the only UI work per frame.
```

```
ImGui::ProgressBar(Progress.Fraction(), ImVec2(-1.0f, 0.0f), Overlay);
ImGui::Text("asset: %s", Progress.asset);
ImGui::Text("LZMA %.2f MB/s", Progress.bytesPerSecond / (1024.0 * 1024.0));
ImGui::Text("encrypted: %s", CookCipherName(Progress.cipher));
```

- Текущий ассет точен: fby_fempak обновляет запись до и после каждого кука, поэтому бар показывает обрабатываемую текстуру, звук или локализацию, а не общую надпись 'cooking'.
- Состояние шифрования точно: cipher 1/2 и флаг encrypted копируются из secure-сборки; отсутствующий ключ или отклонённый AEAD-тег завершают стадию как Failed, а не Succeeded.
- Скорость LZMA измеряется по байтам прогресса архиватора и прошедшему времени. Поскольку вызов 7-Zip SDK сжимает одну запись .fempak целиком, скорость сэмплируется на границе записи, а не каждые 64 КиБ; UI помечает значение как измеренное, а не предсказанное.

32.4 Чек-лист релиза. Одни и те же артефакты идут в Steam, Epic Games Store и GOG. Дерево кука проектно-специфично и воспроизводимо: фиксированный seed и детерминированный писатель .fempak делают хеш архива штампом сборки; .femarc хранит настройки LZMA и AEAD в заголовке; AOT-плагин записывает хеши графа и раскладки компонентов. Для сабмита в магазин следует приложить хеш game.femarc, плагин, идентификатор seed (не сам seed) и снимок BuildProfile.

Магазин	Нужный артефакт	Проверка ReleaseCooker	Ручной шаг
Steam	game.femarc + плагин + depot VDF	Заголовок LZMA/AEAD, хеш архива	Загрузка SteamPipe, build id
Epic Games Store	game.femarc + плагин + EOS manifest	те же байты, хеш EOS-артефакта	EOS BuildPatch, sandbox
GOG Galaxy	game.femarc + плагин + Galaxy shim	те же байты, ABI шима	Сборка Galaxy, облачные сохранения
Все магазины	Идентификатор seed, BuildProfile	Хеш графа + LayoutHash компонентов	Хранить seed офлайн, менять при утечке

Оркестратору сборки нужен исполняемый файл fby_fempak в build/bin. Если редактор собран с FBY_BUILD_TOOLS=OFF, Start() завершается до запуска процессов, а статус-бар называет отсутствующий упаковщик. Стадия AOT необязательна по конструкции: проект без привязанного LogicGraph всё равно кукает и шифрует ассеты, а результат сообщает keyBaked=false.

34. SystemGraph: контракт параллельного планирования, затем контракт циклов блендинга

G8 превращает SystemGraph::Run из обхода в порядке регистрации в диспетчер, знающий зависимости. Build() уже сворачивает каждое before-ребро в after-маску; теперь он ещё вычисляет longest-path уровень каждого узла. Два узла одного уровня не могут зависеть друг от друга, поэтому уровень — безопасная параллельная волна. Run(phase) запускает все включённые узлы волны через JobSystem, оставляет один узел вызывающему и дожидается волны через counting_semaphore на стеке вызывающего. Здесь нет std::function, аллокаций и общего семафора на систему; пул из 24 воркеров выполняет 24 независимые системы в одной волне, а не 24 последовательных вызова.

```
// Build(): longest-path level, same-phase edges only.
for (Position in TopologicalOrder) {
    Node = Order[Position];
    Level = 0;
    for (Dep in DependencyMask(Node))
        if (Phase(Dep) == Phase(Node)) Level = max(Level, MLevels[Dep] + 1);
    MLevels[Node] = Level;
}

// Run(Phase): one wave, one per-call semaphore.
std::counting_semaphore<kMaxGraphSystems> Done{0};
for (I = 1; I < WaveSize; ++I) {
    Wave[I].completed = &Done;
    Jobs.Submit(&InvokeGraphNode, &Wave[I], 0, 0);   // no allocation
}
Wave[0].run(Context, Wave[0].user);                    // caller keeps one node
for (I = 1; I < WaveSize; ++I) Done.acquire();          // join before next
level
```

Гарантия	Почему это выполняется
Узел никогда не стартует раньше всех after/before зависимостей	Level = max(level зависимости) + 1; Run дожидается волны до следующего уровня
Узлы одной волны не зависят друг от друга	Ребро зависимости всегда повышает уровень зависимого узла
Ни один воркер не теряется и не ожидается дважды	Один release на отправленный thunk, один acquire на отправку, семафор на стеке
Render остаётся на render-потоке	Run(Render) идёт последовательным путём: командный поток RHI однопоточный
Мутация инвалидирует расписание	Add/RemoveUser/Clear ставят MBuilt = false; Run ничего не делает до успешного Build()

Маска зависимостей — это весь контракт гонок данных. Две системы, пишущие один пул компонентов, обязаны иметь ребро; «они случайно зарегистрированы в таком порядке» — не зависимость. Цепочка FixedUpdate редактора (events_clear -> movement -> physics -> contacts -> spriteAnimation -> skeletalAnimation_animate) поэтому объявляет пять after-

битов, а независимые узлы PostUpdate (skeletalAnimation_evaluate, physics_interpolate, cloth collect) образуют одну волну. JobSystem — часть фикса: MPending и MRunning разнесены на разные 64-байтные линии, поэтому трафик submit/completion с разных ядер не трясёт одну кэш-линию, а ParallelFor использует семафор на вызов вместо общего члена.

CompileGraph() — не сериализатор, а последнее место, где авторский граф можно отвергнуть до того, как AnimStateComponent станет рантайм-состоянием. Контракт двухслойный. Локальный: Clip-узел владеет одним ненулевым state id, Float-узел — одним уникальным индексом параметра, Blend-узел — ровно одним source-линком, ровно одним target-линком и не более чем одним parameter-линком; состояния source и target обязаны различаться. Глобальный: в скомпилированном графе переходов не должно быть цикла, который может срабатывать вечно без выхода.

Правило	Результат компилятора	Действие автора
Состояние клипа равно 0 или kAnimAnyState	Отказ: needs a non-zero state id	Назначить уникальный 64-битный id
Состояние source == состояние target	Отказ: cross-fades a state into itself	Выбрать два разных клипа
У бленда 0 или 2 source/target линка	Отказ: exactly one source, one target	Удалить лишнюю связь
Цикл горячих переходов без выхода	Отказ: dead blend cycle without an exit	Добавить gated-выход или убрать петлю
Цикл с любым включённым выходом	Принято	Держать выход раньше горячего ребра

Переход «горячий», если он Enabled, не имеет tag/float-условия и его StateTime-предикат уже истинен при StateTime == 0: StateTimeAtLeast с порогом <= 0 и/или StateTimeAtMost с порогом >= 0. AnimGraphSystem сбрасывает StateTime в ноль на каждом сработавшем переходе и добавляет dt до вычисления следующего тика, поэтому горячее ребро срабатывает при каждом входе. Горячий цикл — мёртвая петля только если ни один включённый переход не выходит из множества его состояний; gated-выход сохраняет граф легальным, даже если сама петля горячая.

```
// Iterative three-colour DFS. Bounded by kMaxAnimClips == 8; no recursion.
std::array<std::uint8_t, kMaxAnimClips> Color{};   // 0 white, 1 gray, 2 black
std::array<std::uint16_t, kMaxAnimClips> Stack{};
for (std::uint16_t Start = 0; Start < N; ++Start) {
    if (Color[Start] != 0) continue;
    std::int32_t Top = 0;
    Stack[0] = Start; Color[Start] = 1;
    while (Top >= 0) {
        const std::uint16_t V = Stack[Top];
        std::uint32_t& Edges = HotEdges[V];
        if (Edges == 0) { Color[V] = 2; --Top; continue; }
        std::uint16_t W = 0;
        while (((Edges >> W) & 1u) == 0) ++W;
        Edges &= ~(1u << W);
        if (Color[W] == 0) { Color[W] = 1; Stack[++Top] = W; continue; }
```

```
        if (Color[W] == 1) {
            // W is on the gray stack: reconstruct the cycle, test for an exit.
            if (!HasEnabledExit(Stack, Top, W)) {
                SetStatus("Anim compile: dead blend cycle without an exit: %s",
Path);
                LogLine(LogLevel::Warn, "AnimGraph", MStatus);
                return false;
            }
        }
    }
}
```

Валидатор работает по скомпилированному множеству рёбер, а не по документу ImNodes, поэтому видит ровно то, что будет вычислять рантайм: выключенные линки уже отфильтрованы AnimTransitionMatches, а state id разрешены из линков. Предупреждение идёт через LogLine (не std::format), поэтому путь ошибки остаётся noexcept, и LogBoy всё равно получает запись. Полностью циклический документ — максимум 8 вершин: явный стек переполнить нельзя, худший случай — 64 прохода по рёбрам.

35. Очистка Box2D при откате, затем PIE-передача анимации

R1 — ошибка владения, а не ошибка Box2D. Registry::clear() уничтожает ECS-объекты; он не знает, что RigidBody2D::body — нативный b2BodyId, поэтому старые тела остаются в мире, а восстановленные строки снапшота несут body == 0. Следующий EnsureBodies()/Step() создаёт второе тело для каждой сущности. Фикс упорядочен: (1) попросить PhysicsSystem уничтожить все отслеживаемые живые тела и просканировать реестр на тела joint-цепочек, созданные не им; (2) очистить и восстановить снапшот; (3) пересоздать тела из восстановленных байтов. Тело уничтожается только после b2Body_IsValid, поэтому joint-система, уже убравшая свои якоря, безопасна.

```
NetRollbackSession::Result NetRollbackSession::RestoreSnapshot(
    std::span<const std::byte> InSnapshot) noexcept
{
    if (!MInitialized || MRegistry == nullptr || MSchema == nullptr)
        return Result::NotInitialized;

    // R1: native bodies die BEFORE Registry::clear().
    if (MPhysics != nullptr)
        (void)MPhysics->DestroyAllBodies(MRegistry);

    if (!ecs::RestoreSnapshot(*MRegistry, *MSchema, InSnapshot, &MRng))
        return Result::RestoreFailed;

    // Snapshot bytes never carry a b2 handle: rebuild from the restored rows.
    if (MPhysics != nullptr)
        MPhysics->EnsureBodies(*MRegistry);
    return Result::Ok;
}
```

```
// PhysicsSystem::DestroyAllBodies: tracked pass + registry sweep.
for (Record : MBodies)
    if (b2Body_IsValid(Load(Record.body)))
        { b2DestroyBody(Load(Record.body)); ++Destroyed; }
MBodies.clear();
for (Body : Registry.view<RigidBody2D>())
    if (Body.body != 0) {
        if (b2Body_IsValid(Load(Body.body))) { b2DestroyBody(Load(Body.body)); +
+Destroyed; }
        Body.body = 0;                       // no stale handle survives
    }
for (Collider : Registry.view<Collider2D>()) Collider.shape = 0;
```

Фаза restore	Нативное состояние	Инвариант после фазы
DestroyAllBodies(registry)	Все валидные tracked-тела + все тела реестра уничтожены	В мире нет тела, принадлежащего старому кадру
Registry::clear() + декод снапшота	Нативные объекты не трогаются	Восстановленные строки RigidBody2D имеют body == 0
EnsureBodies(registry)	Одно b2Body/b2Shape на восстановленную строку	Число тел равно предроллбэковому, а не удвоенному
Реплей до текущего тика	Step() видит body != 0 и не создаёт тело	Никакого дублирования физики на реплейных тиках

Play-In-Editor применяет то же правило на другом уровне. Клон владеет отдельным PhysicsSystem, поэтому Play никогда не вставляет тела клона в мир редактора. Stop вызывает DestroyAllBodies(clone), затем гасит этот мир; авторская сцена не менялась, поэтому Stop — откат за константное время без осиротевших тел. Именно этот путь делает кнопки Play/Pause/Stop честными: они запускают реальную симуляцию, а не флаг предпросмотра.

Холсту нужны шесть скаляров на кадр: CurrentState, PreviousState, BlendWeight, BlendTime, StateTime и LastTransition. Чтение их из живого AnimStateComponent в потоке ImGui — не безобидная гонка: воркеры SystemGraph могут находиться внутри AnimGraphSystem::Run, а сам поиск в EnTT небезопасен, пока создаются/уничтожаются пулы компонентов. Атомарное чтение шести полей не решило бы часть с реестром и временем жизни. Поэтому панель владеет 64-байтным POD и двумя слотами seqlock: поток симуляции публикует, поток редактора потребляет, а живой компонент никогда не разыменовывается из Draw().

```
struct PIEAnimFrame {                        // exactly 64 bytes
    std::uint64_t CurrentState = 0;
    std::uint64_t PreviousState = 0;
    float BlendWeight = 0.0f, BlendTime = 0.0f, StateTime = 0.0f;
    std::uint16_t LastTransition = kNoAnimTransition;
    std::array<std::byte, 34> Reserved{};
};
```

```
// Producer (simulation thread, after the FixedUpdate join):
void PublishPIEState(const AnimStateComponent& S) noexcept {
    const std::uint32_t Write = MPieWriteSlot.load(std::memory_order_relaxed);
    PIEFrameSlot& Slot = MPieSlots[Write ^ 1u];
    const std::uint64_t Seq = Slot.Sequence.load(std::memory_order_relaxed);
    Slot.Sequence.store(Seq + 1u, std::memory_order_relaxed);
    std::atomic_thread_fence(std::memory_order_release);
    StoreRelaxed(Slot.Frame.CurrentState, S.CurrentState);
    StoreRelaxed(Slot.Frame.PreviousState, S.PreviousState);
    StoreRelaxed(Slot.Frame.BlendWeight, S.BlendWeight);
    StoreRelaxed(Slot.Frame.BlendTime, S.BlendTime);
    StoreRelaxed(Slot.Frame.StateTime, S.StateTime);
    StoreRelaxed(Slot.Frame.LastTransition, S.LastTransition);
    std::atomic_thread_fence(std::memory_order_release);
    Slot.Sequence.store(Seq + 2u, std::memory_order_release);
    MPieWriteSlot.store(Write ^ 1u, std::memory_order_release);
    MPiePublished.store(1u, std::memory_order_release);
}

// Consumer (ImGui/Render thread): retry only if the sequence changed.
const std::uint64_t Before = Slot.Sequence.load(std::memory_order_acquire);
if ((Before & 1u) == 0) {
    ReadRelaxed(Frame.CurrentState, Slot.Frame.CurrentState);
    /* ... five more ... */
    std::atomic_thread_fence(std::memory_order_acquire);
    if (Before == Slot.Sequence.load(std::memory_order_relaxed)) return true;
}
```

Операция	Упорядочивание	Почему
инкремент sequence (нечётный)	relaxed + release fence	Помечает слот как записываемый; fence не даёт payload-записям уехать раньше
записи payload	relaxed atomic_ref	Нет гонки данных; порядок задают скобки sequence
инкремент sequence (чётный)	release store	Публикует payload до того, как читатель сможет его принять
публикация слота writer	release store	Читатель берёт индекс слота с acquire и видит целый кадр
повтор читателя	acquire + relaxed re-check	Отвергает рваное чтение, если writer обогнал reader

- BindRuntimeWorld() инвалидирует буфер только при реальной смене указателей мира или флага PIE; EditorSystem вызывает его каждый кадр, поэтому безусловная инвалидация стирала бы sample до Draw().

- PublishPIEStateFromWorlds() — удобный producer: находит EditorSelected, берёт AnimStateComponent из runtime-мира и публикует. Он обязан работать на потоке симуляции, никогда из Draw().
- RefreshPIEState() и DrawPIEInspector() читают только слот seqlock. Инспектор больше не обходит Mirror-таблицу по живому указателю; он печатает опубликованные скаляры.
- Передача добавляет ровно один кадр задержки и одну 64-байтную копию на опубликованный кадр. Она ничего не аллоцирует и никогда не берёт мьютекс.

Если PIE позже начнёт шагать на пуле воркеров, хост обязан публиковать после join пула и до отрисовки редактора. Не вызывайте PublishPIEState() из render-потока, пока воркеры ещё работают: producer трогает реестр и живой компонент, а они не защищены seqlock. Seqlock защищает передачу, а не снятие.

36. Глобальный аудит: блокирующие находки и гейт

Исторический снимок. Таблица заморожена на аудите 2026-10-27: часть находок уже исправлена, а живой статус с файлами и строками — Documentation/ROADMAP.md. Не планируйте работы по этой таблице.

Аудит покрыл VulkanRHI/MetalRHI, NetRollbackSession, Snapshot v4, Box2D-remap, погодный compute-путь, VFS и кукер ассетов, JobSystem, SystemGraph, hot reload и анимационный холст. Подтверждено более пятидесяти дефектов. Эта глава фиксирует замороженный снимок для справки, а живой гейт — Documentation/ROADMAP.md.

ID	Severity	Находка	Минимальный фикс
R1	Critical	Restore отката не уничтожает старые Box2D-тела; каждый откат дублирует мир	PhysicsSystem::DestroyAllBodies() внутри NetRollbackSession::RestoreSnapshot(), до Registry::clear()
L1	High	FilterDeltaForPlayer проверяет длину источника, а не ёмкость приёмника	Считать ёмкость из OutFiltered и проверять до каждого memcpy
C1/C2	Critical	Readback идёт до submit кадра; неудачный кадр оставляет fence/semaphore сигнализированными	Писать copy в CB кадра; сбрасывать fence только перед успешным submit
S1	High	ParticleEmitterComponent теряет шесть неотражённых полей при restore	Добавить шесть FBY_FIELD
V2	Critical	Store-кодек копирует управляемый атакующим	Требовать StoredSize == RawSize для Store

		stored.size()	
V3	High	Видеодекодер держит mmap-span VFS через hot reload	Владеть байтами или отдавать shared mapping + generation
G1	Critical	У нового проекта нет CMakeLists/GameMain, его нельзя собрать из дашборда	ProjectScaffold.hpp + CreateProject пишет CMakeLists.txt, project.femproj и GameMain.cpp
G2	Critical	Play/Pause/Stop декоративны; редактор не делает снапшот и не шагает изолированный мир	Toolbar EditorSystem -> PlayInEditor::Start/Step/Stop с приватным физическим миром
G8	High	SystemGraph::Run — последовательный обход независимо от after/before	Уровни зависимостей + волна на уровень + семафор на вызов + разделение кэш-линий
H1	High	Вложенный ParallelFor делит один семафор и может вернуться до завершения helper'ов	Семафор на вызов + TLS-флаг вложенности
H4	High	SystemGraph::Run игнорирует MBuilt; Add/RemoveUser оставляют старый/пустой порядок	Гейт Run по MBuilt; rebuild при мутации
H3 (RHI)	High	Instance-дельты применяются к срезу N-3	Полная копия при framesInFlight > 1 или timestamp срезов
W1	Critical	Splash-счётчик не сбрасывается; всплески прекращаются после capacity	Сбрасывать каждый кадр в pre-pass или ring по модулю capacity

Гейт	Содержимое	Критерий выхода
A — блокирует freeze	R1, L1, C1, C2, S1, V2, V3, G1, G2, G8, H1, H3, H4, W1-W3	У каждого пункта есть фикс или подписанный waiver плюс регрессионный тест
B — до RC	R2, R3, S2-S8, C4-C6, V4-V7, H5/H6, W2/W6, M8-M10	Фикс или задокументированное

		ограничение в release notes
C — после freeze	Low-находки, SoA-разделение, оставшаяся SIMD-работа	Запланировано на 1.1 с профилировочным baseline

Аудит также подтвердил корректные части, которые нельзя «чинить» случайно: канонический порядок и раскладку заголовков снапшота, совместимость Pcg32 с reference, range-checks VFS-пака, границы LZ-декомпрессора, fence-before-acquire, present-семафоры на изображение, учёт дескрипторов, порядок выгрузки плагинов и main-thread контракт TTS. Часть находок латентна, потому что погодные записи пока не потребляются backend'ом; они всё равно станут блокерами, как только этот потребитель появится.

37. Профилирование кэш-миссов и векторизация к 8000+ FPS

8000 FPS — это 125 микросекунд на кадр. Это число имеет смысл только вместе со сценой, разрешением и железом: 2D-батч из нескольких тысяч спрайтов при 1080p — не та же задача, что 4K-кадр с постобработкой. Рассматривайте 8000 FPS как цель для собственной стоимости кадра движка (симуляция + extract + submission) на Zen 5 с прогретым кэшем и всегда сообщайте 99-й процентиль времени кадра, а не только средний FPS. Стабильные 8000+ — это p99 <= 125 мкс, а не один удачный кадр.

- Шаг 1 — baseline. Записывайте CPU- и GPU-время кадра отдельно. На Windows — WPA/ETW или AMD uProf; на Linux — perf stat с cache-references, cache-misses, L1-dcache-load-misses, LLC-load-misses и stalled-cycles-frontend. Сохраняйте числа в репозитории, чтобы регрессия была видна.
- Шаг 2 — найдите горячий массив. Тик анимации идёт по AnimStateComponent на сущность (шаг 4 КБ); extract идёт по RenderItem; пул задач трогает одну общую линию кэша. Шаг 4 КБ с 8 переходами и 8 клипами на компонент вымывает L1 и L2 ещё до вычислений.
- Шаг 3 — разделите hot/cold. Держите 64-байтный AnimHot на сущность (состояния, StateTime, Blend*), а Transitions/Clips/RestPose вынесите в общий объект графа или холодную side-таблицу. Система начинает потоково идти по плотному массиву; общие таблицы остаются в L1.
- Шаг 4 — векторизуйте презентационный blend, а не детерминированную симуляцию. Используйте __restrict, SoA-массивы float (X[], Y[], Rotation[], Scale[]) и проверяйте отчёт векторизатора. Не меняйте порядок FP-редукций, используемых rollback или физикой.
- Шаг 5 — сократите RHI-bind'ы. Сортируйте draw'ы по (pipeline, material, texture array), используйте push constants для draw id, держите один bindless set и записывайте один indirect draw на батч. На 8k FPS churn дескрипторных сетов обычно дороже самого draw.
- Шаг 6 — параллельте фазы, а не только сущности. SystemGraph::Build() один раз считает уровни зависимостей; Run() запускает каждый уровень одной волной через JobSystem и дожидается её семафором на вызов до Render. MPending и MRunning лежат на разных 64-байтных линиях, поэтому трафик submit/completion не трясёт общую линию между 24 воркерами. Оставляйте главный поток для submission и резервируйте одно физическое ядро под render/UI; 23 SMT-воркера на 12 физических ядрах часто проигрывают 11-12 выделенным на cache-bound работе.

```
// SoA blend: one AVX-512 iteration covers 16 bones on Zen 5.
void BlendBones(float* __restrict OutX, float* __restrict OutY,
                const float* __restrict AX, const float* __restrict AY,
                const float* __restrict BX, const float* __restrict BY,
                std::size_t Count, float Weight) noexcept
{
    for (std::size_t I = 0; I < Count; ++I)
    {
        OutX[I] = AX[I] + (BX[I] - AX[I]) * Weight;
        OutY[I] = AY[I] + (BY[I] - AY[I]) * Weight;
    }
}

// Verify the vectoriser actually fired:
// clang-cl /O2 /Rpass=loop-vectorize /Rpass-missed=loop-vectorize
// clang++  -O2 -Rpass=loop-vectorize -Rpass-missed=loop-vectorize
```

Счётчик	Здоровый диапазон	Если хуже
L1d load misses	< 2% загрузок	Hot/cold split, prefetch, уменьшить stride
LLC load misses	< 1-2%	Уменьшить working set, huge pages, переупорядочить циклы
stalled-cycles-frontend	< 10%	Layout веток, inline горячих путей, меньше i-cache thrash
IPC	> 2.5	Проверить цепочки зависимостей, false sharing, конкуренцию за lock
p99 время кадра	<= 125 мкс для 8000 FPS	Ограничить fixed steps, убрать WaitIdle из кадра, бюджетировать submission

Используйте RenderDoc или PIX, чтобы разделить CPU и GPU: CPU-кадр на 8k FPS при GPU-кадре 2 мс всё равно даёт 500 FPS на экране. Не гонитесь за CPU-целью, удаляя корректную работу (валидацию, барьеры, чексуммы) — переносите её в фоновую задачу или более редкий проход. И держите зелёным тест детерминизма rollback после каждого SIMD-изменения: векторизованный порядок float — классический источник desync «у меня работает».

38. Материалы в движке: кук, кэш и кадр

Система материалов была последней подсистемой, рантайм которой оставался на стороне редактора. Модель графа и эмиттер GLSL поставлялись в `fby_engine`, но драйвер, превращающий граф в пайплайн — glslang, SPIR-V, `CreateShaderPipeline` и приватная таблица на 128 записей — поставлялся только в `fby_editor`. Игра линкует лишь `FemboyEngine2D::Engine`, поэтому могла провалидировать материал и не могла им

рисовать. [ADR 0002](../../adr/0002-materials-in-engine.md) переносит недостающую половину в движок и оставляет редактору тонкий адаптер живой перекомпиляции.

Этап	Владелец	Что проверяет
Авторинг: `MaterialGraph`	Панель редактора или инструмент, пишущий `.fgraph`	Один выходной узел, подключённые обязательные пины, совместимость типов, ацикличность, пределы ёмкости
Кук: `fby_matc`	Шаг сборки, линкует glslang	Весь граф, затем каждый сгенерированный GLSL-этап через glslang, затем кодировщик payload
Загрузка: `DecodeCookedMaterial`	Поставленный рантайм, без аллокаций	Magic, версия, размер против длин этапов, оба SPIR-V модуля, согласованность контракта
Acquire: `MaterialPipelineCache`	Поставленный рантайм, один кэш на устройство	Хеш и контракт против резидентных записей, счётчик ссылок, LRU-эвикция, возможность бэкенда
Кадр: `MeshRenderItem::materialPipeline`	Извлечение кадра и бэкенд	Vulkan группирует записи мешей по id и биндит один материальный пайплайн на группу; остальные бэкенды остаются на шейдере движка

Кук материала — одна команда, а результат едет как любой другой кук-ассет:

```
# fby_matc <graph.fgraph> [-o <cooked.femmat>]
build/bin/fby_matc assets/materials/tinted.fgraph -o
assets/materials/tinted.femmat

# The tool refuses a graph that does not validate, prints the hash and the SPIR-
V word
# counts, and never writes a payload the runtime would reject.
```

```
// One cache per device, owned by the game. Acquire at load time.
Render::MaterialPipelineCache ScopedMaterials;

// `InBytes` comes from the VFS or a package entry.
Render::CookedMaterialView ScopedTinted;
if (!Render::DecodeCookedMaterial(InBytes, &ScopedTinted))
{
    LogError("materials", "tinted.femmat is not a cooked material");
    return false;
}
```

```
const Rhi::ShaderPipelineHandle ScopedTintedPipeline =
    ScopedMaterials.Acquire(InRhi, ScopedTinted);

// A missing pipeline is not a crash: the item falls back to the engine shader.
ScopedItem.materialPipeline = ScopedTintedPipeline.IsValid()
    ? ScopedTintedPipeline.id
    : Rhi::kDefaultMaterialPipeline;

// At shutdown, while the device is still alive.
ScopedMaterials.Shutdown(InRhi);
```

- Удержание вместо пересоздания: пайплайн с нулём ссылок остаётся резидентным, иначе игра, берущая материал каждый кадр и отпускающая его в конце кадра, пересобирала бы пайплайн шестьдесят раз в секунду. Стоимость ограничивают предел в 128 записей и LRU-эвикция.
- Маска контракта — часть проверки идентичности. Хеш графа, приходящий с другим контрактом, пока запись жива, отклоняется; тот же хеш с другим контрактом без ссылок заменяется на месте — так редактор обрабатывает смену поддержки push-констант.
- Бэкенд сам говорит, умеет ли он собирать рантайм-пайплайн. Vulkan возвращает true; Null и Metal возвращают false и остаются на шейдерах движка, поэтому игра под них падает при загрузке с читаемой ошибкой, а не рисует несвязанным пайплайном.
- На этом пути ничего не аллоцирует и не форматирует текст покадрово: кэш — фиксированный массив, декодирование — view, ошибка — фиксированный буфер.

Редактор сохраняет живой опыт через тот же кэш. `MaterialGraphRuntime` хранит указатель на граф на вкладку, перекомпилирует при смене `Hash()`, держит последний удачный пайплайн, пока правка невалидна, и отдаёт ошибку по вкладке. Своей таблицы пайплайнов у него больше нет: `Pipeline()` возвращает хендл, выданный движковым кэшем, поэтому сессия редактора и игра делят один бюджет.

Тест	Что покрывает
`material_graph`	Валидация DAG, детерминированный GLSL, приёмка glslang
`material_asset`	Round-trip графа, порча payload, флаги контракта
`material_cook`	Сквозной: инструмент кука, бинарь только с движком декодирует и проверяет хеш
`material_pipeline_cache`	Разделение, счётчики ссылок, эвикция, исчерпание, отказы
`material_mesh_path`	Два меша делят один пайплайн, третий остаётся на шейдере движка

Спрайтовый путь теперь часть той же системы. Спрайтовый материал — второй контракт шейдера, а не второй компилятор: `MaterialCompileOptions::domain` переключает фрагментный интерфейс на varying-и `quad.vert`, `vLayer` даёт слой атласа элемента, а

спрайтовый push-блок несёт часы материала по смещению 80, поэтому `Time` работает в обоих доменах; отклоняются только `Fresnel` и `VertexPosition`, которым нужна view-projection. 64-битный ключ отрисовки несёт материал-бакет (16 бит слой, 8 бит материал, 32 бита глубина, 8 бит последовательность), поэтому `DrawOrder::MaterialBucket` и сортирует спрайт в его материальный прогон, и через `ChangeMaskRenderExtract::SetMaterialPipelines` превращает прогон в один `DrawBatch` с id кэшированного пайплайна. Спрайтовый цикл Vulkan биндит этот пайплайн на батч и возвращает пайплайн движка для следующего прогона; бэкенд без рантайм-пайплайнов остаётся на пайплайне quad и рисует спрайт без эффекта графа, но никогда несвязанным хендлом. Композиция слоёв на этапе компиляции, если появится, станет видами узлов того же графа, а не буфером замыканий переменной длины.

39. Приложение A - сгенерированный индекс символов

Генерируется из заголовков при каждом запуске. Заголовок и строка — авторитетное расположение.

Editor/Infrastructure (94)

Символ	Вид	Заголовок	Строка
AIStateTreeCanvasPanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	39
AnimationGraphCanvasPanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	40
AnimationTimelinePanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	41
AssetDatabaseWatchSystem	Infrastructure	Source/Editor/Infrastructure/AssetDatabaseWatchSystem.hpp	20
AssetDatabaseWatchSystem	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	42
AssetImportService	Infrastructure	Source/Editor/Infrastructure/AssetImportService.hpp	30
AssetImportService	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	43
Callbacks	Infrastructure	Source/Editor/Infrastructure/AssetDatabaseWatchSystem.hpp	38
Change	Infrastructure	Source/Editor/Infrastructure/AssetDatabaseWatchSystem.hpp	102
Codec	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	217
Command	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	231
CommandKind	Infrastructure	Source/Editor/Infrastructure/	30

		CommandStack.hpp	
CommandStack	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	27
CommandStack	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	44
Config	Infrastructure	Source/Editor/ Infrastructure/ AssetDatabaseWatch System.hpp	27
Config	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	39
Config	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	85
ContentBrowserPan el	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	45
CookRequest	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	115
CookResult	Infrastructure	Source/Editor/ Infrastructure/ AssetImportService.hp p	126
EditorCamera	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	85
EditorDesc	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	56
EditorFontRole	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	22
EditorFonts	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	79
EditorFonts	Infrastructure	Source/Editor/ Infrastructure/ EditorTypography.hpp	7
EditorFontSpec	Infrastructure	Source/Editor/	38

		Infrastructure/ EditorFonts.hpp	
EditorFontStatus	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	66
EditorSettings	Infrastructure	Source/Editor/ Infrastructure/ EditorSettings.hpp	18
EditorSettingsPanel	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	46
EditorSystem	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	94
Entry	Infrastructure	Source/Editor/ Infrastructure/ TextureThumbnails.hp p	52
ExternalClientDesc	Infrastructure	Source/Editor/ Infrastructure/ PlayInEditor.hpp	37
FrameComposer	Infrastructure	Source/Editor/ Infrastructure/ FrameComposer.hpp	28
ImFont	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	10
ImGuiIO	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	11
Impl	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	200
InteractionTransacti on	Infrastructure	Source/Editor/ Infrastructure/ EditTransactions.hpp	16
LoadedSource	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	133
MaterialGraphPanel	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	47
MaterialGraphRunti me	Infrastructure	Source/Editor/ Infrastructure/ MaterialGraphRuntime	41

		.hpp	
MaterialRuntimeStats	Infrastructure	Source/Editor/Infrastructure/MaterialGraphRuntime.hpp	25
NetSession	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	27
NetworkDiagnosticsPanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	48
PendingEdit	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	252
PendingSpawn	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	261
PhysicsDebugFlags	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	19
PhysicsDebugFrame	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	47
PhysicsDebugLine	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	32
PhysicsDebugPoint	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	40
PhysicsDebugVisualizer	Infrastructure	Source/Editor/Infrastructure/PhysicsDebugDraw.hpp	58
PiePlayerInput	Infrastructure	Source/Editor/Infrastructure/PlayInEditor.hpp	27
PlayInEditor	Infrastructure	Source/Editor/Infrastructure/PlayInEditor.hpp	44
ProjectConfig	Infrastructure	Source/Editor/Infrastructure/	52

		EditorSystem.hpp	
ProjectConfig	Infrastructure	Source/Editor/ Infrastructure/ ProjectConfig.hpp	142
ProjectConfigStore	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	49
ProjectConfigStore	Infrastructure	Source/Editor/ Infrastructure/ ProjectConfig.hpp	202
ProjectDashboard	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	50
ProjectPaths	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	53
ProjectPaths	Infrastructure	Source/Editor/ Infrastructure/ ProjectConfig.hpp	164
ProjectPaths	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	21
ProjectSettings	Infrastructure	Source/Editor/ Infrastructure/ ProjectConfig.hpp	90
ProjectSettings	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	22
ProjectSettingsPanel	Infrastructure	Source/Editor/ Infrastructure/ EditorSystem.hpp	51
Range	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	246
ReleaseCookConfig	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	70
ReleaseCooker	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	154
ReleaseCookProgress	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	117
ReleaseCookResult	Infrastructure	Source/Editor/ Infrastructure/	138

		ReleaseCooker.hpp	
ReleaseCookStage	Infrastructure	Source/Editor/ Infrastructure/ ReleaseCooker.hpp	28
Result	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	40
Result	Infrastructure	Source/Editor/ Infrastructure/ PlayInEditor.hpp	47
SdfCollisionSystem	Infrastructure	Source/Editor/ Infrastructure/ PhysicsDebugDraw.hpp	13
SdfField	Infrastructure	Source/Editor/ Infrastructure/ FrameComposer.hpp	17
Services	Infrastructure	Source/Editor/ Infrastructure/ PlayInEditor.hpp	220
Slot	Infrastructure	Source/Editor/ Infrastructure/ MaterialGraphRuntime.hpp	118
Source	Infrastructure	Source/Editor/ Infrastructure/ EditTransactions.hpp	19
SpawnComponent	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	275
SpawnHeader	Infrastructure	Source/Editor/ Infrastructure/ CommandStack.hpp	268
SpawnTransaction	Infrastructure	Source/Editor/ Infrastructure/ EditTransactions.hpp	190
StableSource	Infrastructure	Source/Editor/ Infrastructure/ EditorFonts.hpp	140
Stats	Infrastructure	Source/Editor/ Infrastructure/ AssetDatabaseWatchSystem.hpp	88
Stats	Infrastructure	Source/Editor/ Infrastructure/	55

		AssetImportService.hpp	
Stats	Infrastructure	Source/Editor/Infrastructure/CommandStack.hpp	92
TextureArrayAsset	Infrastructure	Source/Editor/Infrastructure/AssetImportService.hpp	23
TextureBinding	Infrastructure	Source/Editor/Infrastructure/AssetImportService.hpp	142
TextureThumbnailCache	Infrastructure	Source/Editor/Infrastructure/TextureThumbnails.hpp	17
VirtualFileSystem	Infrastructure	Source/Editor/Infrastructure/AssetImportService.hpp	20
VirtualFileSystem	Infrastructure	Source/Editor/Infrastructure/EditorFonts.hpp	15
VirtualFileSystem	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	18
VirtualTextureSettings	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	32
VirtualTextureSettings	Infrastructure	Source/Editor/Infrastructure/ProjectConfig.hpp	30
VirtualTextureSystem	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	22
VisualScriptPanel	Infrastructure	Source/Editor/Infrastructure/EditorSystem.hpp	54
WindowManager	Infrastructure	Source/Editor/Infrastructure/EditorFonts.hpp	16

Editor/Panels (76)

Символ	Вид	Заголовок	Строка
Action	Panels	Source/Editor/Panels/ProjectDashboard.hpp	25
AIStateTreeCanvasPanel	Panels	Source/Editor/Panels/AIStateTreeCanvasPanel.hpp	55
AIStateTreeCompiled	Panels	Source/Editor/Panels/AIStateTreeCanvasPanel.hpp	35
AIStateTreeDocument	Panels	Source/Editor/Panels/AIStateTreeCanvasPanel.hpp	24
AnimationGraphCanvasPanel	Panels	Source/Editor/Panels/AnimationGraphCanvasPanel.hpp	116
AnimationGraphDocument	Panels	Source/Editor/Panels/AnimationGraphCanvasPanel.hpp	79
AnimationGraphLink	Panels	Source/Editor/Panels/AnimationGraphCanvasPanel.hpp	65
AnimationGraphLinkKind	Panels	Source/Editor/Panels/AnimationGraphCanvasPanel.hpp	33
AnimationGraphNode	Panels	Source/Editor/Panels/AnimationGraphCanvasPanel.hpp	45
AnimationGraphNodeKind	Panels	Source/Editor/Panels/AnimationGraphCanvasPanel.hpp	26
AnimationTimelinePanel	Panels	Source/Editor/Panels/AnimationTimelinePanel.hpp	34
AssetImportService	Panels	Source/Editor/	24

		Panels/ ContentBrowserPanel. hpp	
AudioSystem	Panels	Source/Editor/ Panels/ AnimationTimelinePan el.hpp	15
AudioSystem	Panels	Source/Editor/ Panels/ VoiceDevicePanel.hpp	11
CanvasModelOps	Panels	Source/Editor/ Panels/ NodeCanvasPanel.hp p	21
CommandStack	Panels	Source/Editor/ Panels/ ContentBrowserPanel. hpp	25
CommandStack	Panels	Source/Editor/ Panels/ SettingsPanels.hpp	28
ComponentRegistry	Panels	Source/Editor/ Panels/ ScriptDebuggerPanel. hpp	18
Config	Panels	Source/Editor/ Panels/ ContentBrowserPanel. hpp	35
ContentBrowserPan el	Panels	Source/Editor/ Panels/ ContentBrowserPanel. hpp	27
DragPayload	Panels	Source/Editor/ Panels/ ContentBrowserPanel. hpp	60
EditorFonts	Panels	Source/Editor/ Panels/ ProjectDashboard.hpp	20
EditorFonts	Panels	Source/Editor/ Panels/ VisualScriptPanel.hpp	20
EditorSettingsPanel	Panels	Source/Editor/ Panels/ SettingsPanels.hpp	72

EditSource	Panels	Source/Editor/Panels/AIStateTreeCanvasPanel.hpp	58
EditSource	Panels	Source/Editor/Panels/AnimationGraphCanvasPanel.hpp	119
Entry	Panels	Source/Editor/Panels/ContentBrowserPanel.hpp	43
FileDialog	Panels	Source/Editor/Panels/ContentBrowserPanel.hpp	140
ImNodesContext	Panels	Source/Editor/Panels/AIStateTreeCanvasPanel.hpp	13
ImNodesContext	Panels	Source/Editor/Panels/AnimationGraphCanvasPanel.hpp	15
LicensesPanel	Panels	Source/Editor/Panels/LicensesPanel.hpp	11
LogBoyPanel	Panels	Source/Editor/Panels/LogBoyPanel.hpp	394
LogBoyRecord	Panels	Source/Editor/Panels/LogBoyPanel.hpp	105
LogBoyRing	Panels	Source/Editor/Panels/LogBoyPanel.hpp	121
LogBoySeverity	Panels	Source/Editor/Panels/LogBoyPanel.hpp	33
LogBoySystem	Panels	Source/Editor/Panels/LogBoyPanel.hpp	186
LogicGraphDocument	Panels	Source/Editor/Panels/VisualScriptPanel.hpp	23
MaterialGraphPanel	Panels	Source/Editor/	20

		Panels/ MaterialGraphPanel.hpp	
Mode	Panels	Source/Editor/ Panels/ ProjectDashboard.hpp	84
NetSession	Panels	Source/Editor/ Panels/ NetworkDiagnosticsPanel.hpp	6
NetworkDiagnosticsPanel	Panels	Source/Editor/ Panels/ NetworkDiagnosticsPanel.hpp	15
NodeCanvasPanel	Panels	Source/Editor/ Panels/ NodeCanvasPanel.hpp	54
PIEAnimFrame	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	101
PieClientViewport	Panels	Source/Editor/ Panels/ PieClientViewport.hpp	15
PIEFrameSlot	Panels	Source/Editor/ Panels/ AnimationGraphCanvasPanel.hpp	283
ProjectConfigStore	Panels	Source/Editor/ Panels/ SettingsPanels.hpp	29
ProjectConfigStore	Panels	Source/Editor/ Panels/ VoiceDevicePanel.hpp	17
ProjectDashboard	Panels	Source/Editor/ Panels/ ProjectDashboard.hpp	22
ProjectSettingsPanel	Panels	Source/Editor/ Panels/ SettingsPanels.hpp	31
QueryKind	Panels	Source/Editor/ Panels/ LogBoyPanel.hpp	430
SceneDiagnosticsPa	Panels	Source/Editor/ Panels/	37

nel		SceneDiagnosticsPanel.hpp	
SceneDiagnosticsSample	Panels	Source/Editor/Panels/SceneDiagnosticsPanel.hpp	15
ScriptDebuggerPanel	Panels	Source/Editor/Panels/ScriptDebuggerPanel.hpp	28
ScriptRuntime	Panels	Source/Editor/Panels/AnimationTimelinePanel.hpp	16
SinkUser	Panels	Source/Editor/Panels/AnimationTimelinePanel.hpp	88
SkeletonInspectorState	Panels	Source/Editor/Panels/SkeletonAssetPanel.hpp	19
SpawnContext	Panels	Source/Editor/Panels/ContentBrowserPanel.hpp	68
SpriteAnimationInspectorState	Panels	Source/Editor/Panels/SpriteAnimationPanel.hpp	21
SystemContext	Panels	Source/Editor/Panels/SettingsPanels.hpp	22
SystemContext	Panels	Source/Editor/Panels/SkeletonAssetPanel.hpp	13
SystemContext	Panels	Source/Editor/Panels/SpriteAnimationPanel.hpp	13
SystemName	Panels	Source/Editor/Panels/LogBoyPanel.hpp	189
TextPreviewPanel	Panels	Source/Editor/Panels/	24

		TextPreviewPanel.hpp	
TextureThumbnailCache	Panels	Source/Editor/Panels/SpriteAnimationPanel.hpp	19
TilemapEditorPanel	Panels	Source/Editor/Panels/TilemapEditorPanel.hpp	20
TimelinePanelContext	Panels	Source/Editor/Panels/AnimationTimelinePanel.hpp	22
VirtualFileSystem	Panels	Source/Editor/Panels/ContentBrowserPanel.hpp	18
VirtualFileSystem	Panels	Source/Editor/Panels/ProjectDashboard.hpp	13
VirtualFileSystem	Panels	Source/Editor/Panels/TextPreviewPanel.hpp	15
VirtualTexturePanel	Panels	Source/Editor/Panels/VirtualTexturePanel.hpp	18
VirtualTextureSystem	Panels	Source/Editor/Panels/VirtualTexturePanel.hpp	10
VisualScriptPanel	Panels	Source/Editor/Panels/ScriptDebuggerPanel.hpp	24
VisualScriptPanel	Panels	Source/Editor/Panels/VisualScriptPanel.hpp	32
VoiceDevicePanel	Panels	Source/Editor/Panels/VoiceDevicePanel.hpp	19
WidgetCanvas	Panels	Source/Editor/Panels/SettingsPanels.hpp	16
WindowManager	Panels	Source/Editor/Panels/	14

		ProjectDashboard.hpp	

Engine/Assets (57)

Символ	Вид	Заголовок	Строка
AeadAlgorithm	Assets	Source/Engine/Assets/FemCrypto.hpp	18
AeadKey	Assets	Source/Engine/Assets/FemCrypto.hpp	39
AeadNonce	Assets	Source/Engine/Assets/FemCrypto.hpp	46
AeadTag	Assets	Source/Engine/Assets/FemCrypto.hpp	54
ArchiveBuildOptions	Assets	Source/Engine/Assets/FemArchiver.hpp	99
ArchiveCipher	Assets	Source/Engine/Assets/FemArchiver.hpp	29
ArchiveCodec	Assets	Source/Engine/Assets/FemArchiver.hpp	22
ArchiveEntryView	Assets	Source/Engine/Assets/FemArchiver.hpp	115
ArchiveInput	Assets	Source/Engine/Assets/FemArchiver.hpp	72
ArchiveProgress	Assets	Source/Engine/Assets/FemArchiver.hpp	85
ArchiveProgressStage	Assets	Source/Engine/Assets/FemArchiver.hpp	78
ArchiveReader	Assets	Source/Engine/Assets/FemArchiver.hpp	126
ArchiveReader	Assets	Source/Engine/Assets/FemArchiver.hpp	145

ArchiveStatus	Assets	Source/Engine/Assets/FemArchiver.hpp	36
AssetDatabase	Assets	Source/Engine/Assets/AssetDatabase.hpp	28
AssetDependency	Assets	Source/Engine/Assets/AssetMeta.hpp	83
AssetHandle	Assets	Source/Engine/Assets/AssetMeta.hpp	22
AssetImportSettings	Assets	Source/Engine/Assets/AssetMeta.hpp	45
AssetMeta	Assets	Source/Engine/Assets/AssetMeta.hpp	61
AssetRecord	Assets	Source/Engine/Assets/AssetDatabase.hpp	19
AstcBlock	Assets	Source/Engine/Assets/TextureCook.hpp	25
AtlasDesc	Assets	Source/Engine/Assets/TextureArray.hpp	28
AtlasLayer	Assets	Source/Engine/Assets/TextureArray.hpp	21
AudioClipHeader	Assets	Source/Engine/Assets/AudioClipFormat.hpp	87
BlockExtent	Assets	Source/Engine/Assets/TextureFormat.hpp	79
CleanRenderExtract	Assets	Source/Engine/Assets/ProjectScaffold.hpp	127
CookedAudioView	Assets	Source/Engine/Assets/AudioClipFormat.hpp	116
CookedTexture	Assets	Source/Engine/Assets/TextureCook.hpp	41
CookedTextureView	Assets	Source/Engine/Assets/TextureFormat.hpp	179

CookProgressFlags	Assets	Source/Engine/Assets/CookProgress.hpp	31
CookProgressRecord	Assets	Source/Engine/Assets/CookProgress.hpp	38
CookProgressStage	Assets	Source/Engine/Assets/CookProgress.hpp	18
CookTarget	Assets	Source/Engine/Assets/TextureCook.hpp	17
FanmHeader	Assets	Source/Engine/Assets/AnimationClipAsset.hpp	22
FemArchiver	Assets	Source/Engine/Assets/FemArchiver.hpp	128
FempakInput	Assets	Source/Engine/Assets/FempakWriter.hpp	15
FempakWriter	Assets	Source/Engine/Assets/FempakWriter.hpp	22
LocaleTable	Assets	Source/Engine/Assets/Localization.hpp	14
LocalizationGender	Assets	Source/Engine/Assets/Localization.hpp	21
LocalizationPack	Assets	Source/Engine/Assets/Localization.hpp	13
LocalizationTable	Assets	Source/Engine/Assets/Localization.hpp	28
MappedFile	Assets	Source/Engine/Assets/VFS.hpp	19
MountRecord	Assets	Source/Engine/Assets/VFS.hpp	128
PakEntry	Assets	Source/Engine/Assets/PakFormat.hpp	26
PakHeader	Assets	Source/Engine/	16

		Assets/ PakFormat.hpp	
PcmFormat	Assets	Source/Engine/ Assets/ AudioClipFormat.hpp	21
PinnedFile	Assets	Source/Engine/ Assets/VFS.hpp	63
ReloadNotice	Assets	Source/Engine/ Assets/ AssetDatabase.hpp	86
StagedRow	Assets	Source/Engine/ Assets/ AssetDatabase.hpp	108
StringEntry	Assets	Source/Engine/ Assets/ Localization.hpp	15
TextureArrayAsset	Assets	Source/Engine/ Assets/ TextureArray.hpp	46
TextureCookOptions	Assets	Source/Engine/ Assets/ TextureCook.hpp	31
TextureFormat	Assets	Source/Engine/ Assets/ TextureFormat.hpp	22
TextureHeader	Assets	Source/Engine/ Assets/ TextureFormat.hpp	160
VfsEntry	Assets	Source/Engine/ Assets/VFS.hpp	55
VirtualFileSystem	Assets	Source/Engine/ Assets/ TextureArray.hpp	16
VirtualFileSystem	Assets	Source/Engine/ Assets/VFS.hpp	69

Engine/Audio (78)

Символ	Вид	Заголовок	Строка
AudioBus	Audio	Source/Engine/ Audio/Audio.hpp	26
AudioBusComponent	Audio	Source/Engine/ Audio/Audio.hpp	40
AudioCaptureMonito	Audio	Source/Engine/	144

r		Audio/ AudioDevices.hpp	
AudioClip	Audio	Source/Engine/ Audio/Audio.hpp	69
AudioCommand	Audio	Source/Engine/ Audio/DspGraph.hpp	138
AudioCommandType	Audio	Source/Engine/ Audio/DspGraph.hpp	121
AudioContextComponent	Audio	Source/Engine/ Audio/ AudioZones.hpp	67
AudioDeviceEnumerator	Audio	Source/Engine/ Audio/ AudioDevices.hpp	93
AudioDeviceInfo	Audio	Source/Engine/ Audio/ AudioDevices.hpp	54
AudioDeviceKind	Audio	Source/Engine/ Audio/ AudioDevices.hpp	15
AudioDeviceList	Audio	Source/Engine/ Audio/ AudioDevices.hpp	66
AudioDevicePreferences	Audio	Source/Engine/ Audio/ AudioDevices.hpp	73
AudioDeviceStatus	Audio	Source/Engine/ Audio/ AudioDevices.hpp	21
AudioLimiter	Audio	Source/Engine/ Audio/Limiter.hpp	23
AudioListener	Audio	Source/Engine/ Audio/Audio.hpp	94
AudioSource	Audio	Source/Engine/ Audio/Audio.hpp	79
AudioSourceComponent	Audio	Source/Engine/ Audio/Audio.hpp	53
AudioSystem	Audio	Source/Engine/ Audio/Audio.hpp	100
AudioZoneComponent	Audio	Source/Engine/ Audio/ AudioZones.hpp	32
AudioZoneKind	Audio	Source/Engine/ Audio/	14

		AudioZones.hpp	
AudioZoneSystem	Audio	Source/Engine/ Audio/ AudioZones.hpp	91
BusAtomics	Audio	Source/Engine/ Audio/DspGraph.hpp	370
BusFilterParams	Audio	Source/Engine/ Audio/DspGraph.hpp	108
BusRuntime	Audio	Source/Engine/ Audio/Audio.hpp	212
BusShadow	Audio	Source/Engine/ Audio/DspGraph.hpp	291
BusState	Audio	Source/Engine/ Audio/DspGraph.hpp	338
CommandQueue	Audio	Source/Engine/ Audio/DspGraph.hpp	170
CookedClipRecord	Audio	Source/Engine/ Audio/Audio.hpp	187
DspClipView	Audio	Source/Engine/ Audio/DspGraph.hpp	205
DspGraph	Audio	Source/Engine/ Audio/DspGraph.hpp	216
DspGraph	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	16
DspResult	Audio	Source/Engine/ Audio/DspGraph.hpp	34
DspResult	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	19
DuckingParams	Audio	Source/Engine/ Audio/DspGraph.hpp	80
DuckingState	Audio	Source/Engine/ Audio/DspGraph.hpp	298
FilterState	Audio	Source/Engine/ Audio/DspGraph.hpp	327
FloatRing	Audio	Source/Engine/ Audio/Audio.hpp	24
FloatRing	Audio	Source/Engine/ Audio/ AudioStream.hpp	16
FloatRing	Audio	Source/Engine/	32

		Audio/DspGraph.hpp	
JitterSlot	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	110
LimiterDesc	Audio	Source/Engine/ Audio/Limiter.hpp	11
OggOpusStream	Audio	Source/Engine/ Audio/OggOpus.hpp	131
OggPacket	Audio	Source/Engine/ Audio/OggOpus.hpp	76
OggPacketReader	Audio	Source/Engine/ Audio/OggOpus.hpp	86
OggPage	Audio	Source/Engine/ Audio/OggOpus.hpp	36
OggPageReader	Audio	Source/Engine/ Audio/OggOpus.hpp	57
OpusHeadInfo	Audio	Source/Engine/ Audio/OggOpus.hpp	106
OwnedPcmRecord	Audio	Source/Engine/ Audio/Audio.hpp	194
Peer	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	120
RateBucket	Audio	Source/Engine/ Audio/ VoiceNetworkBridge.hpp	62
ResolvedZone	Audio	Source/Engine/ Audio/ AudioZones.hpp	113
ReverbParams	Audio	Source/Engine/ Audio/DspGraph.hpp	96
ReverbState	Audio	Source/Engine/ Audio/DspGraph.hpp	311
SourceRuntime	Audio	Source/Engine/ Audio/Audio.hpp	204
State	Audio	Source/Engine/ Audio/ AudioDevices.hpp	164
Stats	Audio	Source/Engine/ Audio/ AudioZones.hpp	94
VoiceAec	Audio	Source/Engine/	44

		Audio/VoiceAec.hpp	
VoiceAecSettings	Audio	Source/Engine/ Audio/VoiceAec.hpp	22
VoiceCapture	Audio	Source/Engine/ Audio/ AudioStream.hpp	64
VoiceChannel	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	65
VoiceChannelConfig	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	36
VoiceChannelStats	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	46
VoiceCodecConfig	Audio	Source/Engine/ Audio/ VoiceCodec.hpp	15
VoiceDecoder	Audio	Source/Engine/ Audio/ VoiceCodec.hpp	51
VoiceDenoiseTier	Audio	Source/Engine/ Audio/VoiceDsp.hpp	10
VoiceDspChain	Audio	Source/Engine/ Audio/VoiceDsp.hpp	99
VoiceDspSettings	Audio	Source/Engine/ Audio/VoiceDsp.hpp	43
VoiceEncoder	Audio	Source/Engine/ Audio/ VoiceCodec.hpp	27
VoiceNetworkBridge	Audio	Source/Engine/ Audio/ VoiceNetworkBridge.hpp	18
VoiceParticipant	Audio	Source/Engine/ Audio/ VoiceSession.hpp	19
VoicePresetKind	Audio	Source/Engine/ Audio/VoiceDsp.hpp	68
VoiceRouting	Audio	Source/Engine/ Audio/ VoiceChannel.hpp	30
VoiceSession	Audio	Source/Engine/ Audio/	15

		VoiceChannel.hpp	
VoiceSession	Audio	Source/Engine/ Audio/ VoiceSession.hpp	31
VoiceState	Audio	Source/Engine/ Audio/DspGraph.hpp	350
VoiceSystem	Audio	Source/Engine/ Audio/ VoiceSystem.hpp	34
VoiceSystemConfig	Audio	Source/Engine/ Audio/ VoiceSystem.hpp	20
VorbisIdentInfo	Audio	Source/Engine/ Audio/OggOpus.hpp	121

Engine/Core (87)

Символ	Вид	Заголовок	Строка
AssetId	Core	Source/Engine/Core/ AssetId.hpp	43
AssetKind	Core	Source/Engine/Core/ AssetId.hpp	15
BigIntT	Core	Source/Engine/Core/ BigInt.hpp	179
BigNumberStore	Core	Source/Engine/Core/ BigNumberStore.hpp	41
BinaryKind	Core	Source/Engine/Core/ BigNumberStore.hpp	259
BinaryKind	Core	Source/Engine/Core/ DecimalStore.hpp	206
BufferTag	Core	Source/Engine/Core/ Types.hpp	301
BuildInfo	Core	Source/Engine/Core/ BuildInfo.hpp	12
BuildProfile	Core	Source/Engine/Core/ BuildProfile.hpp	135
CameraView	Core	Source/Engine/Core/ Grid.hpp	337
Cell	Core	Source/Engine/Core/ Grid.hpp	74
CivilDate	Core	Source/Engine/Core/ Calendar.hpp	21

Clock	Core	Source/Engine/Core/Time.hpp	28
ClockTime	Core	Source/Engine/Core/Calendar.hpp	32
Color32	Core	Source/Engine/Core/Types.hpp	203
ComponentDesc	Core	Source/Engine/Core/Mirror.hpp	48
ComponentRegistry	Core	Source/Engine/Core/PluginHost.hpp	27
ComponentType	Core	Source/Engine/Core/PluginHost.hpp	18
DateTime	Core	Source/Engine/Core/Calendar.hpp	44
DecimalStore	Core	Source/Engine/Core/DecimalStore.hpp	17
DecimalT	Core	Source/Engine/Core/Decimal.hpp	34
DurationStyle	Core	Source/Engine/Core/Time.hpp	571
EddsaKeyPair	Core	Source/Engine/Core/CryptoAsymmetric.hpp	31
FieldDesc	Core	Source/Engine/Core/Mirror.hpp	37
FieldKind	Core	Source/Engine/Core/Mirror.hpp	17
FileFilter	Core	Source/Engine/Core/WindowManager.hpp	154
FormatSpec	Core	Source/Engine/Core/BigInt.hpp	36
FrameArena	Core	Source/Engine/Core/FrameArena.hpp	12
FrameTimer	Core	Source/Engine/Core/Time.hpp	501
GridDesc	Core	Source/Engine/Core/Grid.hpp	91
Handle	Core	Source/Engine/Core/BigNumberStore.hpp	20
Handle	Core	Source/Engine/Core/Types.hpp	275

Host	Core	Source/Engine/Core/Plugin.hpp	20
InputState	Core	Source/Engine/Core/WindowManager.hpp	98
JobSystem	Core	Source/Engine/Core/JobSystem.hpp	25
Key	Core	Source/Engine/Core/WindowManager.hpp	30
KeyLess	Core	Source/Engine/Core/FlatMap.hpp	198
LoadedPlugin	Core	Source/Engine/Core/PluginHost.hpp	64
LogLevel	Core	Source/Engine/Core/Log.hpp	12
Mat4	Core	Source/Engine/Core/Mat4.hpp	14
NoiseGenerator	Core	Source/Engine/Core/NoiseGenerator.hpp	10
Notation	Core	Source/Engine/Core/BigInt.hpp	30
NoticeEntry	Core	Source/Engine/Core/ThirdPartyNotices.hpp	12
OptimizationAdvice	Core	Source/Engine/Core/BuildProfile.hpp	154
OptimizationLevel	Core	Source/Engine/Core/BuildProfile.hpp	10
Pcg32	Core	Source/Engine/Core/Pcg32.hpp	20
PipelineTag	Core	Source/Engine/Core/Types.hpp	303
PluginHost	Core	Source/Engine/Core/PluginHost.hpp	76
PluginSystem	Core	Source/Engine/Core/PluginHost.hpp	54
ProjectionMode	Core	Source/Engine/Core/Grid.hpp	17
Rect	Core	Source/Engine/Core/Types.hpp	173
Reflection	Core	Source/Engine/Core/Mirror.hpp	66

Reflection	Core	Source/Engine/Core/Mirror.hpp	289
Reflection	Core	Source/Engine/Core/Mirror.hpp	301
RngState	Core	Source/Engine/Core/Pcg32.hpp	11
RoundingMode	Core	Source/Engine/Core/Decimal.hpp	22
SamplerTag	Core	Source/Engine/Core/Types.hpp	304
SecureRng	Core	Source/Engine/Core/SecureRng.hpp	11
Sha256Context	Core	Source/Engine/Core/Sha256.hpp	17
Slot	Core	Source/Engine/Core/BigNumberStore.hpp	252
Slot	Core	Source/Engine/Core/DecimalStore.hpp	199
Status	Core	Source/Engine/Core/BigInt.hpp	18
Stopwatch	Core	Source/Engine/Core/Time.hpp	209
Task	Core	Source/Engine/Core/JobSystem.hpp	96
TextureTag	Core	Source/Engine/Core/Types.hpp	302
TextWriter	Core	Source/Engine/Core/Calendar.hpp	189
ThreadCounter	Core	Source/Engine/Core/JobSystem.hpp	109
ThreadLoad	Core	Source/Engine/Core/JobSystem.hpp	31
TimerFired	Core	Source/Engine/Core/Time.hpp	255
TimerHandle	Core	Source/Engine/Core/Time.hpp	240
TimerPool	Core	Source/Engine/Core/Time.hpp	266
TypeNameOverride	Core	Source/Engine/Core/Definitions.hpp	40
TypeNameOverride	Core	Source/Engine/Core/	283

		Definitions.hpp	
TypeObject	Core	Source/Engine/Core/ Definitions.hpp	185
TypeRecord	Core	Source/Engine/Core/ Definitions.hpp	85
TypeTable	Core	Source/Engine/Core/ Definitions.hpp	93
TypeTraits	Core	Source/Engine/Core/ Definitions.hpp	170
TypeTraits	Core	Source/Engine/Core/ Definitions.hpp	267
Unregistered	Core	Source/Engine/ Core/Mirror.hpp	59
Vector2D	Core	Source/Engine/Core/ Types.hpp	13
Vector3D	Core	Source/Engine/Core/ Types.hpp	90
Vector4D	Core	Source/Engine/Core/ Types.hpp	139
WindowDesc	Core	Source/Engine/ Core/ WindowManager.hpp	175
WindowExtent	Core	Source/Engine/ Core/ WindowManager.hpp	21
WindowManager	Core	Source/Engine/ Core/ WindowManager.hpp	186
WindowMode	Core	Source/Engine/ Core/ WindowManager.hpp	166
X25519KeyPair	Core	Source/Engine/Core/ CryptoAsymmetric.hp p	20

Engine/ECS (185)

Символ	Вид	Заголовок	Строка
AccessibilityKernel	ECS	Source/Engine/ECS/ Schedule.hpp	21
AICommandCompon ent	ECS	Source/Engine/ECS/ StateTree.hpp	150

AICommandKind	ECS	Source/Engine/ECS/StateTree.hpp	141
AnimationTimeline	ECS	Source/Engine/ECS/AnimationTimeline.hpp	90
AnimBoneKey	ECS	Source/Engine/ECS/AnimGraph.hpp	70
AnimClip	ECS	Source/Engine/ECS/AnimGraph.hpp	82
AnimConditionFlags	ECS	Source/Engine/ECS/AnimGraph.hpp	27
AnimGraphSystem	ECS	Source/Engine/ECS/AnimGraph.hpp	174
AnimRestBone	ECS	Source/Engine/ECS/AnimGraph.hpp	100
AnimStateComponent	ECS	Source/Engine/ECS/AnimGraph.hpp	110
AnimTransition	ECS	Source/Engine/ECS/AnimGraph.hpp	47
AssetDatabase	ECS	Source/Engine/ECS/Scene.hpp	21
AssetDatabase	ECS	Source/Engine/ECS/Schedule.hpp	29
AudioSystem	ECS	Source/Engine/ECS/Schedule.hpp	16
BigIntComponent	ECS	Source/Engine/ECS/Components.hpp	194
CachedTarget	ECS	Source/Engine/ECS/LightSource.hpp	225
Camera2D	ECS	Source/Engine/ECS/Components.hpp	80
CameraStackSystem	ECS	Source/Engine/ECS/Schedule.hpp	26
ChainIdTag	ECS	Source/Engine/ECS/StableIds.hpp	12
ChainSaveV3	ECS	Source/Engine/ECS/SnapshotWorld.hpp	28
ChangeMaskBit	ECS	Source/Engine/ECS/ChangeMask.hpp	19
CharacterDesc	ECS	Source/Engine/ECS/Descriptors.hpp	51
ClothIdTag	ECS	Source/Engine/ECS/	13

		StableIds.hpp	
ClothSaveV3	ECS	Source/Engine/ECS/ SnapshotWorld.hpp	59
ClothSystem	ECS	Source/Engine/ECS/ Schedule.hpp	23
ColliderSnapshot	ECS	Source/Engine/ECS/ Snapshot.hpp	181
CollisionMode	ECS	Source/Engine/ECS/ Presentation.hpp	21
CollisionPolicyComponent	ECS	Source/Engine/ECS/ Presentation.hpp	101
Command	ECS	Source/Engine/ECS/ CommandBuffer.hpp	29
CommandBuffer	ECS	Source/Engine/ECS/ CommandBuffer.hpp	37
CommandBuffer	ECS	Source/Engine/ECS/ Schedule.hpp	50
CommandKind	ECS	Source/Engine/ECS/ CommandBuffer.hpp	19
CommandLog	ECS	Source/Engine/ECS/ CommandLog.hpp	73
CommandLogHeader	ECS	Source/Engine/ECS/ CommandLog.hpp	47
CommandRecord	ECS	Source/Engine/ECS/ CommandLog.hpp	59
CommandType	ECS	Source/Engine/ECS/ CommandLog.hpp	22
CommandWorldCallbacks	ECS	Source/Engine/ECS/ CommandLog.hpp	37
ComponentChangeMask	ECS	Source/Engine/ECS/ ChangeMask.hpp	52
ComponentOps	ECS	Source/Engine/ECS/ ComponentRegistry.hpp	23
ComponentOpsFor	ECS	Source/Engine/ECS/ ComponentRegistry.hpp	46
ComponentRegistry	ECS	Source/Engine/ECS/ ComponentRegistry.hpp	107
CompressedClip	ECS	Source/Engine/ECS/ AnimationCompressio	47

		n.hpp	
CompressedKey	ECS	Source/Engine/ECS/AnimationCompression.hpp	19
CompressedTrack	ECS	Source/Engine/ECS/AnimationCompression.hpp	31
CompressionReport	ECS	Source/Engine/ECS/AnimationCompression.hpp	63
Cursor	ECS	Source/Engine/ECS/CommandLog.hpp	146
DecimalComponent	ECS	Source/Engine/ECS/Components.hpp	205
DrawOrder	ECS	Source/Engine/ECS/Components.hpp	69
EditorSelected	ECS	Source/Engine/ECS/Components.hpp	94
EmitterIdTag	ECS	Source/Engine/ECS/StableIds.hpp	14
EmitterSaveV3	ECS	Source/Engine/ECS/SnapshotWorld.hpp	79
EntityRemap	ECS	Source/Engine/ECS/Scene.hpp	59
Entry	ECS	Source/Engine/ECS/Schedule.hpp	120
FeskIHeader	ECS	Source/Engine/ECS/SkeletonAsset.hpp	37
FeskINameRef	ECS	Source/Engine/ECS/SkeletonAsset.hpp	87
FeskISprite	ECS	Source/Engine/ECS/SkeletonAsset.hpp	97
FluidSaveV3	ECS	Source/Engine/ECS/SnapshotWorld.hpp	71
FluidWaveSystem	ECS	Source/Engine/ECS/Schedule.hpp	24
FontAtlasSet	ECS	Source/Engine/ECS/Schedule.hpp	37
FrameArena	ECS	Source/Engine/ECS/Schedule.hpp	17
GameplayState	ECS	Source/Engine/ECS/Components.hpp	132

GameplayStateFlag	ECS	Source/Engine/ECS/Components.hpp	106
GraphSystem	ECS	Source/Engine/ECS/SystemGraph.hpp	17
Hidden	ECS	Source/Engine/ECS/Components.hpp	99
IkChain	ECS	Source/Engine/ECS/SkeletonIk.hpp	43
IkComponent	ECS	Source/Engine/ECS/SkeletonIk.hpp	64
IkGoalComponent	ECS	Source/Engine/ECS/SkeletonIk.hpp	77
IkSolver	ECS	Source/Engine/ECS/SkeletonIk.hpp	19
Interpolation	ECS	Source/Engine/ECS/AnimationTimeline.hpp	29
ITextShaper	ECS	Source/Engine/ECS/Schedule.hpp	43
Lifetime	ECS	Source/Engine/ECS/Components.hpp	88
Light2DComponent	ECS	Source/Engine/ECS/Lighting.hpp	33
Light2DOccluderComponent	ECS	Source/Engine/ECS/Lighting.hpp	102
Light2DResultComponent	ECS	Source/Engine/ECS/Lighting.hpp	65
Light2DTargetComponent	ECS	Source/Engine/ECS/Lighting.hpp	49
LightingSettings	ECS	Source/Engine/ECS/Lighting.hpp	113
LightingSystem	ECS	Source/Engine/ECS/Lighting.hpp	138
LightSourceComponent	ECS	Source/Engine/ECS/LightSource.hpp	48
LightSourceFlags	ECS	Source/Engine/ECS/LightSource.hpp	32
LightSourceSettings	ECS	Source/Engine/ECS/LightSource.hpp	149
LightSourceSystem	ECS	Source/Engine/ECS/LightSource.hpp	168
LightSourceType	ECS	Source/Engine/ECS/	22

		LightSource.hpp	
LocalizationTable	ECS	Source/Engine/ECS/Schedule.hpp	22
LocalizedTextComponent	ECS	Source/Engine/ECS/Components.hpp	252
LocalTransform2D	ECS	Source/Engine/ECS/Components.hpp	34
Mat3	ECS	Source/Engine/ECS/Skeleton.hpp	31
NameComponent	ECS	Source/Engine/ECS/Components.hpp	173
NarrativeLabelComponent	ECS	Source/Engine/ECS/Components.hpp	183
NativeCleanupHook	ECS	Source/Engine/ECS/RegistryOps.hpp	12
Parent2D	ECS	Source/Engine/ECS/Components.hpp	42
Phase	ECS	Source/Engine/ECS/Schedule.hpp	55
PhysicsBodyDesc	ECS	Source/Engine/ECS/Descriptors.hpp	22
PhysicsJointSystem	ECS	Source/Engine/ECS/Schedule.hpp	25
PhysicsShapeKind	ECS	Source/Engine/ECS/Descriptors.hpp	15
PhysicsSystem	ECS	Source/Engine/ECS/Schedule.hpp	18
PrefabAsset	ECS	Source/Engine/ECS/Descriptors.hpp	88
PrefabEntityDesc	ECS	Source/Engine/ECS/Descriptors.hpp	76
PrefabEntityFlags	ECS	Source/Engine/ECS/Descriptors.hpp	98
PrefabEntityOverride	ECS	Source/Engine/ECS/PrefabSystem.hpp	18
PrefabOverrides	ECS	Source/Engine/ECS/PrefabSystem.hpp	31
PrefabSystem	ECS	Source/Engine/ECS/PrefabSystem.hpp	36
QuantizedPose	ECS	Source/Engine/ECS/SkeletonIk.hpp	165

RenderMaskComponent	ECS	Source/Engine/ECS/Presentation.hpp	139
RenderPassMask	ECS	Source/Engine/ECS/Presentation.hpp	126
RetargetMap	ECS	Source/Engine/ECS/SkeletonIk.hpp	131
RetargetPair	ECS	Source/Engine/ECS/SkeletonIk.hpp	118
RHI	ECS	Source/Engine/ECS/Schedule.hpp	33
RigidBodySnapshot	ECS	Source/Engine/ECS/Snapshot.hpp	167
SceneComponent	ECS	Source/Engine/ECS/Scene.hpp	38
SceneData	ECS	Source/Engine/ECS/Scene.hpp	52
SceneEntity	ECS	Source/Engine/ECS/Scene.hpp	45
SceneLoadOptions	ECS	Source/Engine/ECS/Scene.hpp	76
SceneMeta	ECS	Source/Engine/ECS/Scene.hpp	31
Schedule	ECS	Source/Engine/ECS/Schedule.hpp	115
SdfLightingFrame	ECS	Source/Engine/ECS/Lighting.hpp	18
SdfSunComponent	ECS	Source/Engine/ECS/Lighting.hpp	77
SkeletonAssetRef	ECS	Source/Engine/ECS/SkeletonAssetRef.hpp	20
SkeletonAssetSource	ECS	Source/Engine/ECS/SkeletonAsset.hpp	189
SkeletonAssetView	ECS	Source/Engine/ECS/SkeletonAsset.hpp	121
SkeletonBoneName	ECS	Source/Engine/ECS/SkeletonAsset.hpp	168
SkeletonComponent	ECS	Source/Engine/ECS/Skeleton.hpp	121
SkeletonNameTable	ECS	Source/Engine/ECS/SkeletonAsset.hpp	175
SkeletonSystem	ECS	Source/Engine/ECS/	198

		Skeleton.hpp	
SkinComponent	ECS	Source/Engine/ECS/Skeleton.hpp	136
SkinnedSprite	ECS	Source/Engine/ECS/Skeleton.hpp	147
SleepingStaticComponent	ECS	Source/Engine/ECS/Presentation.hpp	202
SnapshotBaseline	ECS	Source/Engine/ECS/Snapshot.hpp	314
SnapshotComponent	ECS	Source/Engine/ECS/Snapshot.hpp	76
SnapshotComponentOp	ECS	Source/Engine/ECS/Snapshot.hpp	41
SnapshotExtras	ECS	Source/Engine/ECS/Snapshot.hpp	265
SnapshotFlags	ECS	Source/Engine/ECS/Snapshot.hpp	29
SnapshotHeader	ECS	Source/Engine/ECS/Snapshot.hpp	47
SnapshotRecordFlags	ECS	Source/Engine/ECS/Snapshot.hpp	35
SnapshotRestoreExtras	ECS	Source/Engine/ECS/Snapshot.hpp	270
SnapshotSchema	ECS	Source/Engine/ECS/Snapshot.hpp	91
SnapshotSchema	ECS	Source/Engine/ECS/Snapshot.hpp	206
SnapshotView	ECS	Source/Engine/ECS/Snapshot.hpp	255
Sprite2D	ECS	Source/Engine/ECS/Components.hpp	54
SpriteAnimationComponent	ECS	Source/Engine/ECS/SpriteAnimation.hpp	21
SpriteAnimationEventQueue	ECS	Source/Engine/ECS/SpriteAnimation.hpp	74
SpriteAnimationSystem	ECS	Source/Engine/ECS/SpriteAnimation.hpp	118
SpriteFrameEvent	ECS	Source/Engine/ECS/SpriteAnimation.hpp	66
SpriteFrameRect	ECS	Source/Engine/ECS/SpriteAnimation.hpp	55

SpriteSheetComponent	ECS	Source/Engine/ECS/SpriteAnimation.hpp	40
StableIdAllocator	ECS	Source/Engine/ECS/StableIds.hpp	25
StateTreeComponent	ECS	Source/Engine/ECS/StateTree.hpp	167
StateTreeDefinition	ECS	Source/Engine/ECS/StateTree.hpp	130
StateTreeEvaluator	ECS	Source/Engine/ECS/StateTree.hpp	48
StateTreeEvaluatorKind	ECS	Source/Engine/ECS/StateTree.hpp	26
StateTreeNode	ECS	Source/Engine/ECS/StateTree.hpp	94
StateTreeNodeKind	ECS	Source/Engine/ECS/StateTree.hpp	84
StateTreeSystem	ECS	Source/Engine/ECS/StateTree.hpp	212
StateTreeTask	ECS	Source/Engine/ECS/StateTree.hpp	71
StateTreeTaskKind	ECS	Source/Engine/ECS/StateTree.hpp	61
StateTreeTransition	ECS	Source/Engine/ECS/StateTree.hpp	113
SystemContext	ECS	Source/Engine/ECS/Schedule.hpp	83
SystemGraph	ECS	Source/Engine/ECS/Schedule.hpp	51
SystemGraph	ECS	Source/Engine/ECS/SystemGraph.hpp	29
TextComponent	ECS	Source/Engine/ECS/Components.hpp	224
TextConditionComponent	ECS	Source/Engine/ECS/Components.hpp	308
TextEffectComponent	ECS	Source/Engine/ECS/Components.hpp	278
TextFormatComponent	ECS	Source/Engine/ECS/Components.hpp	290
TextService	ECS	Source/Engine/ECS/Schedule.hpp	41
TextShapeCache	ECS	Source/Engine/ECS/	42

		Schedule.hpp	
TextShapeComponent	ECS	Source/Engine/ECS/Components.hpp	267
TilemapAssetRefComponent	ECS	Source/Engine/ECS/Tilemap.hpp	130
TilemapComponent	ECS	Source/Engine/ECS/Tilemap.hpp	63
TilemapRenderSystem	ECS	Source/Engine/ECS/Tilemap.hpp	150
TileQuad	ECS	Source/Engine/ECS/Tilemap.hpp	142
TimelineEvent	ECS	Source/Engine/ECS/AnimationTimeline.hpp	67
TimelineEventKind	ECS	Source/Engine/ECS/AnimationTimeline.hpp	36
TimelineKey	ECS	Source/Engine/ECS/AnimationTimeline.hpp	43
TimelineTrack	ECS	Source/Engine/ECS/AnimationTimeline.hpp	56
TimelineTriggerSink	ECS	Source/Engine/ECS/AnimationTimeline.hpp	83
TimeScaleComponent	ECS	Source/Engine/ECS/Presentation.hpp	168
TrackKind	ECS	Source/Engine/ECS/AnimationTimeline.hpp	20
Transform2D	ECS	Source/Engine/ECS/Components.hpp	23
Velocity	ECS	Source/Engine/ECS/Components.hpp	48
View	ECS	Source/Engine/ECS/CommandLog.hpp	156
VirtualFileSystem	ECS	Source/Engine/ECS/Scene.hpp	18
VirtualFileSystem	ECS	Source/Engine/ECS/Schedule.hpp	20
VirtualFileSystem	ECS	Source/Engine/ECS/SkeletonAsset.hpp	18

VirtualFileSystem	ECS	Source/Engine/ECS/ Tilemap.hpp	20
WindowManager	ECS	Source/Engine/ECS/ Schedule.hpp	19
WorldSnapshotV3	ECS	Source/Engine/ECS/ SnapshotWorld.hpp	86

Engine/Navigation (7)

Символ	Вид	Заголовок	Строка
NavAgentComponent	Navigation	Source/Engine/ Navigation/ Navigation.hpp	161
NavAgentSettings	Navigation	Source/Engine/ Navigation/ Navigation.hpp	139
NavigationSystem	Navigation	Source/Engine/ Navigation/ Navigation.hpp	184
NavMap	Navigation	Source/Engine/ Navigation/ Navigation.hpp	25
NavPath	Navigation	Source/Engine/ Navigation/ Navigation.hpp	67
PathFinder	Navigation	Source/Engine/ Navigation/ Navigation.hpp	91
PathStats	Navigation	Source/Engine/ Navigation/ Navigation.hpp	80

Engine/Network (89)

Символ	Вид	Заголовок	Строка
CellCoordinate	Network	Source/Engine/ Network/ RelevanceGrid.hpp	168
ChecksumComparison	Network	Source/Engine/ Network/ NetRollback.hpp	457
Config	Network	Source/Engine/ Network/ NetRollback.hpp	316

DatagramSocket	Network	Source/Engine/ Network/ Transport.hpp	32
Endpoint	Network	Source/Engine/ Network/NetLink.hpp	92
Frame	Network	Source/Engine/ Network/NetLink.hpp	84
FrameInfo	Network	Source/Engine/ Network/ NetRollback.hpp	221
FrameMeta	Network	Source/Engine/ Network/ NetRollback.hpp	473
InputChannel	Network	Source/Engine/ Network/ NetProtocol.hpp	56
InputChannelStats	Network	Source/Engine/ Network/ NetProtocol.hpp	44
InputPacket	Network	Source/Engine/ Network/ NetProtocol.hpp	24
InputRingBuffer	Network	Source/Engine/ Network/ NetRollback.hpp	123
LobbyError	Network	Source/Engine/ Network/Lobby.hpp	37
LobbyMessageType	Network	Source/Engine/ Network/Lobby.hpp	25
LobbyRoomInfo	Network	Source/Engine/ Network/Lobby.hpp	73
LobbyServer	Network	Source/Engine/ Network/Lobby.hpp	93
LobbyServerConfig	Network	Source/Engine/ Network/Lobby.hpp	82
LocalEntry	Network	Source/Engine/ Network/ NetRollback.hpp	205
Member	Network	Source/Engine/ Network/Lobby.hpp	126
MemoryEndpoint	Network	Source/Engine/ Network/NetLink.hpp	46
MemoryEndpoint	Network	Source/Engine/	113

		Network/NetLink.hpp	
MemoryNetwork	Network	Source/Engine/ Network/NetLink.hpp	53
NetAddress	Network	Source/Engine/ Network/ Transport.hpp	19
NetDiagnosticsSnapshot	Network	Source/Engine/ Network/ Diagnostics.hpp	12
NetDisconnectReason	Network	Source/Engine/ Network/Session.hpp	65
NetEntityId	Network	Source/Engine/ Network/ NetRollback.hpp	39
NetEntityMap	Network	Source/Engine/ Network/ NetRollback.hpp	167
NetEntry	Network	Source/Engine/ Network/ NetRollback.hpp	200
NetIdentity	Network	Source/Engine/ Network/Identity.hpp	32
NetInputFrame	Network	Source/Engine/ Network/ NetRollback.hpp	70
NetInputStatus	Network	Source/Engine/ Network/ NetRollback.hpp	99
NetLink	Network	Source/Engine/ Network/NetLink.hpp	19
NetLobbyClient	Network	Source/Engine/ Network/Lobby.hpp	160
NetLobbyClientConfig	Network	Source/Engine/ Network/Lobby.hpp	151
NetPlayerId	Network	Source/Engine/ Network/ NetRollback.hpp	59
NetPlayerSlot	Network	Source/Engine/ Network/Session.hpp	134
NetRole	Network	Source/Engine/ Network/Session.hpp	32
NetRollbackSession	Network	Source/Engine/ Network/	240

		NetRollback.hpp	
NetSession	Network	Source/Engine/ Network/Session.hpp	189
NetSessionCallbacks	Network	Source/Engine/ Network/Session.hpp	178
NetSessionConfig	Network	Source/Engine/ Network/Lobby.hpp	15
NetSessionConfig	Network	Source/Engine/ Network/Session.hpp	149
NetSessionMode	Network	Source/Engine/ Network/Session.hpp	22
NetSessionStatus	Network	Source/Engine/ Network/Session.hpp	46
NetSlotState	Network	Source/Engine/ Network/Session.hpp	58
NetTicket	Network	Source/Engine/ Network/Ticket.hpp	35
NetTickInputs	Network	Source/Engine/ Network/ NetRollback.hpp	110
NetTickMode	Network	Source/Engine/ Network/Session.hpp	40
NetTransport	Network	Source/Engine/ Network/ Transport.hpp	127
Outstanding	Network	Source/Engine/ Network/ Transport.hpp	100
Peer	Network	Source/Engine/ Network/ SecureSessionTransport.hpp	87
PendingEvent	Network	Source/Engine/ Network/Session.hpp	279
PendingState	Network	Source/Engine/ Network/ Replication.hpp	129
Phase	Network	Source/Engine/ Network/ SecureChannel.hpp	256
PhysicsSystem	Network	Source/Engine/ Network/ NetRollback.hpp	20

RateEntry	Network	Source/Engine/ Network/ SecureSessionTransport.hpp	120
Reassembly	Network	Source/Engine/ Network/ Transport.hpp	112
RelevanceBand	Network	Source/Engine/ Network/ RelevanceGrid.hpp	32
RelevanceCell	Network	Source/Engine/ Network/ RelevanceGrid.hpp	71
RelevanceConfig	Network	Source/Engine/ Network/ NetRollback.hpp	34
RelevanceConfig	Network	Source/Engine/ Network/ RelevanceGrid.hpp	56
RelevanceEntity	Network	Source/Engine/ Network/ RelevanceGrid.hpp	81
RelevanceGrid	Network	Source/Engine/ Network/ NetRollback.hpp	36
RelevanceGrid	Network	Source/Engine/ Network/ RelevanceGrid.hpp	131
RelevanceGridStats	Network	Source/Engine/ Network/ RelevanceGrid.hpp	121
RelevancePlan	Network	Source/Engine/ Network/ NetRollback.hpp	35
RelevancePlan	Network	Source/Engine/ Network/ RelevanceGrid.hpp	93
ReplicationStatus	Network	Source/Engine/ Network/ Replication.hpp	28
Result	Network	Source/Engine/ Network/ NetRollback.hpp	243
RollbackReport	Network	Source/Engine/ Network/	229

		NetRollback.hpp	
Room	Network	Source/Engine/ Network/Lobby.hpp	132
SecureChannel	Network	Source/Engine/ Network/ SecureChannel.hpp	198
SecureChannelConfig	Network	Source/Engine/ Network/ SecureChannel.hpp	151
SecureCookieIssuer	Network	Source/Engine/ Network/ SecureChannel.hpp	131
SecureEndpoint	Network	Source/Engine/ Network/ SecureEndpoint.hpp	16
SecureSessionAuthConfig	Network	Source/Engine/ Network/ SecureSessionTransport.hpp	24
SecureSessionTransport	Network	Source/Engine/ Network/ SecureSessionTransport.hpp	38
SecureStatus	Network	Source/Engine/ Network/ SecureChannel.hpp	65
Slot	Network	Source/Engine/ Network/ NetRollback.hpp	155
SnapshotReceiver	Network	Source/Engine/ Network/ Replication.hpp	177
SnapshotReceiverConfig	Network	Source/Engine/ Network/ Replication.hpp	166
SnapshotSender	Network	Source/Engine/ Network/ Replication.hpp	94
SnapshotSenderConfig	Network	Source/Engine/ Network/ Replication.hpp	70
SnapshotSenderStats	Network	Source/Engine/ Network/ Replication.hpp	80
SortableEntity	Network	Source/Engine/	195

		Network/ RelevanceGrid.hpp	
TicketStatus	Network	Source/Engine/ Network/Ticket.hpp	54
TransportChannel	Network	Source/Engine/ Network/ Transport.hpp	63
TransportPeer	Network	Source/Engine/ Network/ Transport.hpp	85
TransportStats	Network	Source/Engine/ Network/ Transport.hpp	69

Engine/Physics (54)

Символ	Вид	Заголовок	Строка
BodyRecord	Physics	Source/Engine/ Physics/Physics.hpp	203
BodyType	Physics	Source/Engine/ Physics/Physics.hpp	22
ChainError	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	64
ChainRuntime	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	247
ClothComponent	Physics	Source/Engine/ Physics/Cloth.hpp	31
ClothDesc	Physics	Source/Engine/ Physics/Cloth.hpp	63
ClothFlags	Physics	Source/Engine/ Physics/Cloth.hpp	20
ClothHandle	Physics	Source/Engine/ Physics/Cloth.hpp	46
ClothInfo	Physics	Source/Engine/ Physics/Cloth.hpp	99
ClothRenderSystem	Physics	Source/Engine/ Physics/ ClothRenderSystem.hpp	15
ClothSystem	Physics	Source/Engine/ Physics/Cloth.hpp	110

Collider2D	Physics	Source/Engine/ Physics/Physics.hpp	59
CollisionFilterMatrix	Physics	Source/Engine/ Physics/ CollisionFilter.hpp	48
CollisionLayer	Physics	Source/Engine/ Physics/ CollisionFilter.hpp	15
FluidDesc	Physics	Source/Engine/ Physics/Fluid.hpp	114
FluidInteractorComponent	Physics	Source/Engine/ Physics/Fluid.hpp	208
FluidWaveSystem	Physics	Source/Engine/ Physics/Fluid.hpp	219
GerstnerField	Physics	Source/Engine/ Physics/Fluid.hpp	49
GerstnerSample	Physics	Source/Engine/ Physics/Fluid.hpp	39
GerstnerWave	Physics	Source/Engine/ Physics/Fluid.hpp	25
JointChainComponent	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	158
JointChainDesc	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	98
JointChainFlags	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	45
JointChainHandle	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	139
JointChainInfo	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	190
JointChainKind	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	37
JointLinkComponent	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	17
ParticleEmitterComponent	Physics	Source/Engine/ Physics/Particles.hpp	26
ParticleEmitterFlags	Physics	Source/Engine/	14

		Physics/Particles.hpp	
ParticleEmitterGpu	Physics	Source/Engine/ Physics/Particles.hpp	61
ParticleSeedPush	Physics	Source/Engine/ Physics/Particles.hpp	84
ParticleStateGpu	Physics	Source/Engine/ Physics/Particles.hpp	92
PhysicsBodyRemap Entry	Physics	Source/Engine/ Physics/ SnapshotRemap.hpp	17
PhysicsBodyRemap Table	Physics	Source/Engine/ Physics/ SnapshotRemap.hpp	24
PhysicsContactEven t	Physics	Source/Engine/ Physics/Physics.hpp	87
PhysicsDesc	Physics	Source/Engine/ Physics/Physics.hpp	98
PhysicsEventType	Physics	Source/Engine/ Physics/Physics.hpp	80
PhysicsJointSystem	Physics	Source/Engine/ Physics/ PhysicsJoints.hpp	208
PhysicsSystem	Physics	Source/Engine/ Physics/Physics.hpp	107
RigidBody2D	Physics	Source/Engine/ Physics/Physics.hpp	42
RigidBodyFlags	Physics	Source/Engine/ Physics/Physics.hpp	29
Runtime	Physics	Source/Engine/ Physics/Cloth.hpp	156
SdfCollisionDesc	Physics	Source/Engine/ Physics/ SdfCollision.hpp	45
SdfCollisionSystem	Physics	Source/Engine/ Physics/ SdfCollision.hpp	71
SdfContact	Physics	Source/Engine/ Physics/ SdfCollision.hpp	59
SdfField	Physics	Source/Engine/ Physics/ SdfCollision.hpp	19
SdfPrimitiveInfo	Physics	Source/Engine/	137

		Physics/ SdfPrimitives.hpp	
SdfShape	Physics	Source/Engine/ Physics/ SdfPrimitives.hpp	82
SdfShapeComponent	Physics	Source/Engine/ Physics/ SdfPrimitives.hpp	127
SdfShapeType	Physics	Source/Engine/ Physics/ SdfPrimitives.hpp	16
ShallowWaterGrid	Physics	Source/Engine/ Physics/Fluid.hpp	134
Storage	Physics	Source/Engine/ Physics/Cloth.hpp	155
Tracked	Physics	Source/Engine/ Physics/Fluid.hpp	248
WaterView	Physics	Source/Engine/ Physics/Fluid.hpp	85

Engine/Platform (21)

Символ	Вид	Заголовок	Строка
AchievementFlags	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	95
AchievementRequest	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	104
CloudSaveComponent	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	124
CloudSaveFlags	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	115
EgsStoreSystem	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	178
GogAchievementSystem	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	224
GogStorageSystem	Platform	Source/Engine/ Platform/ PlatformSystems.hpp	258

PlatformAccountComponent	Platform	Source/Engine/Platform/PlatformSystems.hpp	137
PlatformCommand	Platform	Source/Engine/Platform/PlatformService.hpp	35
PlatformCommandLog	Platform	Source/Engine/Platform/PlatformSystems.hpp	46
PlatformCommandType	Platform	Source/Engine/Platform/PlatformService.hpp	24
PlatformLogEvent	Platform	Source/Engine/Platform/PlatformSystems.hpp	35
PlatformLogType	Platform	Source/Engine/Platform/PlatformSystems.hpp	26
PlatformService	Platform	Source/Engine/Platform/PlatformService.hpp	76
PlatformServiceCallbacks	Platform	Source/Engine/Platform/PlatformService.hpp	51
PlatformServiceDesc	Platform	Source/Engine/Platform/PlatformService.hpp	71
PlatformServiceStats	Platform	Source/Engine/Platform/PlatformService.hpp	59
PlatformState	Platform	Source/Engine/Platform/PlatformService.hpp	132
StagedSave	Platform	Source/Engine/Platform/PlatformSystems.hpp	204
StagedSave	Platform	Source/Engine/Platform/PlatformSystems.hpp	291
SteamAchievementSystem	Platform	Source/Engine/Platform/PlatformSystems.hpp	148

Engine/RHI (112)

Символ	Вид	Заголовок	Строка
Api	RHI	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	113
Backend	RHI	Source/Engine/RHI/RHI.hpp	19
CameraData	RHI	Source/Engine/RHI/RenderItem.hpp	209
CameraStackDesc	RHI	Source/Engine/RHI/CameraStack.hpp	27
CameraStackFlags	RHI	Source/Engine/RHI/CameraStack.hpp	17
CameraStackSystem	RHI	Source/Engine/RHI/CameraStack.hpp	52
CameraStackSystem	RHI	Source/Engine/RHI/RenderGraph.hpp	16
CameraStackSystem	RHI	Source/Engine/RHI/RenderPasses.hpp	12
ChangeMaskRenderExtract	RHI	Source/Engine/RHI/ChangeMaskExtract.hpp	33
ColorGradingConstants	RHI	Source/Engine/RHI/RenderPasses.hpp	116
ColorGradingPass	RHI	Source/Engine/RHI/RenderPasses.hpp	124
ColorGradingSettings	RHI	Source/Engine/RHI/RenderPasses.hpp	104
CookedMaterialView	RHI	Source/Engine/RHI/MaterialAsset.hpp	152
DeviceDesc	RHI	Source/Engine/RHI/RHI.hpp	49
DeviceInfo	RHI	Source/Engine/RHI/RHI.hpp	74
DirtyRange	RHI	Source/Engine/RHI/RenderItem.hpp	190
DrawBatch	RHI	Source/Engine/RHI/RenderItem.hpp	170
Entry	RHI	Source/Engine/RHI/MaterialPipelineCache.hpp	74
ExportStatus	RHI	Source/Engine/RHI/	100

		FrameExporter.hpp	
Extent2D	RHI	Source/Engine/RHI/RHI.hpp	40
ExtractStats	RHI	Source/Engine/RHI/ChangeMaskExtract.hpp	21
FrameAccessibility	RHI	Source/Engine/RHI/RenderItem.hpp	231
FrameAccessibilityFlags	RHI	Source/Engine/RHI/RenderItem.hpp	218
FrameExportDesc	RHI	Source/Engine/RHI/FrameExporter.hpp	53
FrameExporter	RHI	Source/Engine/RHI/FrameExporter.hpp	112
FramePacket	RHI	Source/Engine/RHI/RenderItem.hpp	304
FrameSink	RHI	Source/Engine/RHI/FrameExporter.hpp	85
FrameStats	RHI	Source/Engine/RHI/RHI.hpp	182
HashEntry	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	197
KeySorter	RHI	Source/Engine/RHI/Sort.hpp	14
LookupEntry	RHI	Source/Engine/RHI/ChangeMaskExtract.hpp	255
MaterialAssetError	RHI	Source/Engine/RHI/MaterialAsset.hpp	42
MaterialCompileOptions	RHI	Source/Engine/RHI/MaterialGraphCompiler.hpp	27
MaterialContract	RHI	Source/Engine/RHI/MaterialAsset.hpp	83
MaterialDomain	RHI	Source/Engine/RHI/MaterialGraphCompiler.hpp	21
MaterialGraph	RHI	Source/Engine/RHI/MaterialGraph.hpp	555
MaterialGraphCompiler	RHI	Source/Engine/RHI/MaterialGraphCompiler.hpp	58

MaterialGraphRequirements	RHI	Source/Engine/RHI/MaterialGraph.hpp	542
MaterialNode	RHI	Source/Engine/RHI/MaterialGraph.hpp	117
MaterialNodeFlags	RHI	Source/Engine/RHI/MaterialGraph.hpp	111
MaterialNodeKind	RHI	Source/Engine/RHI/MaterialGraph.hpp	29
MaterialParam	RHI	Source/Engine/RHI/MaterialGraph.hpp	99
MaterialParamKind	RHI	Source/Engine/RHI/MaterialGraph.hpp	88
MaterialPin	RHI	Source/Engine/RHI/MaterialGraph.hpp	129
MaterialPipelineCache	RHI	Source/Engine/RHI/MaterialPipelineCache.hpp	38
MaterialPipelineCacheStats	RHI	Source/Engine/RHI/MaterialPipelineCache.hpp	18
MaterialShaderProgram	RHI	Source/Engine/RHI/MaterialGraphCompiler.hpp	39
MaterialValueType	RHI	Source/Engine/RHI/MaterialGraph.hpp	78
MeshDesc	RHI	Source/Engine/RHI/Mesh.hpp	102
MeshDrawData	RHI	Source/Engine/RHI/Mesh.hpp	70
MeshDrawFlags	RHI	Source/Engine/RHI/Mesh.hpp	54
MeshHandle	RHI	Source/Engine/RHI/Mesh.hpp	87
MeshItemFlags	RHI	Source/Engine/RHI/Mesh.hpp	62
MeshRenderItem	RHI	Source/Engine/RHI/Mesh.hpp	114
MeshSkinRange	RHI	Source/Engine/RHI/Mesh.hpp	46
MeshVertex	RHI	Source/Engine/RHI/Mesh.hpp	17
NativeTextureView	RHI	Source/Engine/RHI/	151

		RHI.hpp	
PageSlot	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	188
PostFxConstants	RHI	Source/Engine/RHI/RenderPasses.hpp	89
PostFxFlags	RHI	Source/Engine/RHI/PostProcess.hpp	10
PostFxPass	RHI	Source/Engine/RHI/RenderPasses.hpp	145
PostFxSettings	RHI	Source/Engine/RHI/PostProcess.hpp	27
RenderCommandRecord	RHI	Source/Engine/RHI/RenderGraph.hpp	52
RenderFlags	RHI	Source/Engine/RHI/RenderItem.hpp	17
RenderGraph	RHI	Source/Engine/RHI/RenderGraph.hpp	119
RenderGraphContext	RHI	Source/Engine/RHI/RenderGraph.hpp	69
RenderGraphPass	RHI	Source/Engine/RHI/RenderGraph.hpp	106
RenderItem	RHI	Source/Engine/RHI/RenderItem.hpp	59
RenderPassDesc	RHI	Source/Engine/RHI/RenderGraph.hpp	94
RenderPassKind	RHI	Source/Engine/RHI/RenderGraph.hpp	26
RenderResourceKind	RHI	Source/Engine/RHI/RenderGraph.hpp	33
RenderTargetDesc	RHI	Source/Engine/RHI/RHI.hpp	134
RenderTargetHandle	RHI	Source/Engine/RHI/RHI.hpp	140
RHI	RHI	Source/Engine/RHI/RHI.hpp	228
SdfLightingFrame	RHI	Source/Engine/RHI/RenderItem.hpp	279
SdfLightingOccluder	RHI	Source/Engine/RHI/RenderItem.hpp	271
SdfShadowConstant	RHI	Source/Engine/RHI/	37

s		RenderPasses.hpp	
SdfShadowDesc	RHI	Source/Engine/RHI/RenderPasses.hpp	18
SdfShadowPass	RHI	Source/Engine/RHI/RenderPasses.hpp	58
SequenceFooter	RHI	Source/Engine/RHI/FrameExporter.hpp	42
SequenceHeader	RHI	Source/Engine/RHI/FrameExporter.hpp	31
ShaderPipelineDesc	RHI	Source/Engine/RHI/RHI.hpp	111
ShaderPipelineDomain	RHI	Source/Engine/RHI/RHI.hpp	105
ShaderPipelineHandle	RHI	Source/Engine/RHI/RHI.hpp	122
SkinnedVertex	RHI	Source/Engine/RHI/Mesh.hpp	30
Slot	RHI	Source/Engine/RHI/CameraStack.hpp	96
TextureArrayDesc	RHI	Source/Engine/RHI/RHI.hpp	90
TextureArrayHandle	RHI	Source/Engine/RHI/RHI.hpp	204
TintEntry	RHI	Source/Engine/RHI/ViewportPresenter.hpp	81
TransientRing	RHI	Source/Engine/RHI/RingBuffer.hpp	15
UiBackendInfo	RHI	Source/Engine/RHI/RHI.hpp	159
ViewportMode	RHI	Source/Engine/RHI/ViewportModes.hpp	15
ViewportModeDescriptor	RHI	Source/Engine/RHI/ViewportModes.hpp	46
ViewportModeLegend	RHI	Source/Engine/RHI/ViewportModes.hpp	37
ViewportPresenter	RHI	Source/Engine/RHI/ViewportPresenter.hpp	29
ViewportPresenterStats	RHI	Source/Engine/RHI/ViewportPresenter.hpp	17

VirtualFileSystem	RHI	Source/Engine/RHI/FrameExporter.hpp	15
VirtualFileSystem	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	16
VirtualTextureAddress	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	68
VirtualTextureCamera	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	59
VirtualTextureDesc	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	33
VirtualTextureHandle	RHI	Source/Engine/RHI/RenderItem.hpp	248
VirtualTexturePageLocation	RHI	Source/Engine/RHI/VirtualTextureGpu.hpp	11
VirtualTextureResourceDesc	RHI	Source/Engine/RHI/RHI.hpp	216
VirtualTextureRhiTarget	RHI	Source/Engine/RHI/VirtualTextureGpu.hpp	118
VirtualTextureSample	RHI	Source/Engine/RHI/VirtualTextureGpu.hpp	58
VirtualTextureStats	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	45
VirtualTextureSystem	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	111
VirtualTextureTableUpdate	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	101
VirtualTextureTileId	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	92
VirtualTextureTileUpload	RHI	Source/Engine/RHI/VirtualTextureSystem.hpp	77
VisibleSprite	RHI	Source/Engine/RHI/ViewportPresenter.hpp	87

Engine/Script (62)

Символ	Вид	Заголовок	Строка
AotBuilder	Script	Source/Engine/Script/AotBuilder.hpp	71
AotBuildOptions	Script	Source/Engine/Script/AotBuilder.hpp	43
AotBuildResult	Script	Source/Engine/Script/AotBuilder.hpp	60
AotBuildState	Script	Source/Engine/Script/AotBuilder.hpp	19
AotGraphTranspileOptions	Script	Source/Engine/Script/AotGraphCompiler.hpp	13
AotHost	Script	Source/Engine/Script/AotHost.hpp	32
AotHost	Script	Source/Engine/Script/ScriptSystem.hpp	21
AotModule	Script	Source/Engine/Script/AotModule.hpp	22
AotProgramRef	Script	Source/Engine/Script/AotHost.hpp	22
AotTranspileOptions	Script	Source/Engine/Script/AotTranspiler.hpp	12
BoundProgram	Script	Source/Engine/Script/EcsBridge.hpp	101
Code	Script	Source/Engine/Script/Vm.hpp	25
CompiledDebug	Script	Source/Engine/Script/Bytecode.hpp	452
CompiledProgram	Script	Source/Engine/Script/Bytecode.hpp	467
ComponentAccessor	Script	Source/Engine/Script/EcsBridge.hpp	17
ComponentBinding	Script	Source/Engine/Script/Bytecode.hpp	396
Context	Script	Source/Engine/Script/AotAbi.hpp	98
DebugField	Script	Source/Engine/Script/	123

		ScriptDebug.hpp	
DebugFrame	Script	Source/Engine/ Script/ ScriptDebug.hpp	103
DebugSectionHeader	Script	Source/Engine/ Script/Bytecode.hpp	145
DebugSpan	Script	Source/Engine/ Script/Bytecode.hpp	123
DebugSpanFlags	Script	Source/Engine/ Script/Bytecode.hpp	114
DebugVariable	Script	Source/Engine/ Script/Bytecode.hpp	136
EcsBridge	Script	Source/Engine/ Script/EcsBridge.hpp	26
FieldBinding	Script	Source/Engine/ Script/Bytecode.hpp	406
FunctionEntry	Script	Source/Engine/ Script/Bytecode.hpp	419
HostApi	Script	Source/Engine/ Script/AotAbi.hpp	54
Instruction	Script	Source/Engine/ Script/Bytecode.hpp	376
LogicCompareOp	Script	Source/Engine/ Script/ GraphCompiler.hpp	22
LogicCompileOptions	Script	Source/Engine/ Script/ GraphCompiler.hpp	669
LogicCompileResult	Script	Source/Engine/ Script/ GraphCompiler.hpp	676
LogicFieldBinding	Script	Source/Engine/ Script/ GraphCompiler.hpp	90
LogicGraph	Script	Source/Engine/ Script/ GraphCompiler.hpp	319
LogicLink	Script	Source/Engine/ Script/ GraphCompiler.hpp	308
LogicMathOp	Script	Source/Engine/ Script/ GraphCompiler.hpp	56

LogicNode	Script	Source/Engine/Script/GraphCompiler.hpp	290
LogicNodeKind	Script	Source/Engine/Script/GraphCompiler.hpp	111
Module	Script	Source/Engine/Script/AotAbi.hpp	293
NativeId	Script	Source/Engine/Script/Bytecode.hpp	18
OpCode	Script	Source/Engine/Script/Bytecode.hpp	194
PendingField	Script	Source/Engine/Script/Bytecode.hpp	533
PinType	Script	Source/Engine/Script/GraphCompiler.hpp	275
ProgramBuilder	Script	Source/Engine/Script/Bytecode.hpp	498
ProgramHeader	Script	Source/Engine/Script/Bytecode.hpp	429
ResolvedComponent	Script	Source/Engine/Script/EcsBridge.hpp	93
ScriptComponent	Script	Source/Engine/Script/ScriptComponent.hpp	16
ScriptEvent	Script	Source/Engine/Script/Bytecode.hpp	169
ScriptRuntime	Script	Source/Engine/Script/ScriptSystem.hpp	20
ScriptRuntime	Script	Source/Engine/Script/ScriptSystem.hpp	24
ScriptVm	Script	Source/Engine/Script/Vm.hpp	101
ScriptVm	Script	Source/Engine/Script/Vm.hpp	215
Slot	Script	Source/Engine/Script/AotHost.hpp	94
Slot	Script	Source/Engine/Script/ScriptSystem.hpp	112

SourceLocation	Script	Source/Engine/ Script/ ScriptDebug.hpp	17
TextCompileOptions	Script	Source/Engine/ Script/TextScript.hpp	14
Vector2D	Script	Source/Engine/ Script/AotAbi.hpp	28
VmContext	Script	Source/Engine/ Script/Vm.hpp	87
VmDebugState	Script	Source/Engine/ Script/Vm.hpp	140
VmFault	Script	Source/Engine/ Script/Vm.hpp	23
VmHalt	Script	Source/Engine/ Script/Vm.hpp	113
VmPauseReason	Script	Source/Engine/ Script/Vm.hpp	104
VmStopInfo	Script	Source/Engine/ Script/Vm.hpp	120

Engine/Text (67)

Символ	Вид	Заголовок	Строка
AssetSlot	Text	Source/Engine/Text/ TextStore.hpp	218
ChainedShaper	Text	Source/Engine/Text/ TextShaper.hpp	133
CodepointShaper	Text	Source/Engine/Text/ TextShaper.hpp	99
Entry	Text	Source/Engine/Text/ TextShaper.hpp	247
FontCandidate	Text	Source/Engine/Text/ TextFontStack.hpp	24
FontSlot	Text	Source/Engine/Text/ TextFontStack.hpp	59
FormatEntry	Text	Source/Engine/Text/ TextService.hpp	583
FormattedLayoutEntry	Text	Source/Engine/Text/ TextService.hpp	566
FormattedLayoutKey	Text	Source/Engine/Text/ TextService.hpp	556
HarfBuzzShaper	Text	Source/Engine/Text/	110

		TextShaper.hpp	
Impl	Text	Source/Engine/Text/TextShaper.hpp	128
ITextShaper	Text	Source/Engine/Text/TextShaper.hpp	89
LayoutEntry	Text	Source/Engine/Text/TextService.hpp	548
LayoutKey	Text	Source/Engine/Text/TextService.hpp	528
MappedTextAsset	Text	Source/Engine/Text/TextAssetFile.hpp	17
MarkupStackEntry	Text	Source/Engine/Text/TextMarkup.hpp	71
MarkupVariantEntry	Text	Source/Engine/Text/TextService.hpp	574
ScriptBackend	Text	Source/Engine/Text/TextShapeSystem.hpp	111
ScriptShaper	Text	Source/Engine/Text/TextShaper.hpp	207
ShapedGlyph	Text	Source/Engine/Text/TextShaper.hpp	56
ShapedGlyphHit	Text	Source/Engine/Text/TextShapedLayout.hpp	153
ShapedLayoutResult	Text	Source/Engine/Text/TextShapedLayout.hpp	26
ShapedLine	Text	Source/Engine/Text/TextShapedLayout.hpp	18
StyledAsset	Text	Source/Engine/Text/TextService.hpp	595
TextAction	Text	Source/Engine/Text/TextStyle.hpp	61
TextActionKind	Text	Source/Engine/Text/TextStyle.hpp	51
TextAnimationParams	Text	Source/Engine/Text/TextRender.hpp	29
TextAssetBlock	Text	Source/Engine/Text/TextAssetFormat.hpp	34
TextAssetHeader	Text	Source/Engine/Text/	20

		TextAssetFormat.hpp	
TextAssetRef	Text	Source/Engine/Text/ TextAsset.hpp	14
TextAssetStatus	Text	Source/Engine/Text/ TextAssetFormat.hpp	63
TextAssetView	Text	Source/Engine/Text/ TextAssetFormat.hpp	42
TextBlock	Text	Source/Engine/Text/ TextAsset.hpp	37
TextDirection	Text	Source/Engine/Text/ TextShaper.hpp	21
TextFontStack	Text	Source/Engine/Text/ TextFontStack.hpp	54
TextFormatArg	Text	Source/Engine/Text/ TextFormat.hpp	45
TextFormatStatus	Text	Source/Engine/Text/ TextFormat.hpp	33
TextGradientParams	Text	Source/Engine/Text/ TextRender.hpp	22
TextInlineHit	Text	Source/Engine/Text/ TextLayout.hpp	203
TextInlineKind	Text	Source/Engine/Text/ TextStyle.hpp	73
TextInlineObject	Text	Source/Engine/Text/ TextStyle.hpp	83
TextInlineSprite	Text	Source/Engine/Text/ TextStyle.hpp	99
TextLayoutParams	Text	Source/Engine/Text/ TextLayout.hpp	53
TextLayoutResult	Text	Source/Engine/Text/ TextLayout.hpp	71
TextLine	Text	Source/Engine/Text/ TextLayout.hpp	21
TextMarkupStatus	Text	Source/Engine/Text/ TextMarkup.hpp	16
TextRenderEffects	Text	Source/Engine/Text/ TextRender.hpp	44
TextRenderParams	Text	Source/Engine/Text/ TextRender.hpp	57
TextRenderStats	Text	Source/Engine/Text/ TextRender.hpp	76

TextRenderSystemStats	Text	Source/Engine/Text/TextRenderSystem.hpp	19
TextRun	Text	Source/Engine/Text/TextShaper.hpp	47
TextRuntime	Text	Source/Engine/Text/TextRuntime.hpp	17
TextScript	Text	Source/Engine/Text/TextShaper.hpp	30
TextService	Text	Source/Engine/Text/TextService.hpp	18
TextShapeCache	Text	Source/Engine/Text/TextShapeCache.hpp	59
TextShapeHandle	Text	Source/Engine/Text/TextShapeCache.hpp	19
TextShapeKey	Text	Source/Engine/Text/TextShapeCache.hpp	32
TextShapeParams	Text	Source/Engine/Text/TextShaper.hpp	71
TextShapeResult	Text	Source/Engine/Text/TextShaper.hpp	82
TextShapeSystemStats	Text	Source/Engine/Text/TextShapeSystem.hpp	16
TextShapeView	Text	Source/Engine/Text/TextShapeCache.hpp	50
TextSpan	Text	Source/Engine/Text/TextStyle.hpp	40
TextStore	Text	Source/Engine/Text/TextStore.hpp	19
TextStyle	Text	Source/Engine/Text/TextStyle.hpp	27
TextStyledBlock	Text	Source/Engine/Text/TextMarkup.hpp	30
TextStyleFlags	Text	Source/Engine/Text/TextStyle.hpp	12
VerticalFormEntry	Text	Source/Engine/Text/TextVertical.hpp	16

Engine/UI (101)

Символ	Вид	Заголовок	Строка

AccessibilityEditorSettings	UI	Source/Engine/UI/Accessibility.hpp	250
AccessibilityFlags	UI	Source/Engine/UI/Accessibility.hpp	208
AccessibilityKernel	UI	Source/Engine/UI/Accessibility.hpp	304
AccessibilityOverlay	UI	Source/Engine/UI/AccessibilityOverlay.hpp	30
AccessibilityPresentationSettings	UI	Source/Engine/UI/Accessibility.hpp	233
AccessibilityProfile	UI	Source/Engine/UI/Accessibility.hpp	272
AccessibilitySettings	UI	Source/Engine/UI/Accessibility.hpp	217
AudioSystem	UI	Source/Engine/UI/Accessibility.hpp	26
AvSpeechTts	UI	Source/Engine/UI/Tts.hpp	60
BuildStatus	UI	Source/Engine/UI/PinkUI.hpp	1440
CardBoardComponent	UI	Source/Engine/UI/CardBoard.hpp	49
CardBoardSystem	UI	Source/Engine/UI/CardBoard.hpp	74
ColorMatrix3	UI	Source/Engine/UI/Accessibility.hpp	42
ColorVisionDeficiency	UI	Source/Engine/UI/Accessibility.hpp	34
Dropdown	UI	Source/Engine/UI/PinkUI.hpp	534
FlashGuard	UI	Source/Engine/UI/Accessibility.hpp	59
FocusDirection	UI	Source/Engine/UI/PinkUI.hpp	211
FocusPolicy	UI	Source/Engine/UI/PinkUI.hpp	187
FontAtlas	UI	Source/Engine/UI/FontAtlas.hpp	101
FontAtlasHeader	UI	Source/Engine/UI/FontAtlas.hpp	45
FontAtlasSet	UI	Source/Engine/UI/	167

		FontAtlas.hpp	
FontBuildOptions	UI	Source/Engine/UI/FontAtlas.hpp	25
FontBuildOptions	UI	Source/Engine/UI/FontBuilder.hpp	15
FontCoverage	UI	Source/Engine/UI/FontCoverage.hpp	21
FontFamilyFile	UI	Source/Engine/UI/FontAtlas.hpp	74
FontFamilyManifest	UI	Source/Engine/UI/FontFamilyAsset.hpp	17
FontPixelFormat	UI	Source/Engine/UI/FontAtlas.hpp	37
FontStyle	UI	Source/Engine/UI/FontAtlas.hpp	65
FrameOutput	UI	Source/Engine/UI/PinkUI.hpp	1418
GlyphRecord	UI	Source/Engine/UI/FontAtlas.hpp	80
GridView	UI	Source/Engine/UI/PinkUI.hpp	424
HyperlinkRange	UI	Source/Engine/UI/Hypertext.hpp	51
HypertextComponent	UI	Source/Engine/UI/Hypertext.hpp	32
HypertextFlags	UI	Source/Engine/UI/Hypertext.hpp	21
HypertextPool	UI	Source/Engine/UI/Hypertext.hpp	74
HypertextView	UI	Source/Engine/UI/Hypertext.hpp	67
ITextShaper	UI	Source/Engine/UI/PinkUI.hpp	33
LayoutCache	UI	Source/Engine/UI/PinkUI.hpp	736
LazyFontFamily	UI	Source/Engine/UI/FontFamilyAsset.hpp	101
ListView	UI	Source/Engine/UI/PinkUI.hpp	560
LocalizationTable	UI	Source/Engine/UI/Accessibility.hpp	24

LocalizationTable	UI	Source/Engine/UI/Hypertext.hpp	16
LocalizationTable	UI	Source/Engine/UI/PinkUI.hpp	39
NarrativeArchetype	UI	Source/Engine/UI/NarrativeTree.hpp	22
NarrativeEvent	UI	Source/Engine/UI/Accessibility.hpp	83
NarrativeGroup	UI	Source/Engine/UI/NarrativeTree.hpp	36
NarrativeLocale	UI	Source/Engine/UI/NarrativeTree.hpp	16
NarrativeLog	UI	Source/Engine/UI/Accessibility.hpp	99
NarrativeNode	UI	Source/Engine/UI/NarrativeTree.hpp	46
NarrativeTree	UI	Source/Engine/UI/NarrativeTree.hpp	54
OledShiftState	UI	Source/Engine/UI/PinkUI.hpp	463
PinkSdfDraw	UI	Source/Engine/UI/PinkSdf.hpp	74
PinkSdfFlags	UI	Source/Engine/UI/PinkSdf.hpp	24
PinkSdfGpuPrimitive	UI	Source/Engine/UI/PinkSdf.hpp	51
PinkSdfPrimitive	UI	Source/Engine/UI/PinkSdf.hpp	33
PinkSdfShape	UI	Source/Engine/UI/PinkSdf.hpp	15
RadialProgress	UI	Source/Engine/UI/PinkUI.hpp	584
RetainedFont	UI	Source/Engine/UI/FontAtlas.hpp	293
SapiTts	UI	Source/Engine/UI/Tts.hpp	30
ScrollView	UI	Source/Engine/UI/PinkUI.hpp	491
SlotComponent	UI	Source/Engine/UI/CardBoard.hpp	27
SlotFlags	UI	Source/Engine/UI/	15

		CardBoard.hpp	
SlotLayout	UI	Source/Engine/UI/ CardBoard.hpp	65
SoundCueCategory	UI	Source/Engine/UI/ SoundCues.hpp	26
SoundCueDef	UI	Source/Engine/UI/ SoundCues.hpp	62
SoundCueRegistry	UI	Source/Engine/UI/ SoundCues.hpp	71
SoundTrigger	UI	Source/Engine/UI/ Accessibility.hpp	128
StateTrack	UI	Source/Engine/UI/ Accessibility.hpp	411
SubtitleLine	UI	Source/Engine/UI/ Accessibility.hpp	139
SubtitleQueue	UI	Source/Engine/UI/ Accessibility.hpp	163
SystemContext	UI	Source/Engine/UI/ PinkUI.hpp	28
TabBar	UI	Source/Engine/UI/ PinkUI.hpp	515
TextAlign	UI	Source/Engine/UI/ PinkUI.hpp	222
TextShapeParams	UI	Source/Engine/UI/ PinkUI.hpp	34
TextVerticalAlign	UI	Source/Engine/UI/ PinkUI.hpp	229
TimedSoundCue	UI	Source/Engine/UI/ Accessibility.hpp	152
TooltipPlacer	UI	Source/Engine/UI/ PinkUI.hpp	595
TtsBackend	UI	Source/Engine/UI/ Accessibility.hpp	27
TtsBackend	UI	Source/Engine/UI/ Tts.hpp	19
TtsPriority	UI	Source/Engine/UI/ Tts.hpp	11
UiPalette	UI	Source/Engine/UI/ UiTheme.hpp	41
UiPaletteSlot	UI	Source/Engine/UI/ UiTheme.hpp	18

UiThemeAsset	UI	Source/Engine/UI/UiTheme.hpp	46
VirtualFileSystem	UI	Source/Engine/UI/FontAtlas.hpp	19
VoiceCaptionEvent	UI	Source/Engine/UI/Accessibility.hpp	193
VoiceCaptionProvider	UI	Source/Engine/UI/Accessibility.hpp	201
WidgetCanvas	UI	Source/Engine/UI/Accessibility.hpp	30
WidgetCanvas	UI	Source/Engine/UI/PinkUI.hpp	290
WidgetCanvas	UI	Source/Engine/UI/PinkUI.hpp	483
WidgetCanvas	UI	Source/Engine/UI/PinkUI.hpp	607
WidgetCanvasSystem	UI	Source/Engine/UI/PinkUI.hpp	1453
WidgetComponent	UI	Source/Engine/UI/PinkUI.hpp	251
WidgetComponentFlags	UI	Source/Engine/UI/PinkUI.hpp	263
WidgetDesc	UI	Source/Engine/UI/PinkUI.hpp	309
WidgetFlags	UI	Source/Engine/UI/PinkUI.hpp	145
WidgetHandle	UI	Source/Engine/UI/PinkUI.hpp	80
WidgetHyperlink	UI	Source/Engine/UI/PinkUI.hpp	300
WidgetKind	UI	Source/Engine/UI/PinkUI.hpp	99
WidgetLayoutAsset	UI	Source/Engine/UI/WidgetLayoutAsset.hpp	15
WidgetPaintContext	UI	Source/Engine/UI/PinkUI.hpp	291
WidgetPaintContext	UI	Source/Engine/UI/PinkUI.hpp	391

Engine/Video (20)

Символ	Вид	Заголовок	Строка
AssetDatabase	Video	Source/Engine/Video/VideoStream.hpp	23
ClusterScan	Video	Source/Engine/Video/WebmDemuxer.hpp	108
ContainerKind	Video	Source/Engine/Video/WebmDemuxer.hpp	150
DecodeStatus	Video	Source/Engine/Video/VideoTypes.hpp	29
DemuxStatus	Video	Source/Engine/Video/WebmDemuxer.hpp	52
Element	Video	Source/Engine/Video/WebmDemuxer.hpp	85
FrameBuffer	Video	Source/Engine/Video/VideoPlayer.hpp	118
FrameSlot	Video	Source/Engine/Video/VideoPlayer.hpp	125
MappedFile	Video	Source/Engine/Video/VideoPlayer.hpp	22
RHI	Video	Source/Engine/Video/VideoPlayer.hpp	27
VideoPlayer	Video	Source/Engine/Video/VideoPlayer.hpp	33
VideoStreamComponent	Video	Source/Engine/Video/VideoStream.hpp	31
VideoStreamFlags	Video	Source/Engine/Video/VideoTypes.hpp	20
VideoSystem	Video	Source/Engine/Video/VideoStream.hpp	49
VirtualFileSystem	Video	Source/Engine/Video/	20

		VideoStream.hpp	
Vp9Backend	Video	Source/Engine/ Video/ Vp9Backend.hpp	30
WebmDemuxer	Video	Source/Engine/ Video/ WebmDemuxer.hpp	60
WebmFrame	Video	Source/Engine/ Video/ WebmDemuxer.hpp	38
WebmTrackInfo	Video	Source/Engine/ Video/ WebmDemuxer.hpp	14
YuvFrameView	Video	Source/Engine/ Video/ Vp9Backend.hpp	12

Engine/Weather (12)

Символ	Вид	Заголовок	Строка
ClothSystem	Weather	Source/Engine/ Weather/Weather.hpp	13
WeatherAtmosphere Pass	Weather	Source/Engine/ Weather/ WeatherPreconciliated .hpp	128
WeatherContextCom ponent	Weather	Source/Engine/ Weather/Weather.hpp	32
WeatherDesc	Weather	Source/Engine/ Weather/Weather.hpp	91
WeatherFlags	Weather	Source/Engine/ Weather/Weather.hpp	19
WeatherGpuConstan ts	Weather	Source/Engine/ Weather/ WeatherPreconciliated .hpp	46
WeatherGpuResourc es	Weather	Source/Engine/ Weather/ WeatherPreconciliated .hpp	18
WeatherGpuStats	Weather	Source/Engine/ Weather/ WeatherPreconciliated .hpp	68

WeatherPreconciliatedDesc	Weather	Source/Engine/Weather/WeatherPreconciliated.hpp	32
WeatherPreconciliatedSystem	Weather	Source/Engine/Weather/WeatherPreconciliated.hpp	78
WeatherSystem	Weather	Source/Engine/Weather/Weather.hpp	99
WeatherWindField	Weather	Source/Engine/Weather/Weather.hpp	75

40. Приложение B - макросы и платформенные define

Макрос	Заголовок	Строка
FBY_AOT_EXPORT	Source/Engine/Script/AotAbi.hpp	310
FBY_AOT_EXPORT	Source/Engine/Script/AotAbi.hpp	312
FBY_ASSERT	Source/Engine/Core/Assert.hpp	13
FBY_ASSERT	Source/Engine/Core/Assert.hpp	31
FBY_ASSERT_MSG	Source/Engine/Core/Assert.hpp	22
FBY_ASSERT_MSG	Source/Engine/Core/Assert.hpp	32
FBY_DEFINE_PLUGIN	Source/Engine/Core/Plugin.hpp	56
FBY_DETAIL_CONCAT	Source/Engine/Core/Definitions.hpp	261
FBY_DETAIL_CONCAT_INNER	Source/Engine/Core/Definitions.hpp	260
FBY_FIELD	Source/Engine/Core/Mirror.hpp	279
FBY_FLUID_HAS_MDSPAN	Source/Engine/Physics/Fluid.hpp	16
FBY_FLUID_HAS_MDSPAN	Source/Engine/Physics/Fluid.hpp	18
FBY_HAS_MDSPAN	Source/Engine/ECS/	17

	Skeleton.hpp	
FBY_HAS_MDSPAN	Source/Engine/ECS/ Skeleton.hpp	19
FBY_HAS_STATIC_REFLECTION	Source/Engine/Core/ Mirror.hpp	97
FBY_HAS_STATIC_REFLECTION	Source/Engine/Core/ Mirror.hpp	100
FBY_HAS_STATIC_REFLECTION	Source/Engine/Core/ Mirror.hpp	104
FBY_HAS_STD_FLAT_MAP	Source/Engine/Core/ FlatMap.hpp	17
FBY_HAS_STD_FLAT_MAP	Source/Engine/Core/ FlatMap.hpp	19
FBY_PLATFORM_ABI_VERSION	Source/Plugins/ PlatformService/PlatformAPI.h	11
FBY_PLATFORM_CALL	Source/Plugins/ PlatformService/PlatformAPI.h	15
FBY_PLATFORM_CALL	Source/Plugins/ PlatformService/PlatformAPI.h	18
FBY_PLATFORM_EXPORT	Source/Plugins/ PlatformService/PlatformAPI.h	14
FBY_PLATFORM_EXPORT	Source/Plugins/ PlatformService/PlatformAPI.h	17
FBY_PLUGIN_ABI_VERSION	Source/Engine/Core/ PluginAPI.h	28
FBY_PLUGIN_API_H	Source/Engine/Core/ PluginAPI.h	18
FBY_PLUGIN_COMPONENT	Source/Engine/Core/ Plugin.hpp	53
FBY_PLUGIN_ENTRY	Source/Engine/Core/ PluginAPI.h	115
FBY_PLUGIN_EXPORT	Source/Engine/Core/ PluginAPI.h	31
FBY_PLUGIN_EXPORT	Source/Engine/Core/ PluginAPI.h	33
FBY_PLUGIN_EXPORT	Source/Engine/Core/ PluginAPI.h	35
FBY_PLUGIN_INFO_SYMBOL	Source/Engine/Core/ PluginAPI.h	107
FBY_PLUGIN_LOAD_SYMBOL	Source/Engine/Core/ PluginAPI.h	108

FBY_PLUGIN_UNLOAD_SYMBOL	Source/Engine/Core/PluginAPI.h	109
FBY_PROJECT_ASSETS	Source/Engine/Assets/ProjectScaffold.hpp	123
FBY_REFLECT_COMPONENT	Source/Engine/Core/Mirror.hpp	286
FBY_REFLECT_COMPONENT_EMPTY	Source/Engine/Core/Mirror.hpp	298
FBY_REGISTER_TYPE	Source/Engine/Core/Definitions.hpp	264
FBY_RELEVANCE_HAS_MDSPAN	Source/Engine/Network/RelevanceGrid.hpp	19
FBY_RELEVANCE_HAS_MDSPAN	Source/Engine/Network/RelevanceGrid.hpp	21
FBY_RESTRICT	Source/Engine/Core/Definitions.hpp	26
FBY_RESTRICT	Source/Engine/Core/Definitions.hpp	28
FBY_SDF_SHAPE_FIELD	Source/Engine/Physics/SdfPrimitives.hpp	172
FBY_TYPE_NAME	Source/Engine/Core/Definitions.hpp	280
FBY_UI_HAS_MDSPAN	Source/Engine/UI/PinkUI.hpp	21
FBY_UI_HAS_MDSPAN	Source/Engine/UI/PinkUI.hpp	23
FBY_UNREACHABLE	Source/Engine/Core/Assert.hpp	45
FBY_VERIFY	Source/Engine/Core/Assert.hpp	36
FBY_VK_DECLARE	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	117
FBY_VK_DEVICE_FUNCTIONS	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	36
FBY_VK_GLOBAL_FUNCTIONS	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	15
FBY_VK_INSTANCE_FUNCTIONS	Source/Engine/RHI/Vulkan/VulkanLoader.hpp	20
FORCEINLINE	Source/Engine/Core/Definitions.hpp	17
FORCEINLINE	Source/Engine/Core/Definitions.hpp	19
FORCEINLINE	Source/Engine/Core/	21

	Definitions.hpp	